Fondements de l'intelligence artificielle

Unité 8

Recherche pour les jeux à adversité

Adversarial search and Games

Redoaune Ezzahir

red.ezzahir@gmail.com

slides from Russell and Norvig course

Exercice 1

- Soit le jeu défini par la matrice en forme stratégique suivante:

J1 \ J2	X	Y
A	(1,2)	(0,4)
B	(0,5)	(3,2)

- Y-t-a-il un équilibre de Nash en stratégies pures ?

- Trouvez l'équilibre en stratégies mixtes et les gains associés

Exercice 2

- Soit le jeu défini par la matrice en forme stratégique suivante:

A\B	X	Y	Z
X	1, -1	-3, 3	-2, 2
Y	0, 0	1, -1	-1, 1
Z	-4, 4	2, -2	-3, 3

- Quelle est la nature de ce jeu? Peut-on simplifier la matrice de ce jeu?

- Déterminez les stratégie prudentes de chaque joueur.

Objectifs

Comment choisir une stratégie gagnante dans un jeu à somme nulle et à information parfaite?

* Un jeu dans lequel les joueurs jouent les uns après les autres, et où tout le monde sait tout, observe tout, et se rappelle de tout.
    * TIC-TAC-TEO (X,O)
    * Dame
    * …
*

Objectifs

* Comprendre et pouvoir appliquer l'algorithme ***minimax***
* Comprendre et pouvoir appliquer l'algorithme d'élagage ***alpha-bêta***
* Discuter le cas où le temps de réflexion est limité ou les configurations dans le jeu sont exponentielles

Jeux: Machine vs Humain

* Jeu de dames: En 1994, Chinook a mis fin aux 40 ans de règne du champion du monde Marion Tinsley.
* Jeu d'échecs: En 1997, Deep Blue a battu le champion du monde Garry Kasparov dans un match de six jeux.
* Othello: les champions humains refusent la compétition contre des ordinateurs, parce que ces derniers sont trop bons!
* Go: AlphaGo bat le champion mondial pour la première fois en 2015/2016!

Arbre de jeu

Pour concevoir un Agent de jeu on définit l'arbre de jeu

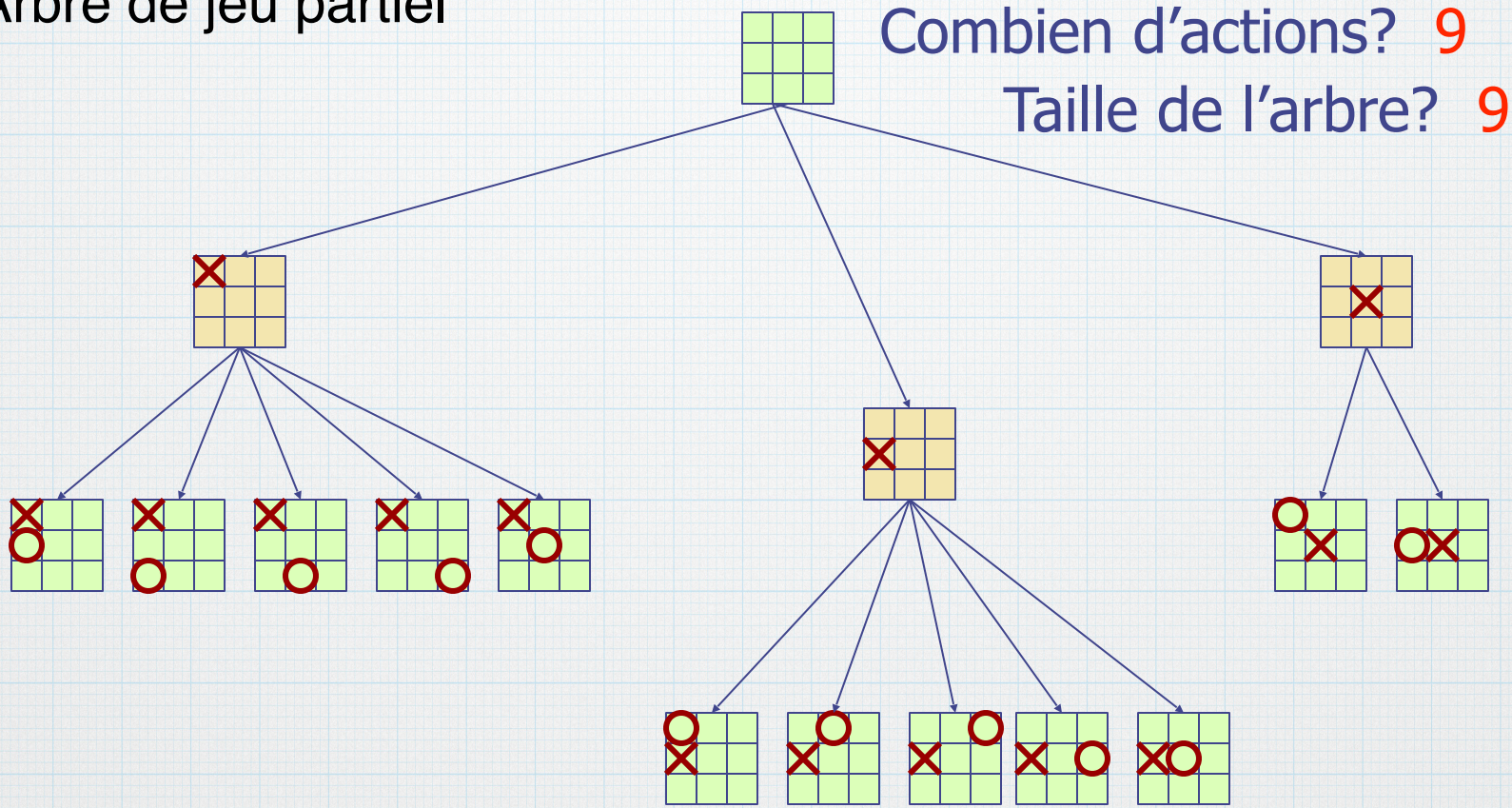
- On représente la suite des coups à jouer par un arbre n-aire
  - Racine : l'état initial du jeu
  - nœud : situation de jeu
  - fils : possibilité de coup joué par l'autre joueur à partir de la nouvelle position de jeu
  - branche : séquence de coups
  - feuille : fin de partie ou fin du nombre de coups à anticiper
  - degré de l'arbre : nombre maximum de possibilité de coups pour un tour de jeu

Tic-Tac-Teo *(jeu du morpion)*

Arbre de jeu partiel

Combien d'actions? 9

Taille de l'arbre? 9!



Un arbre de jeu complet a autant de nœuds que le jeu a de résultats possibles pour chaque coup légal effectué.

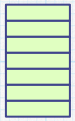
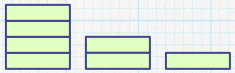
Hypothèses pour ce cours

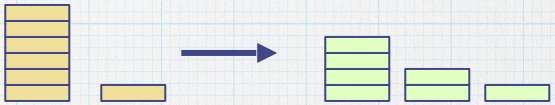
- Dans ce cours, nous aborderons les :
  - jeux à deux adversaires
  - jeux à tour de rôle
  - jeux à somme nulle
  - jeux avec états complètement observés
  - jeux déterministes (sans hasard ou incertitude)

Jeu parfait à deux joueurs

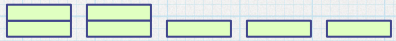
- Noms des joueurs : MAX vs. MIN
- Deux joueurs MAX et MIN tour à tour (avec MAX jouant en premier)
  - Etat de l'espace (State space)
  - Etat initial (Initial state)
  - Fonction successeur (Successor function)
  - Teste d'état terminal (Terminal test)
  - Fonction d'évaluation (Score function): qui indique si un état terminal est une victoire (pour MAX), une défaite ou un match nul
    - La fonction d'évaluation est un nombre statique qui, conformément aux caractéristiques du jeu lui-même, est attribué à chaque nœud.
- Parfaite connaissance des états, pas d'incertitude en fonction successeur

Exemple : le jeu de Grundy

* Etat initial: une pile de 7 pièces 
* État: un ensemble de piles 
* Fonction successeur : Casser une pile de pièces en deux piles inégales

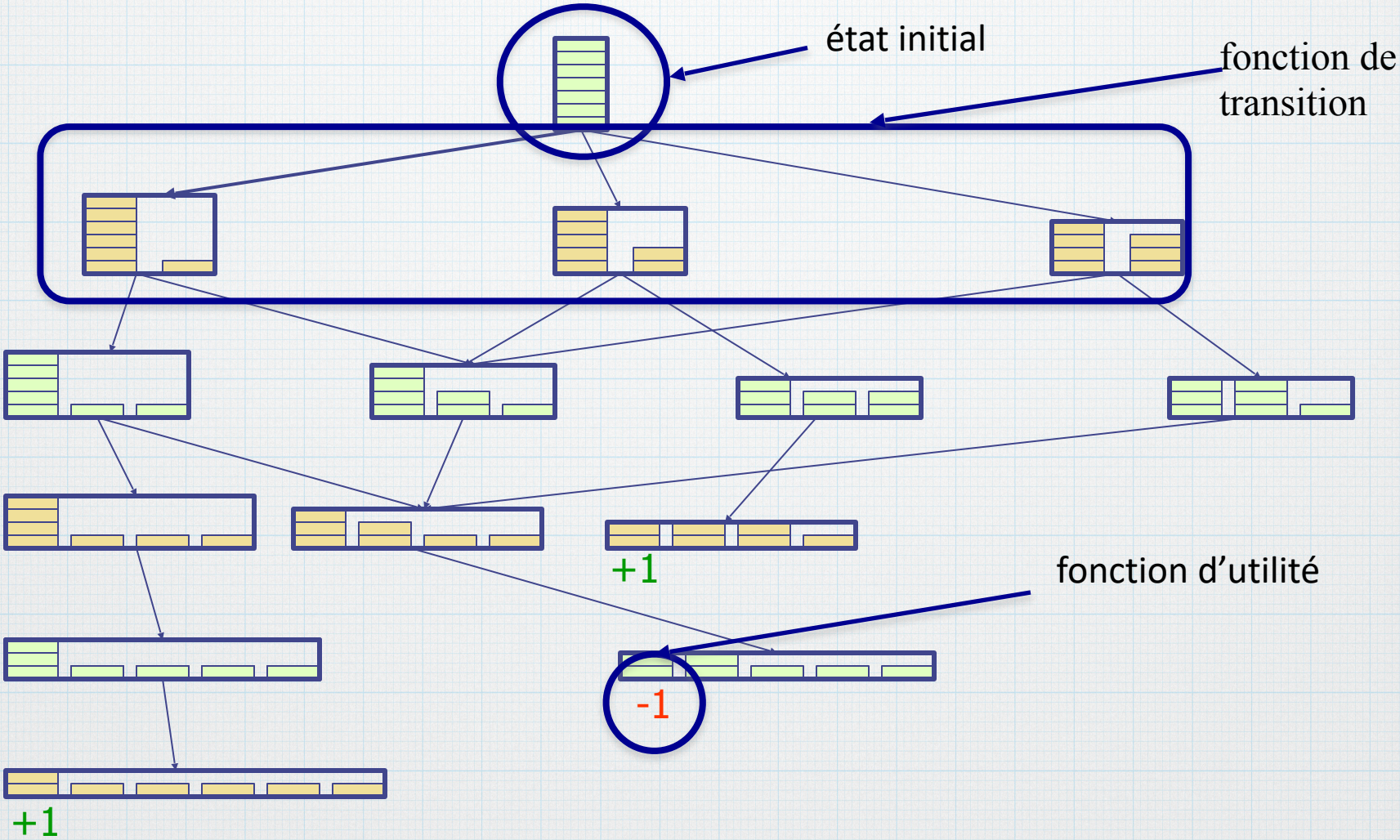


* Etat terminal : Toutes les piles contiennent une ou deux pièces



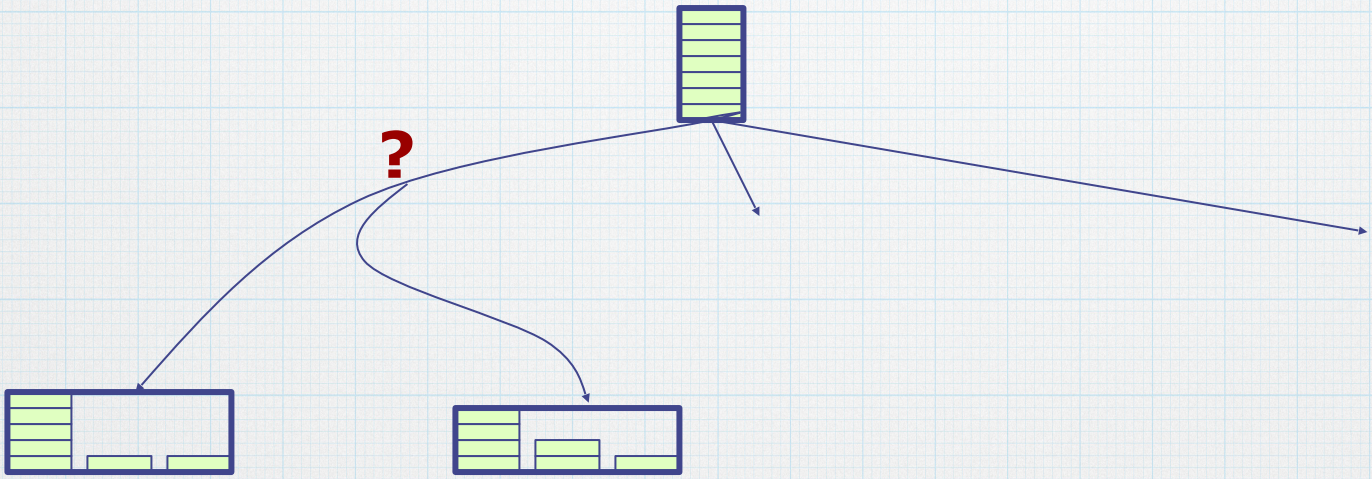
* Fonction score : l'état terminal est un gain (+1) pour MAX s'il a été généré par MAX, et une perte sinon (-1)

Game Graph/Tree



Incertitude dans le modèle d'action

- Où se situe l'incertitude dans les jeux compétitifs?
  - Le fait que l'on ne peut jamais reconnaître avec certitude absolue la prochaine action de l'adversaire.

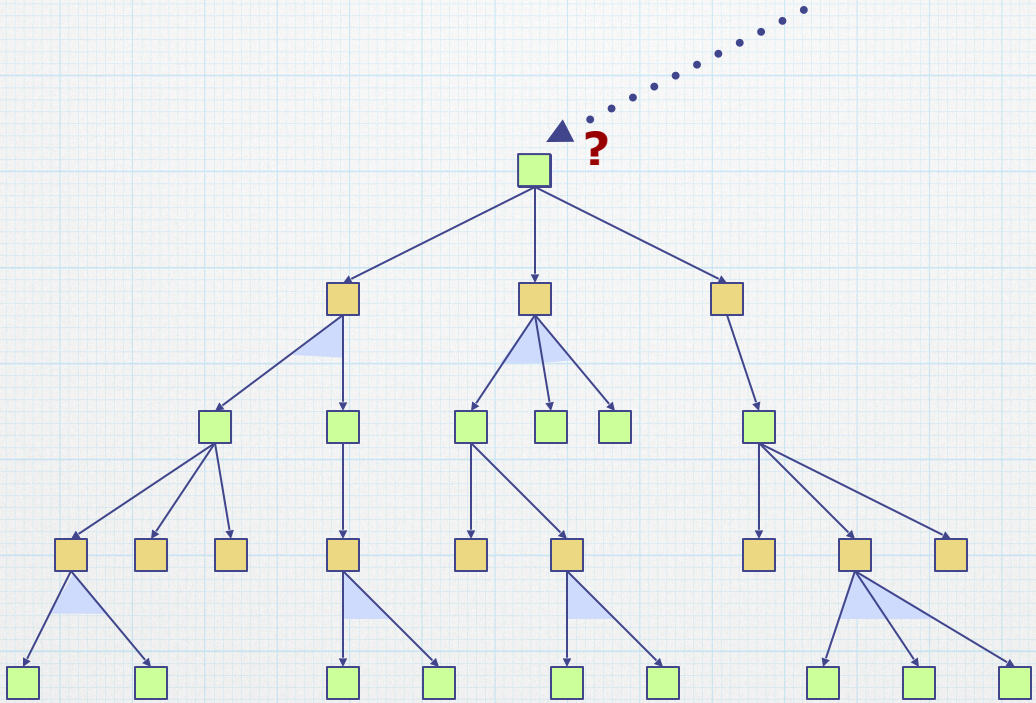


Prendre la meilleure décision en tenant compte du résultat le plus défavorable de chaque action

Arbre ET/OU (AND/OR Tree)

MAX

MIN



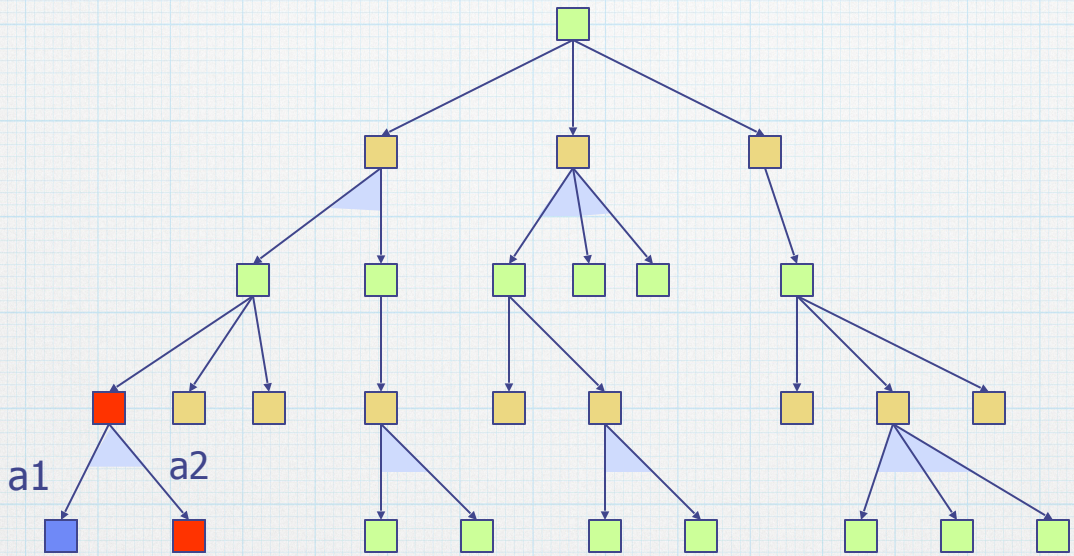
Arbre ET/OU (AND/OR Tree)

MAX

MIN

Défaite

Gain



a1, a2 : actions possible de MIN

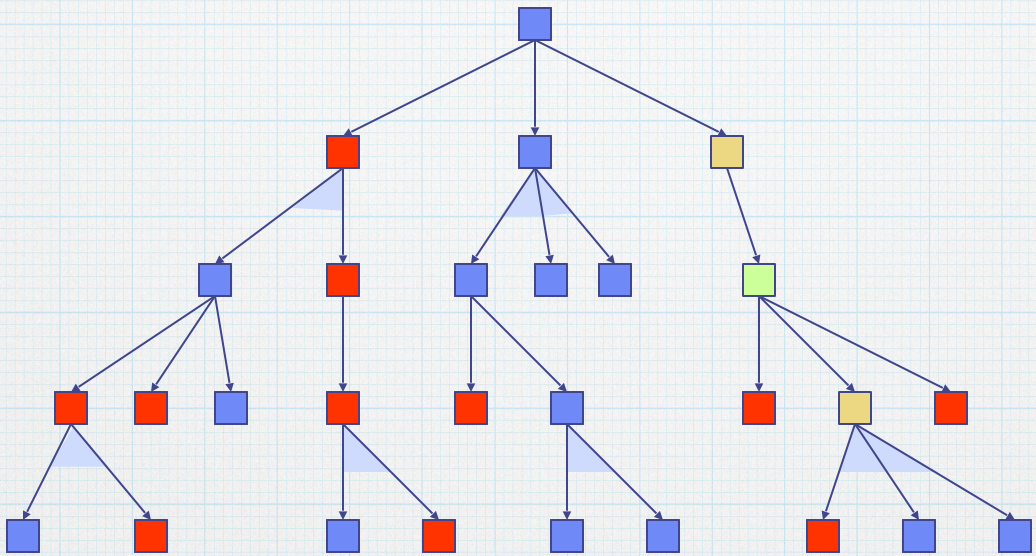
Étiquetage de l'arbre ET/OU

MAX

MIN

Défaite

Gain



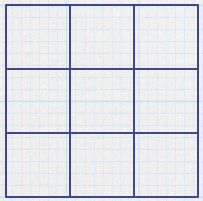
Fonction d'évaluation d'un Etat

Exemple

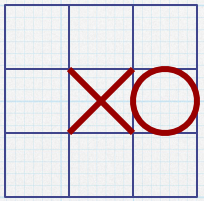
* e(s) = +∞ ou +1 si s est une victoire pour MAX
* e(s) = -∞ ou -1 si s est une victoire pour MIN
* e(s) = une mesure du degré de « favorable » de choix de s pour MAX
  - > 0 si s est considéré comme favorable à MAX
  - < 0 sinon

Exemple : morpion (Tic-Tac-Toe)

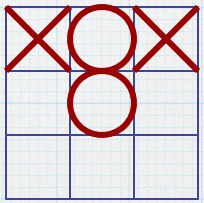
e(s) = nombre de lignes, de colonnes et de diagonales ouvertes pour MAX
- nombre de lignes, de colonnes et de diagonales ouvertes pour MIN



8-8 = 0



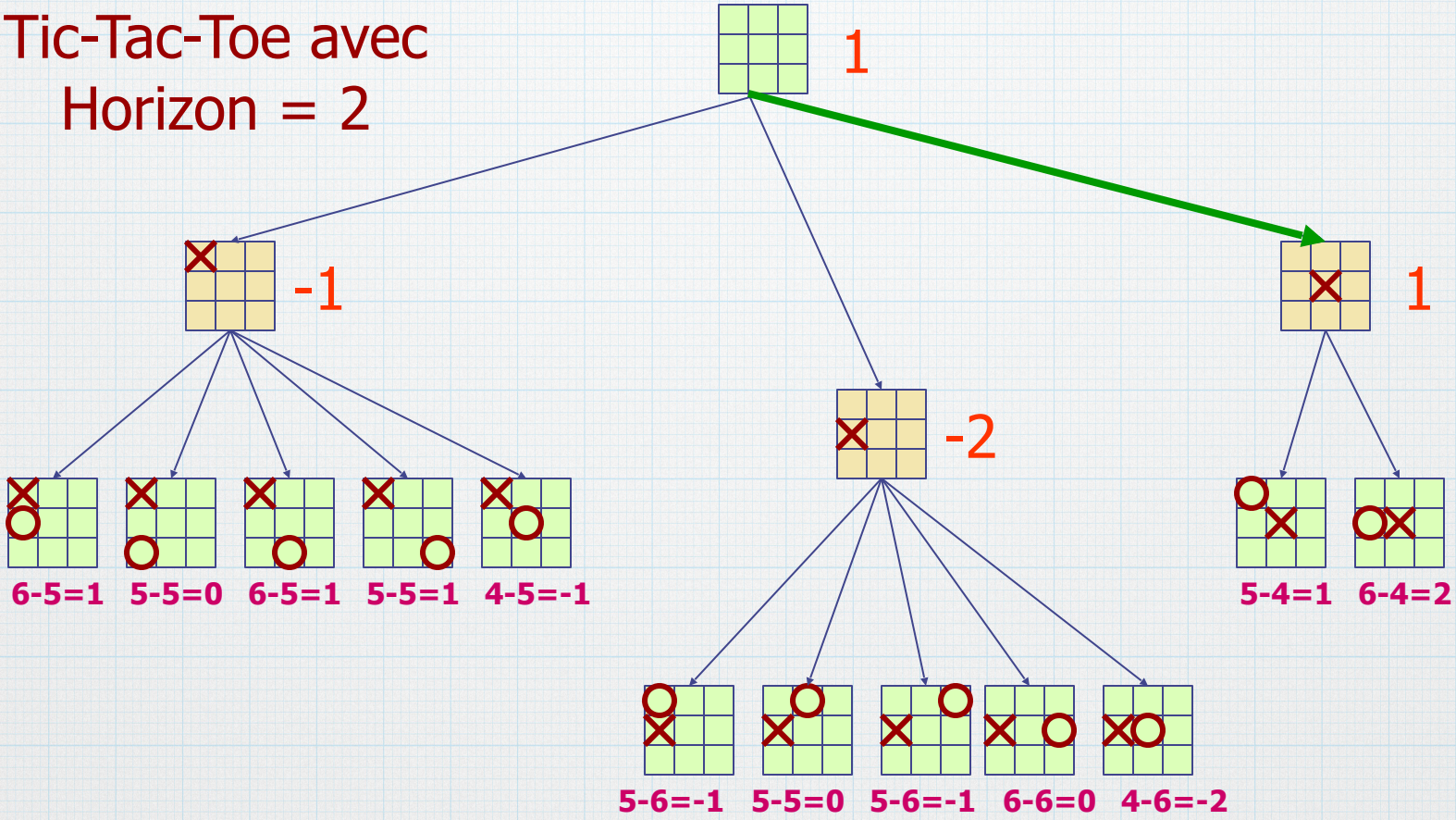
6-4 = 2



3-3 = 0

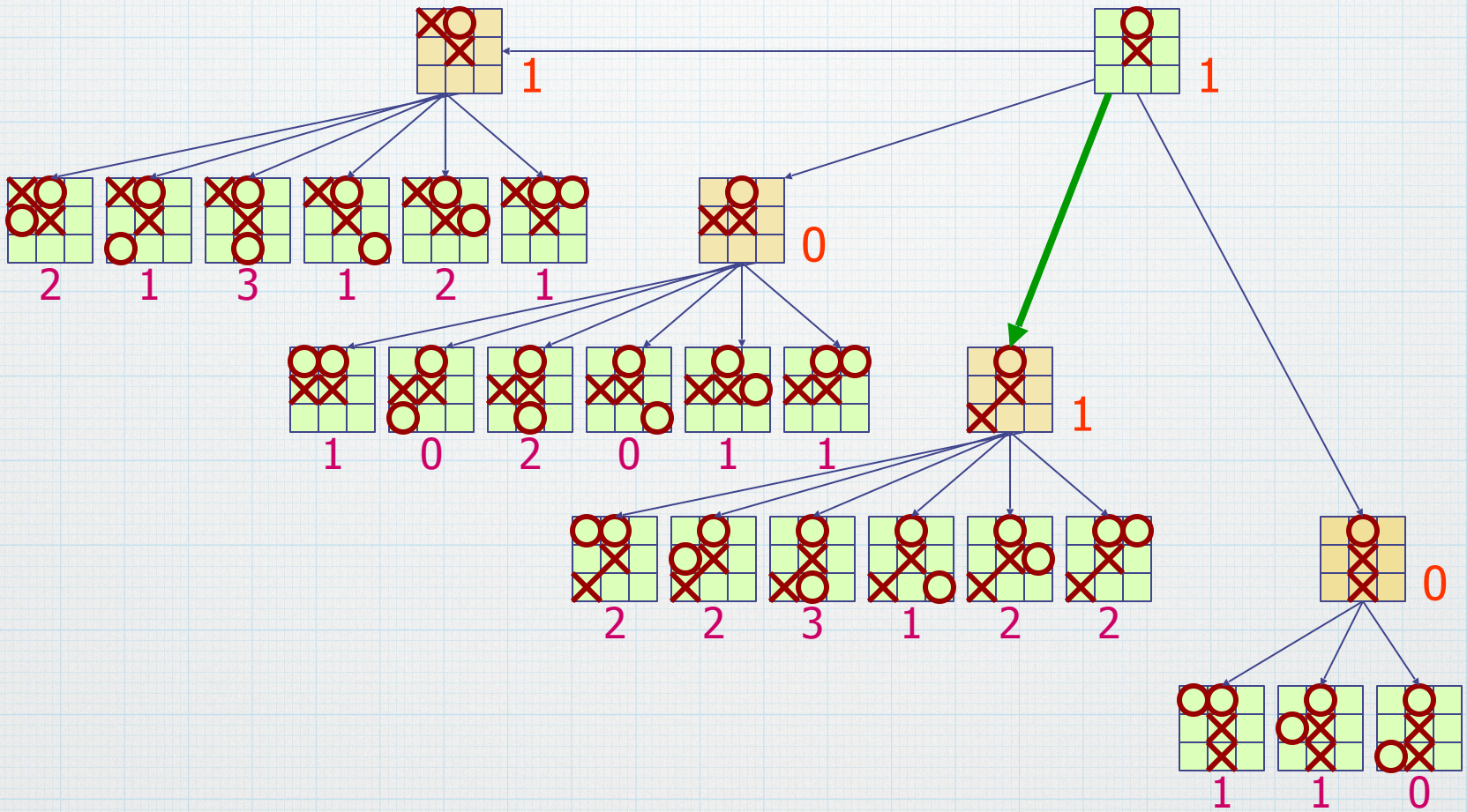
Exemple

Tic-Tac-Toe avec
Horizon = 2



e(s) = nombre de lignes, de colonnes et de diagonales ouvertes pour MAX
- nombre de lignes, de colonnes et de diagonales ouvertes pour MIN

Exemple



e(s) = nombre de lignes, de colonnes et de diagonales ouvertes pour MAX
- nombre de lignes, de colonnes et de diagonales ouvertes pour MIN

Procédure Minimax

1. Développez l'arborescence de jeu uniformément de l'état actuel (où c'est le tour de MAX de jouer) à la profondeur h
2. Calculer la fonction d'évaluation à chaque feuille de l'arbre
3. Sauvegarde les valeurs des feuilles à la racine de l'arbre comme suit :
   1. Un nœud MAX obtient le <u>maximum</u> de l'évaluation de ses successeurs
   2. Un nœud MIN obtient le <u>minimum</u> de l'évaluation de ses successeurs
4. Sélectionnez le déplacement vers le nœud MIN qui a la valeur maximale sauvegardée

Horizon de la procédure
Nécessaire pour limiter la taille de l'arbre ou pour retourner un décision dans le délai imparti

Jouer au jeu (pour MAX)

Répétez jusqu'à gagner, perdre ou faire match nul

1. Sélectionnez le déplacement à l'aide de la procédure Minimax
2. Exécuter le déplacement
3. Observez le mouvement de MIN

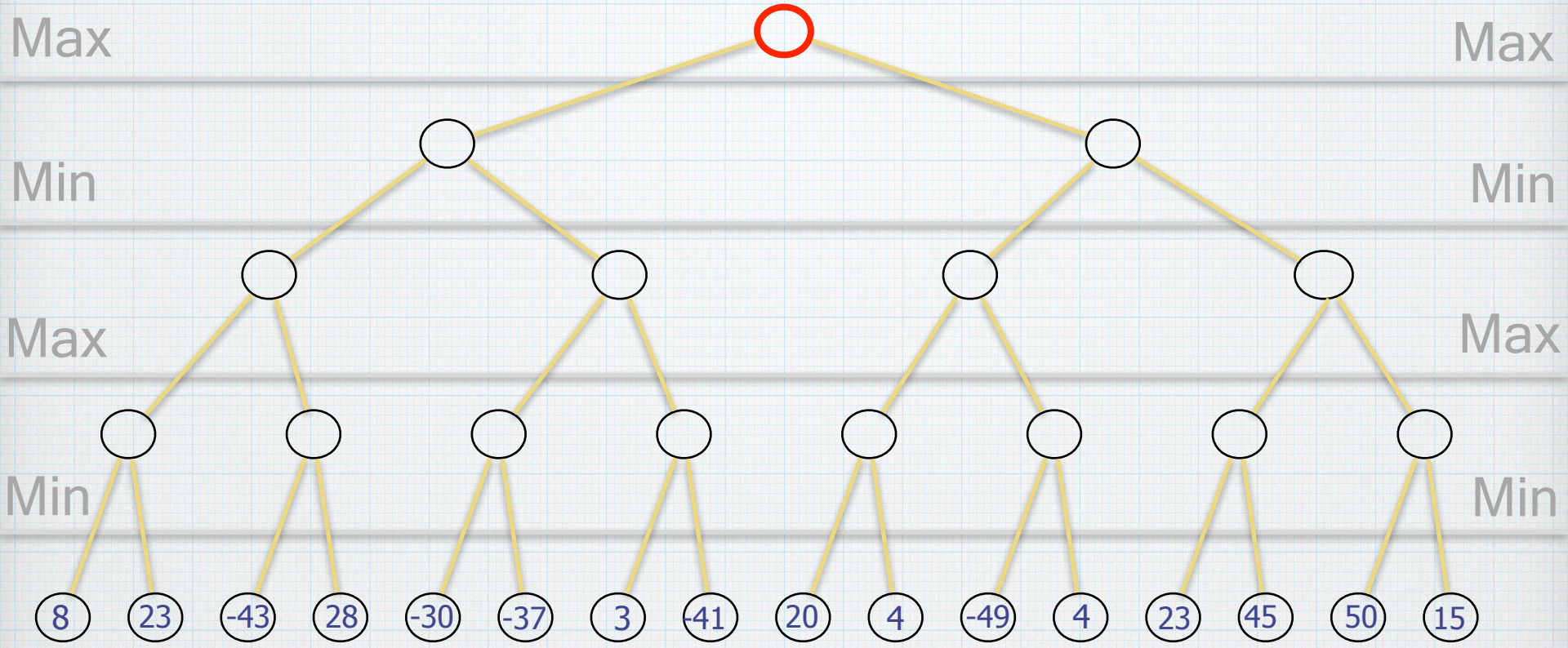
Algorithme Minmax

```
function MINIMAX-DECISION(state) returns an action
    v ← MAX-VALUE(state)
    return the action in SUCCESSORS(state) with value v
```

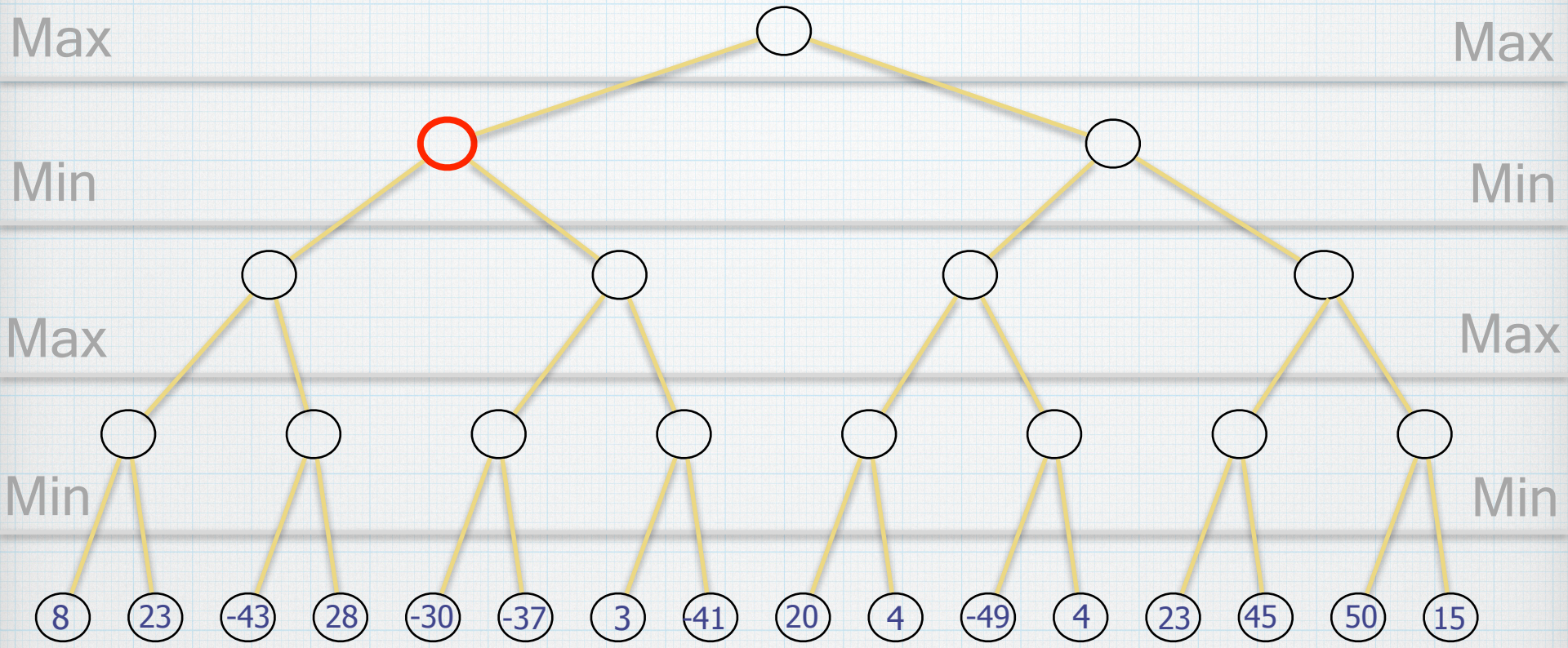
```
function MAX-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s))
    return v
```

```
function MIN-VALUE(state) returns a utility value
    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← ∞
    for a, s in SUCCESSORS(state) do
        v ← MIN(v, MAX-VALUE(s))
    return v
```

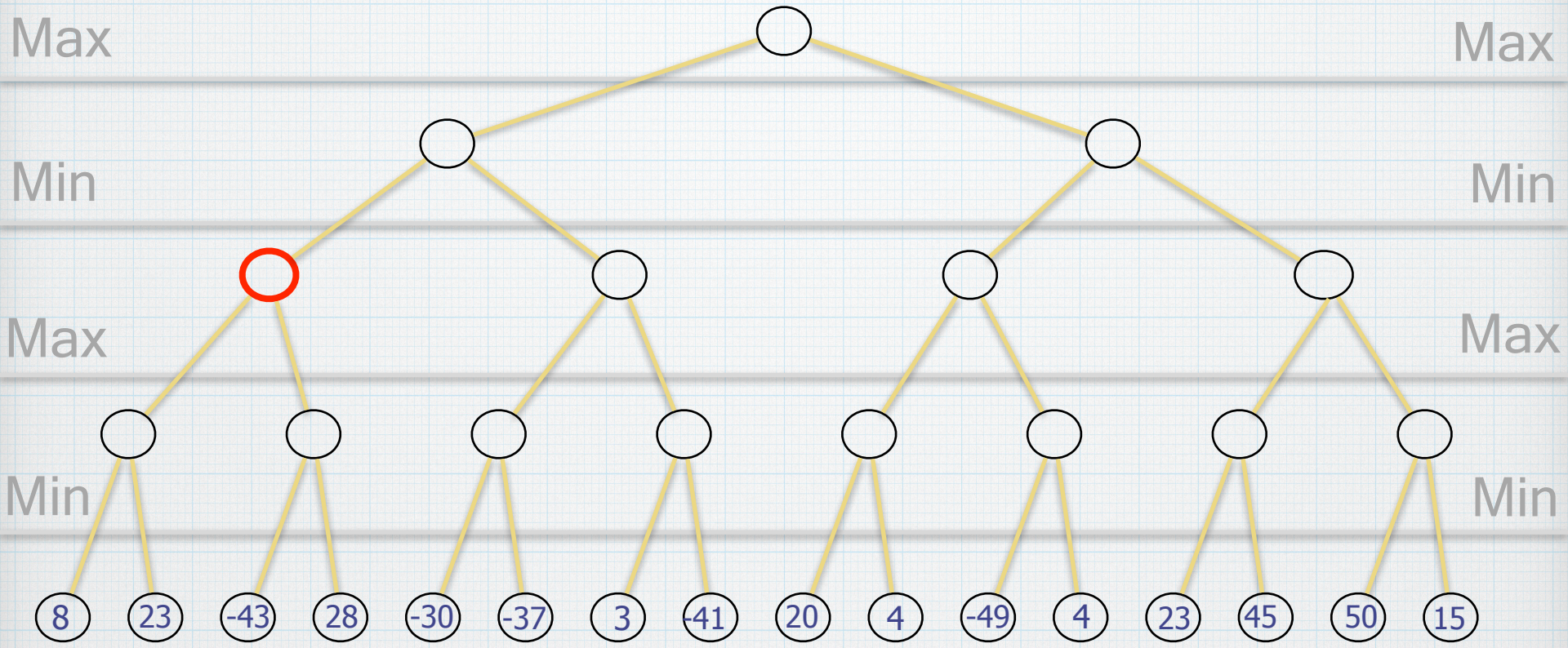
Exemple MinMax



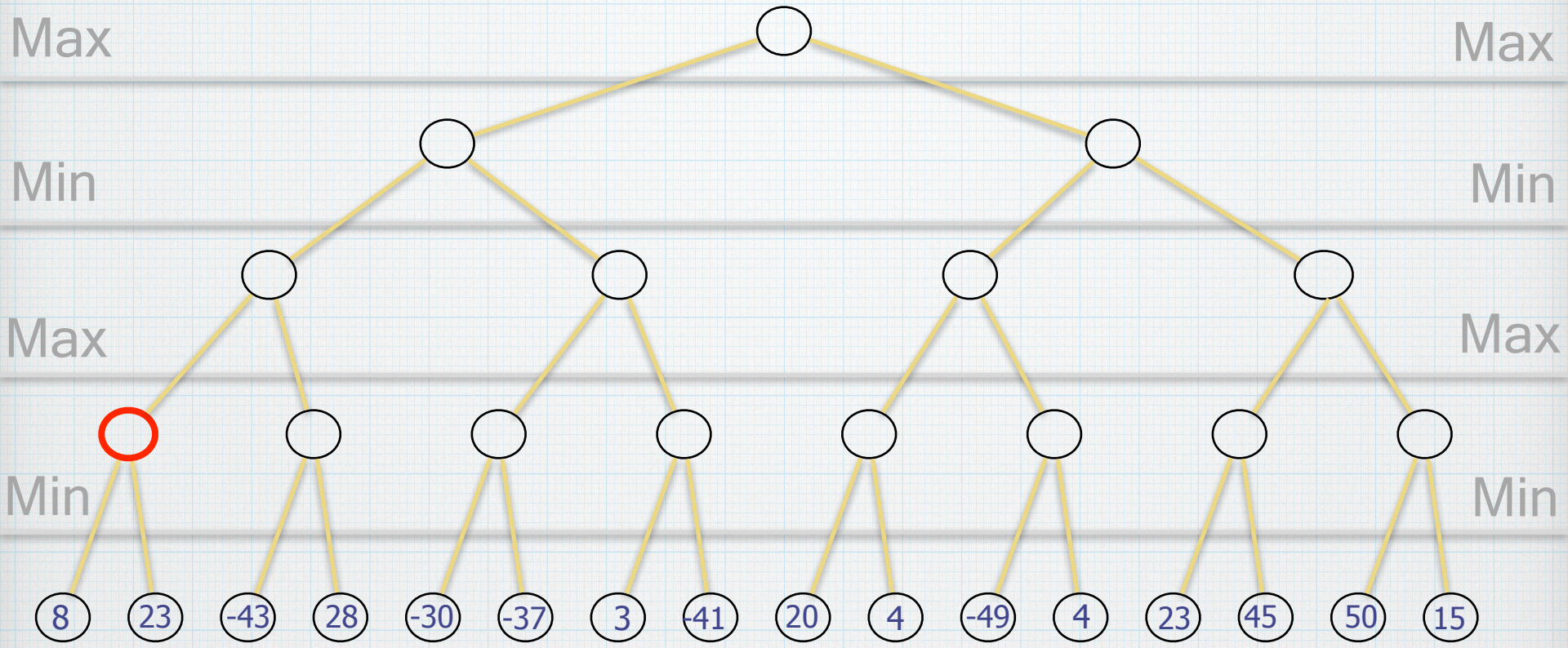
Exemple MinMax



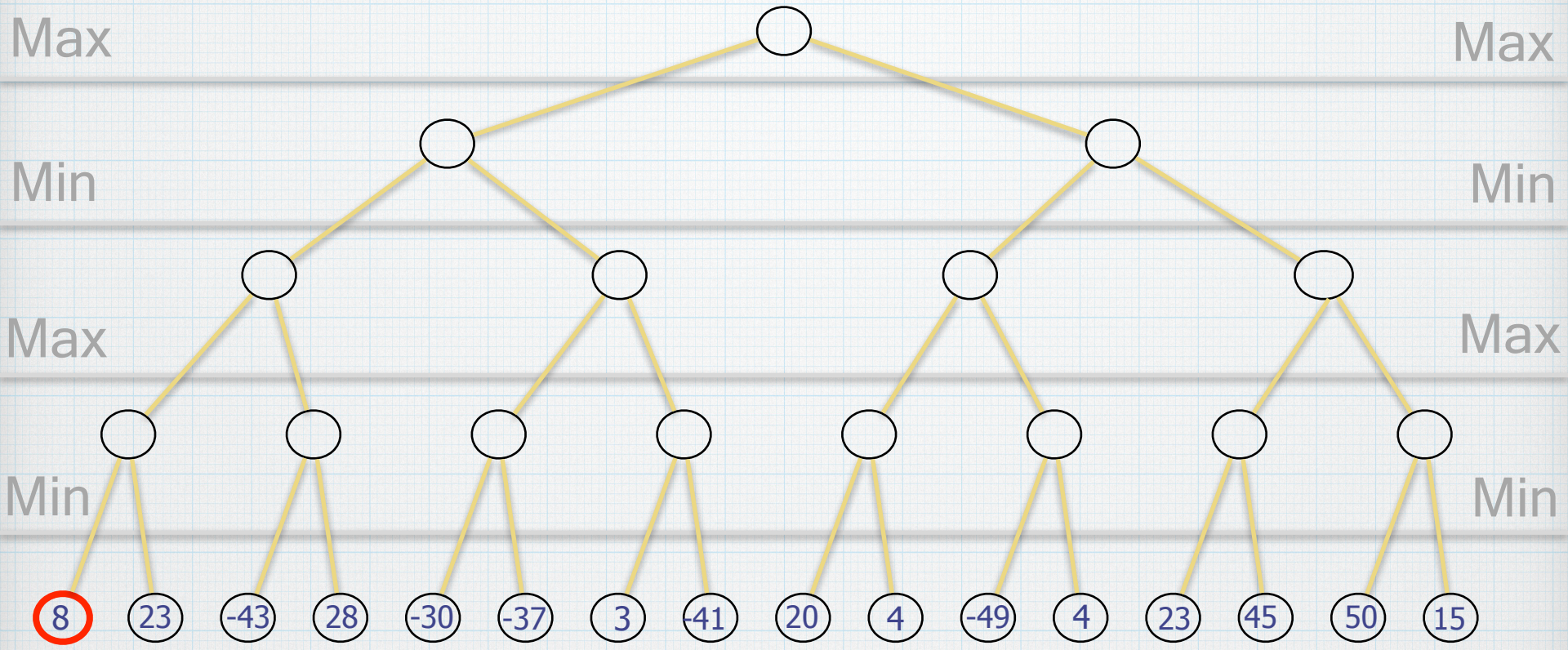
Exemple MinMax



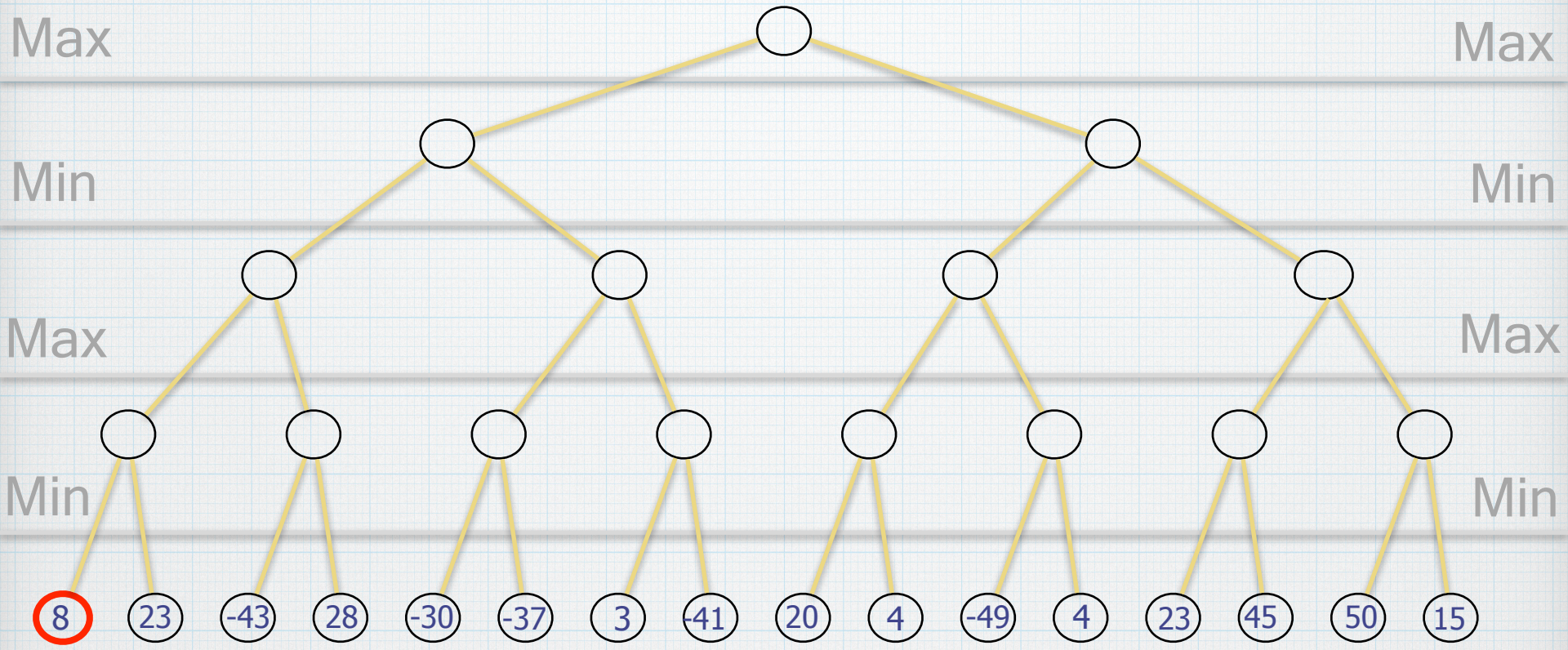
Exemple MinMax



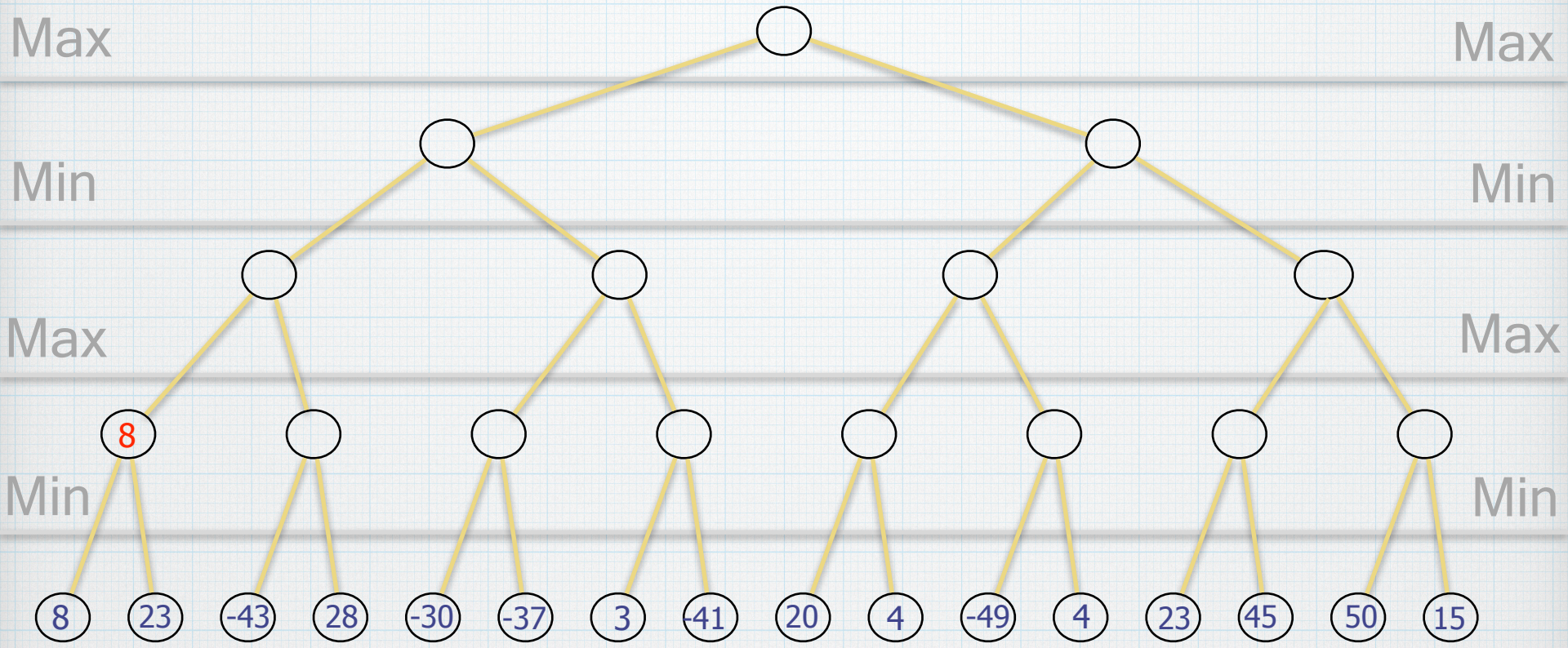
Exemple MinMax



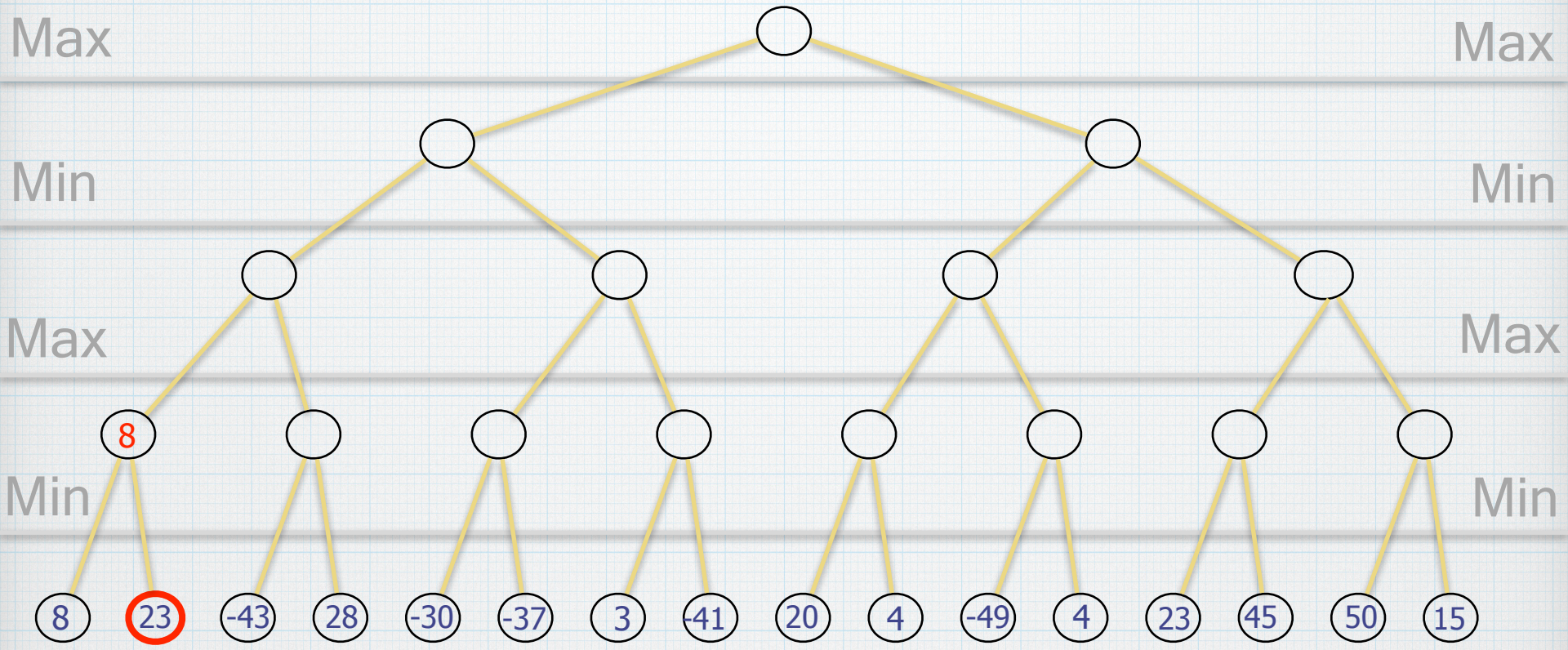
Exemple MinMax



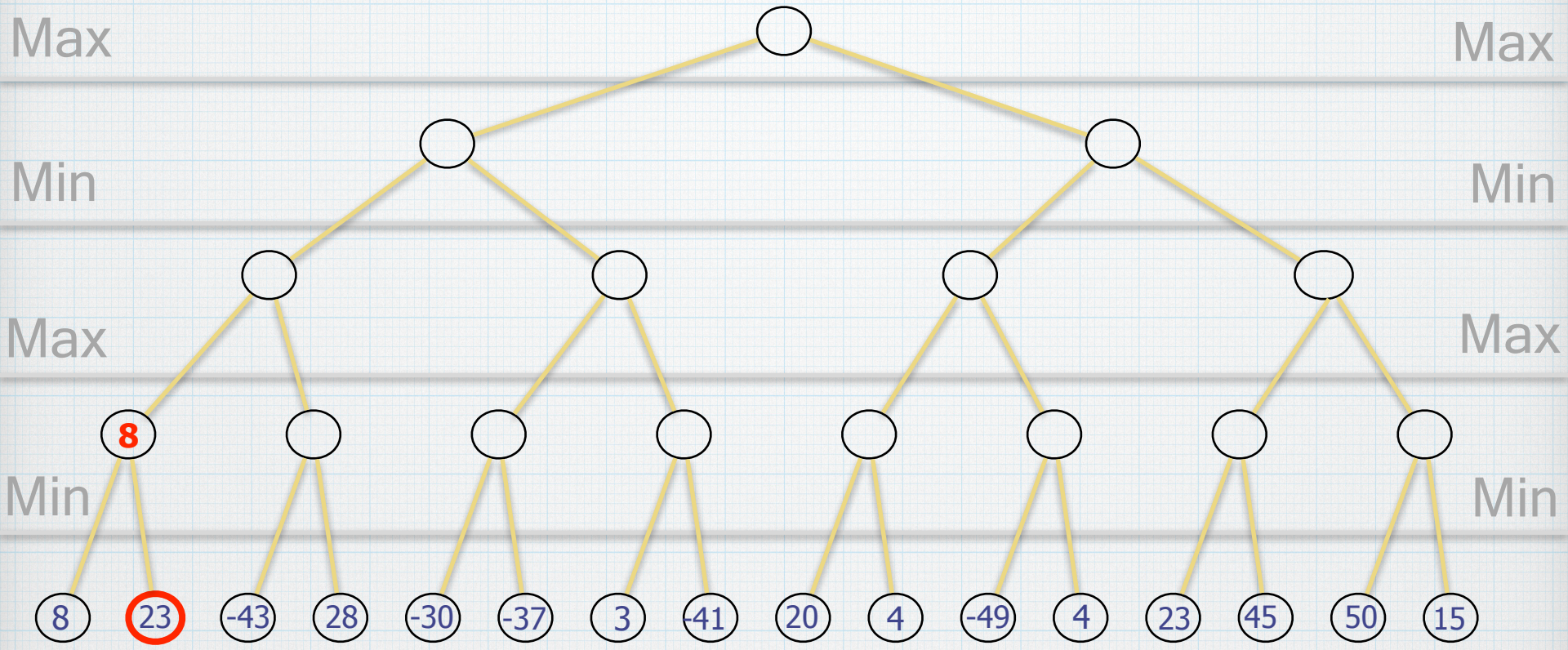
Exemple MinMax



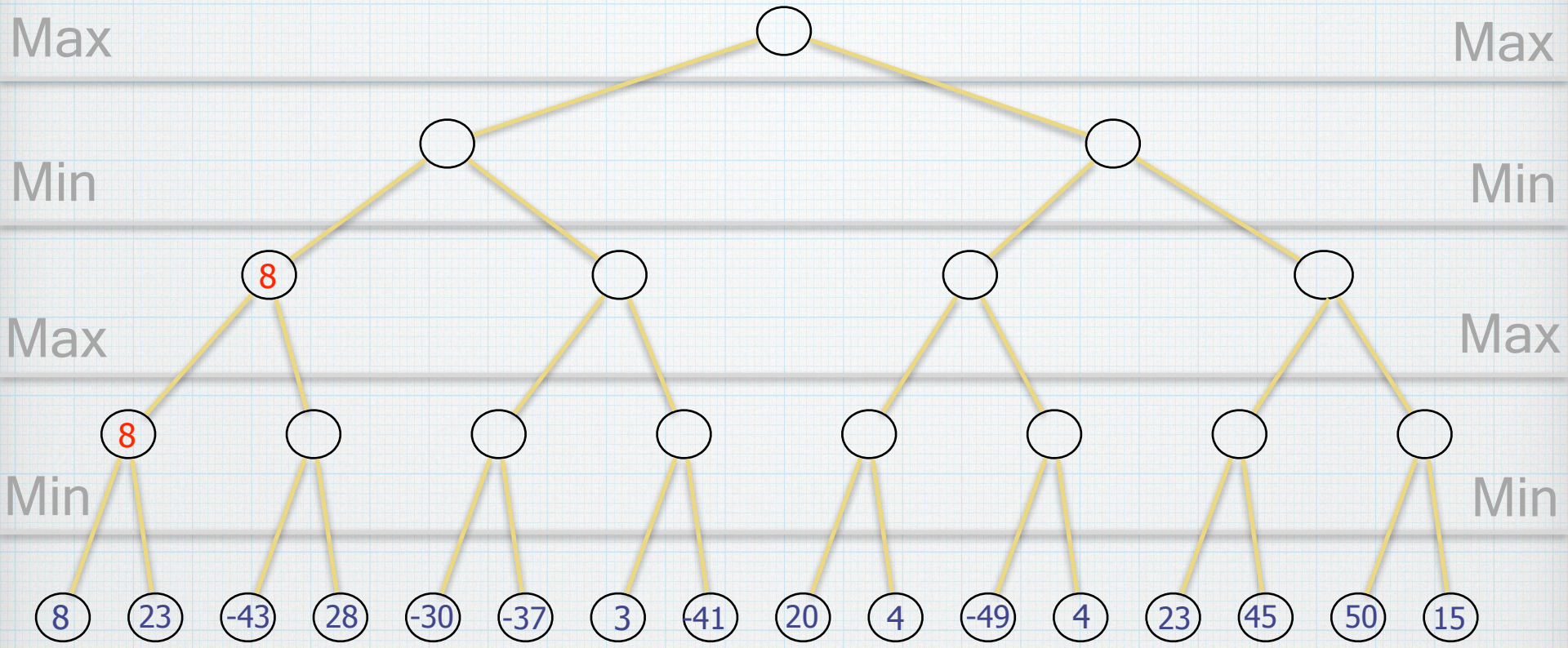
Exemple MinMax



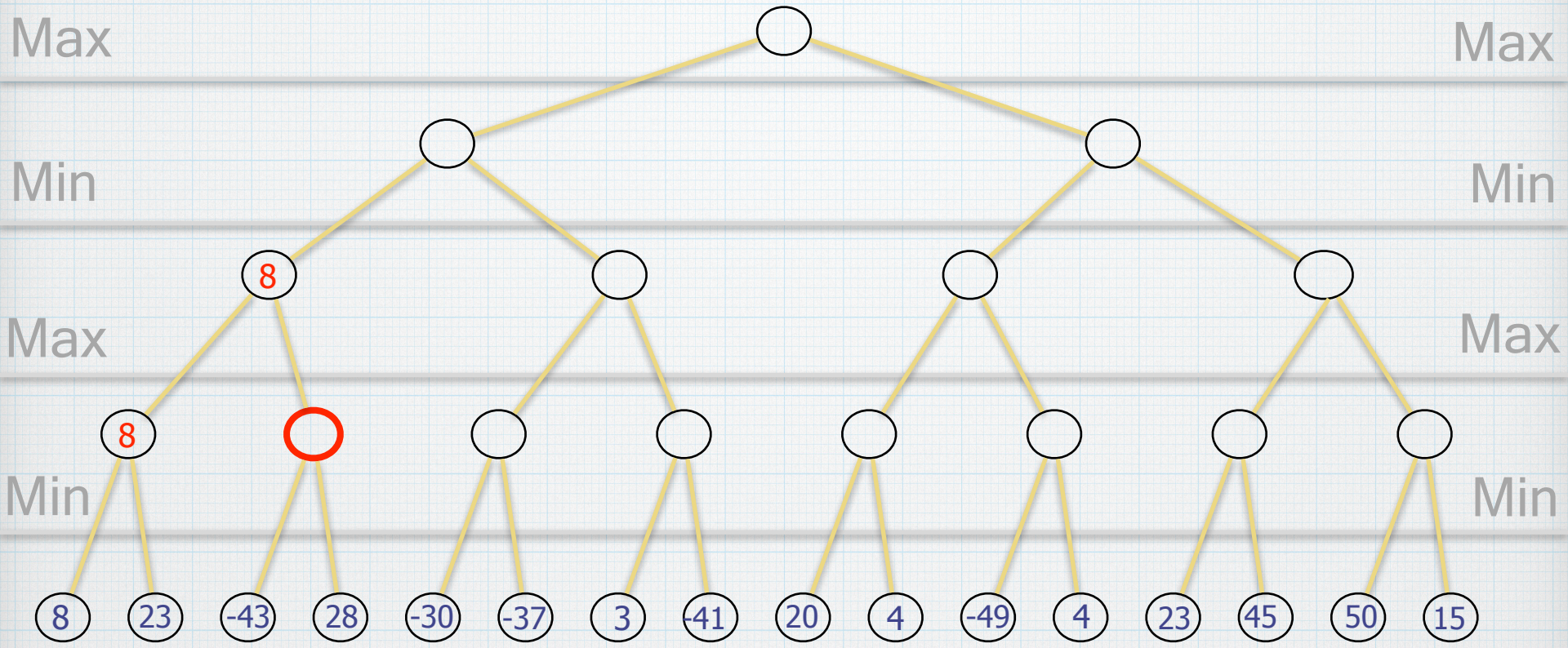
Exemple MinMax



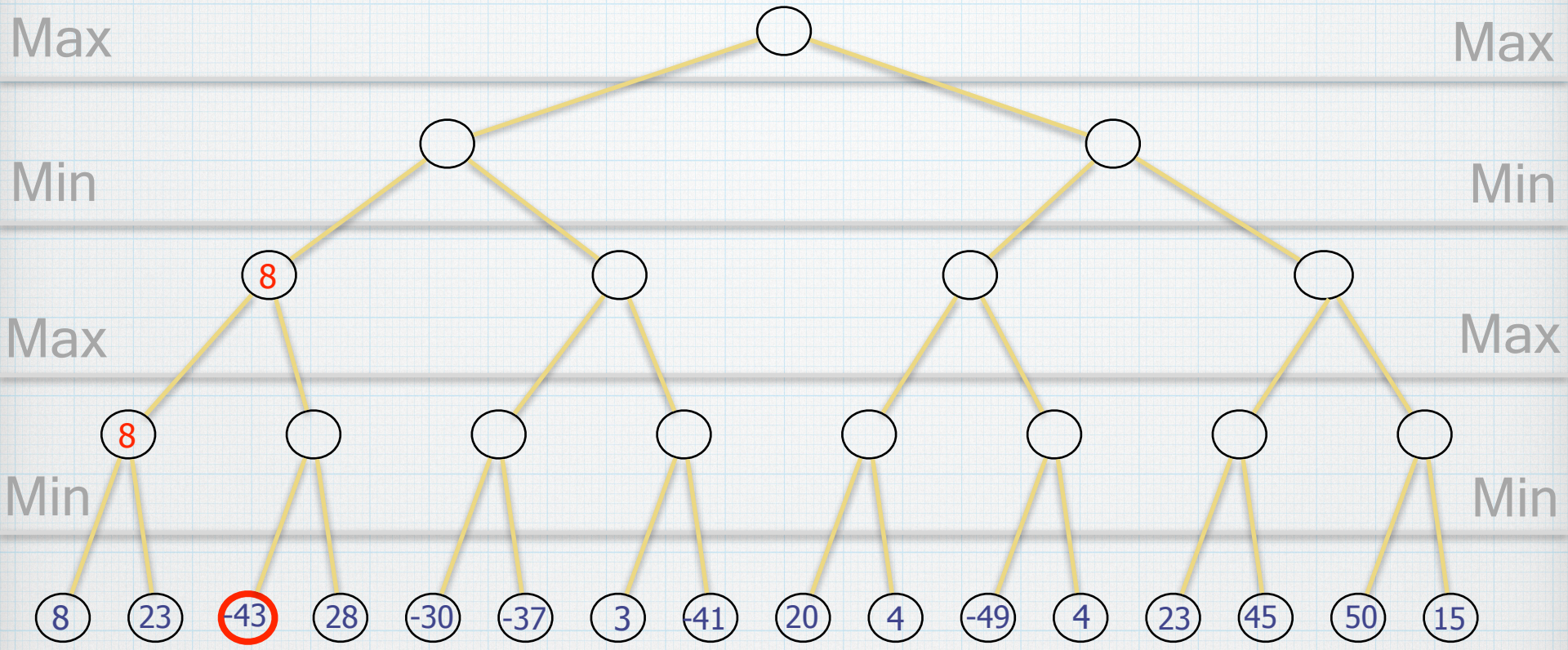
Exemple MinMax



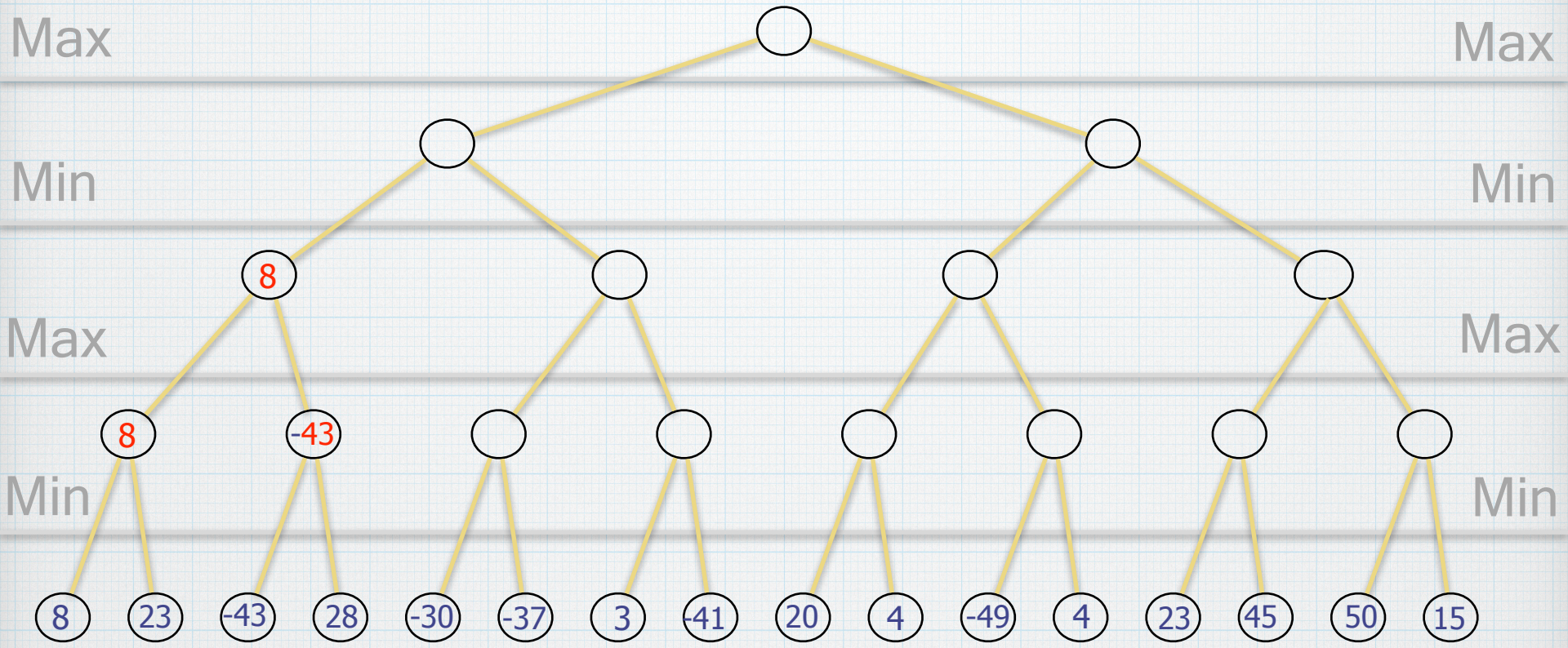
Exemple MinMax



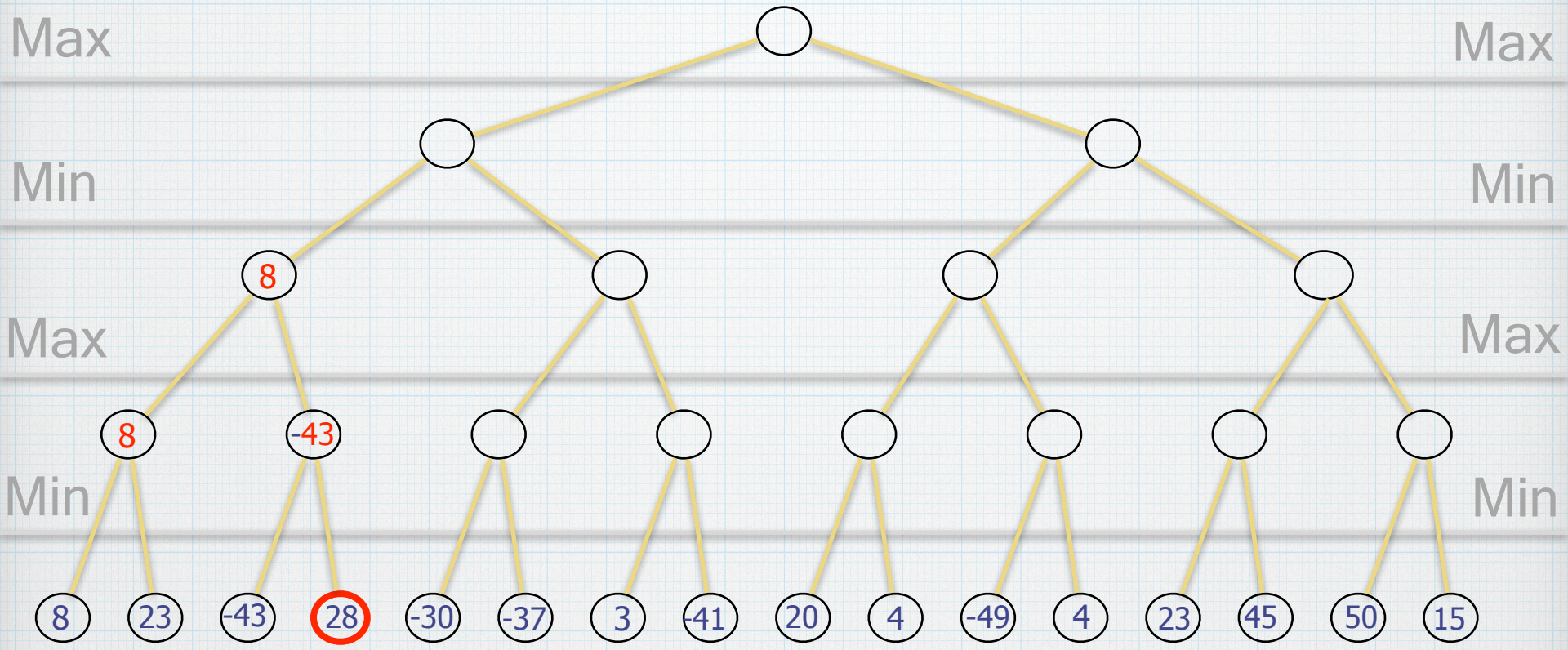
Exemple MinMax



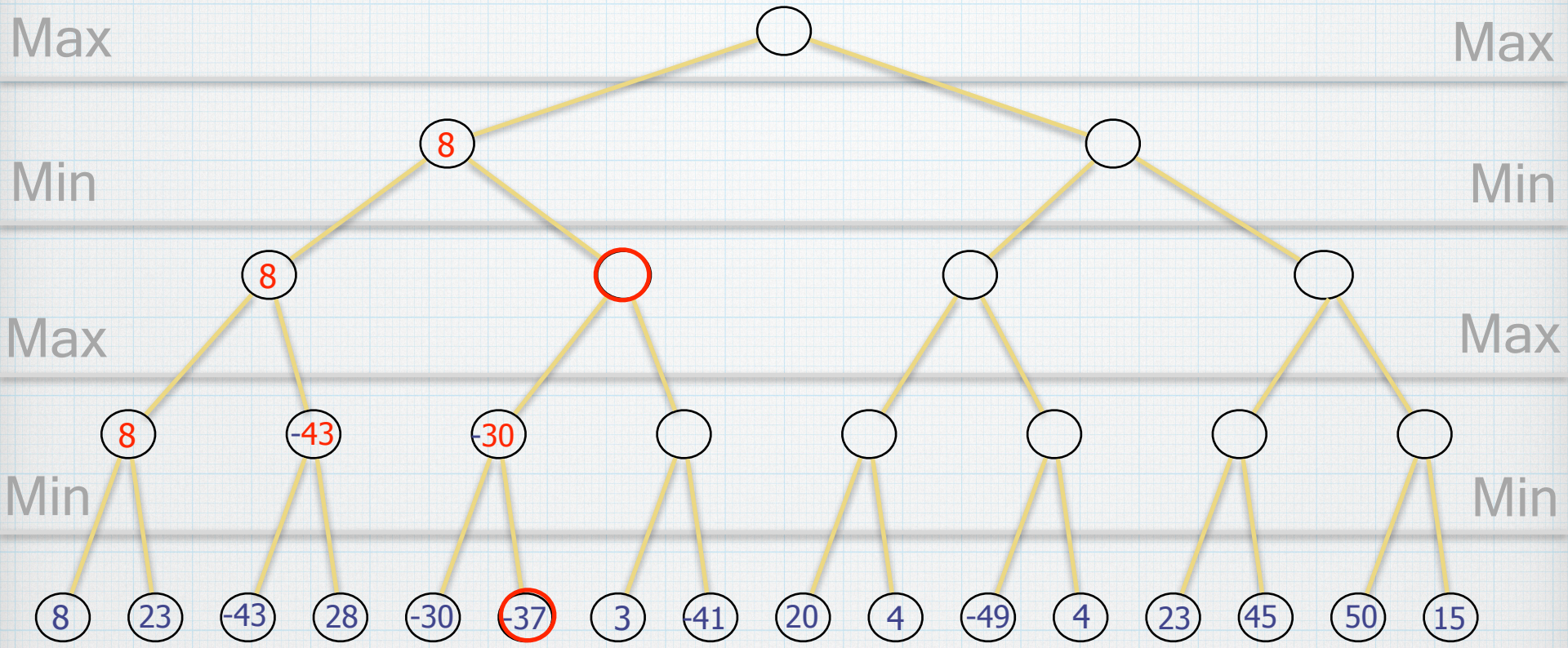
Exemple MinMax



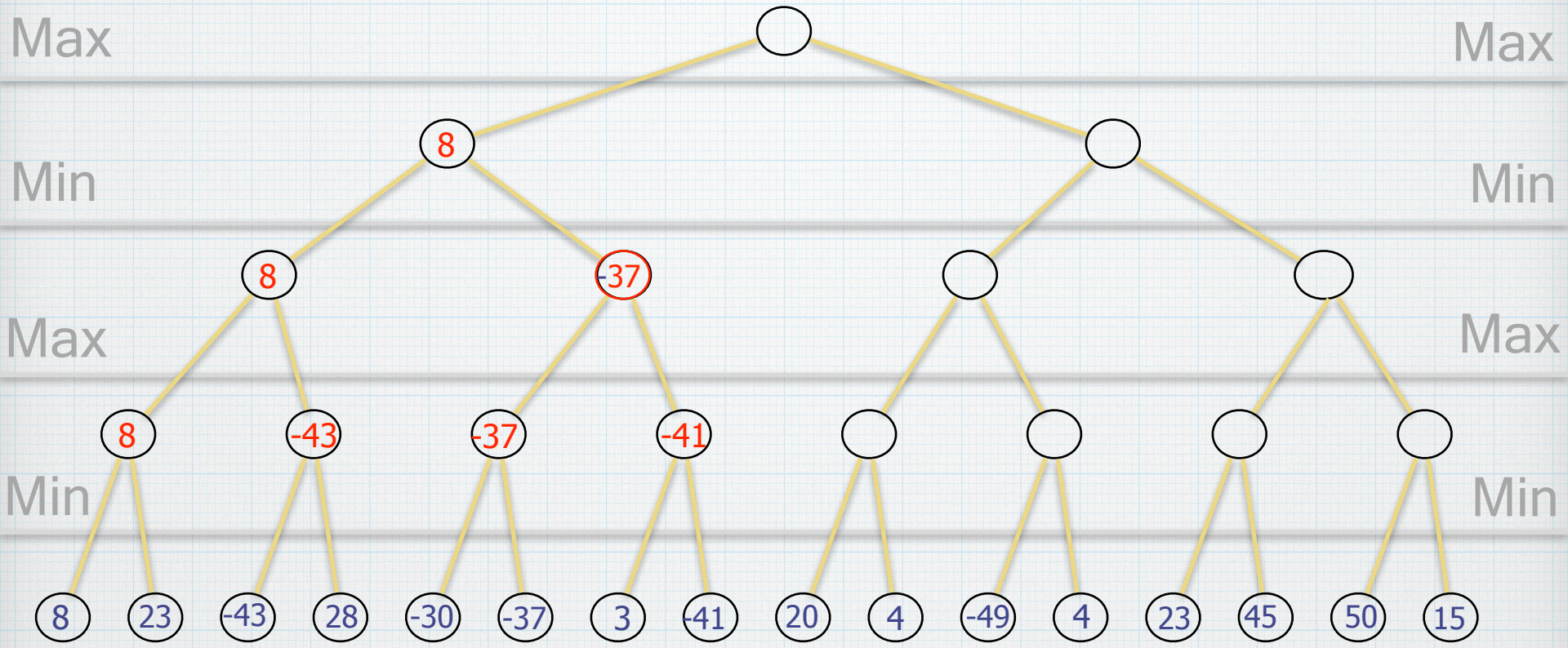
Exemple MinMax



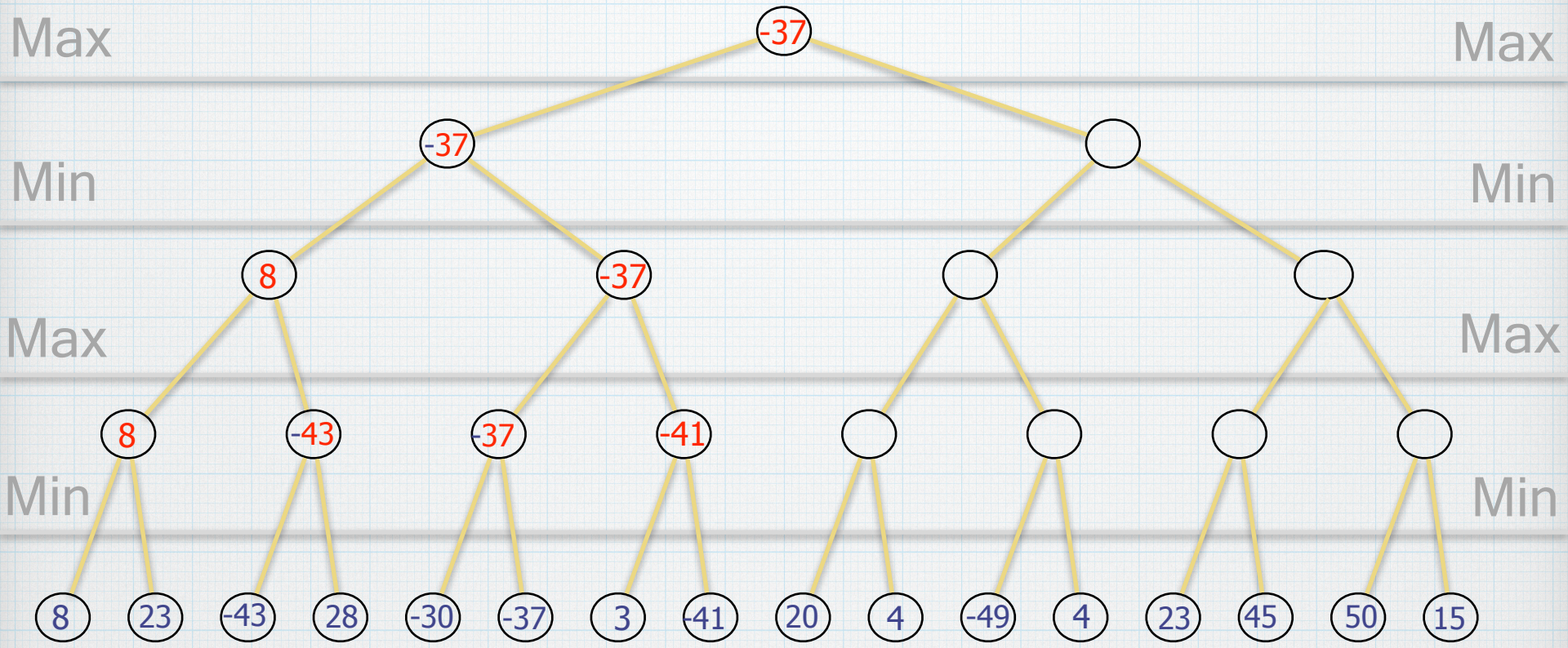
Exemple MinMax



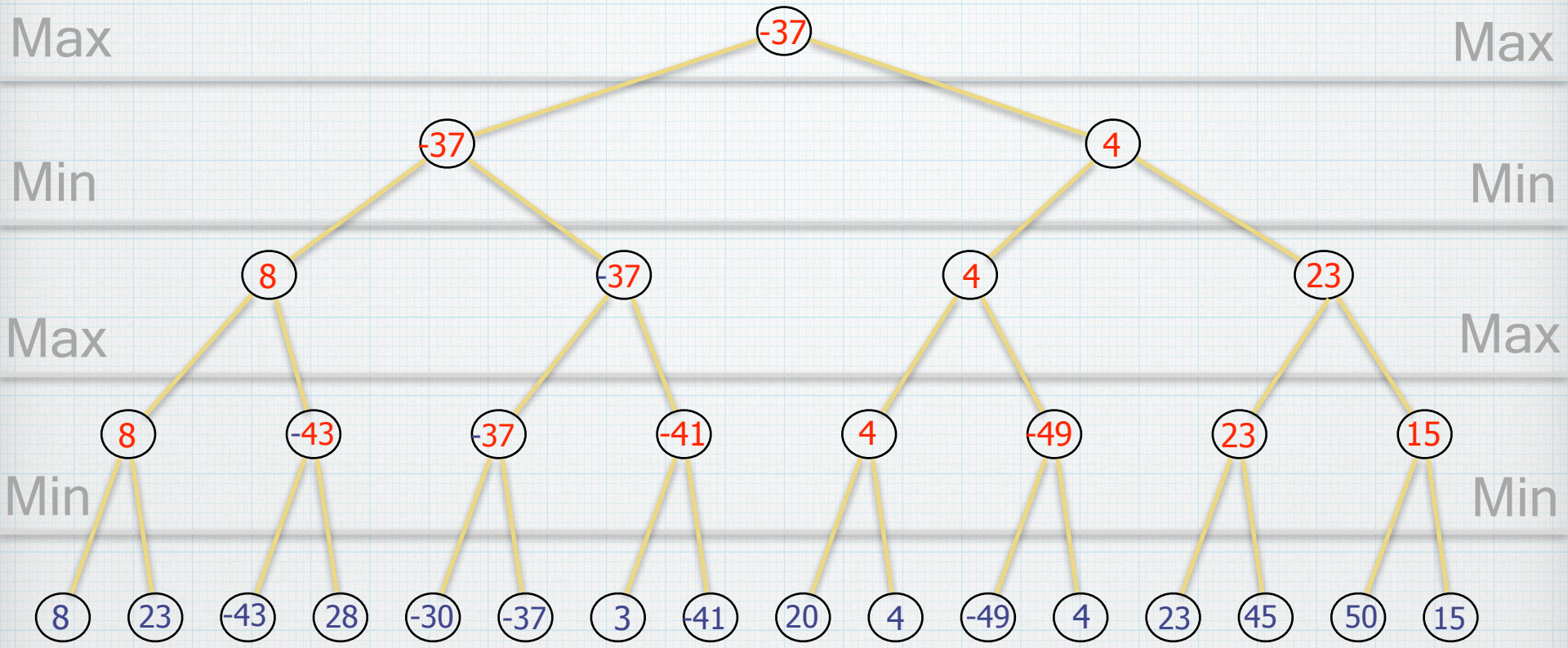
Exemple MinMax



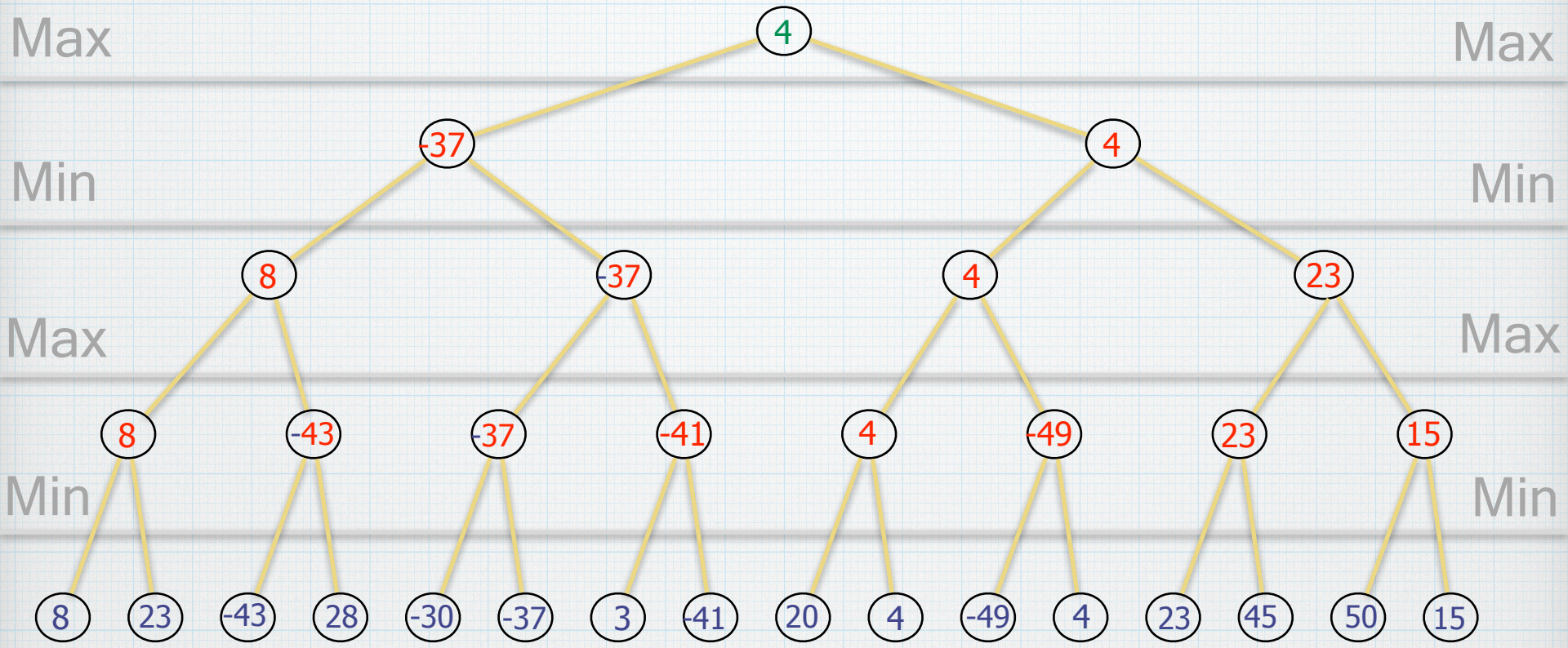
Exemple MinMax



Exemple MinMax



Exemple MinMax



Problèmes

En général l'arbre de recherche est trop grand pour permettre d'atteindre un état terminal !

Exemples:

- Jeu de Dames: ~10^{40} nodes
- Jeu d'échec: ~10^{120} nodes

* Choix de l'horizon h
  * Taille de la mémoire nécessaire
  * Nombre de nœuds examinés

Horizon adaptatif

* Attendre le calm (quand le joueur adverse délibère)
* Étendre les nœuds singuliers /Recherche secondaire ou parallèle.
* Notez que l'horizon peut alors ne pas être le même sur chaque chemin de l'arbre

Comment accélérer la recherche

* Deux approches
  * la première maintient l'exactitude de la solution
  * la deuxième introduit une approximation

1. **Élagage alpha-bêta** (*alpha-beta pruning*)
   * **idée :** identifier des chemins dans l'arbre qui sont inutiles à explorer

2. **Couper la recherche et remplacer l'utilité par une fonction d'évaluation heuristique**
   * **idée :** faire une recherche la plus profonde possible en fonction du temps à notre disposition et tenter de prédire le résultat de la partie si on n'arrive pas à la fin

Bornes de l'élagage alpha-bêta

* L'élagage alpha-bêta tire son nom de deux bornes transmises lors du calcul, qui restreignent l'ensemble des solutions possibles en fonction de la partie de l'arbre de recherche qui a déjà été vue.
* Spécifiquement:
  * Beta est la limite supérieure minimale des solutions possibles
  * Alpha est la borne inférieure maximale des solutions possibles

Procédure Alpha-Bêta

* Générer l'arbre de jeu en profondeur d'abord jusqu'à la profondeur h.
* Sauvegarder les estimations (valeurs, alpha et bêta) des fonctions d'évaluation chaque fois que possible
* Couper (Prune) les branches qui ne conduisent pas au changement de la décision finale.

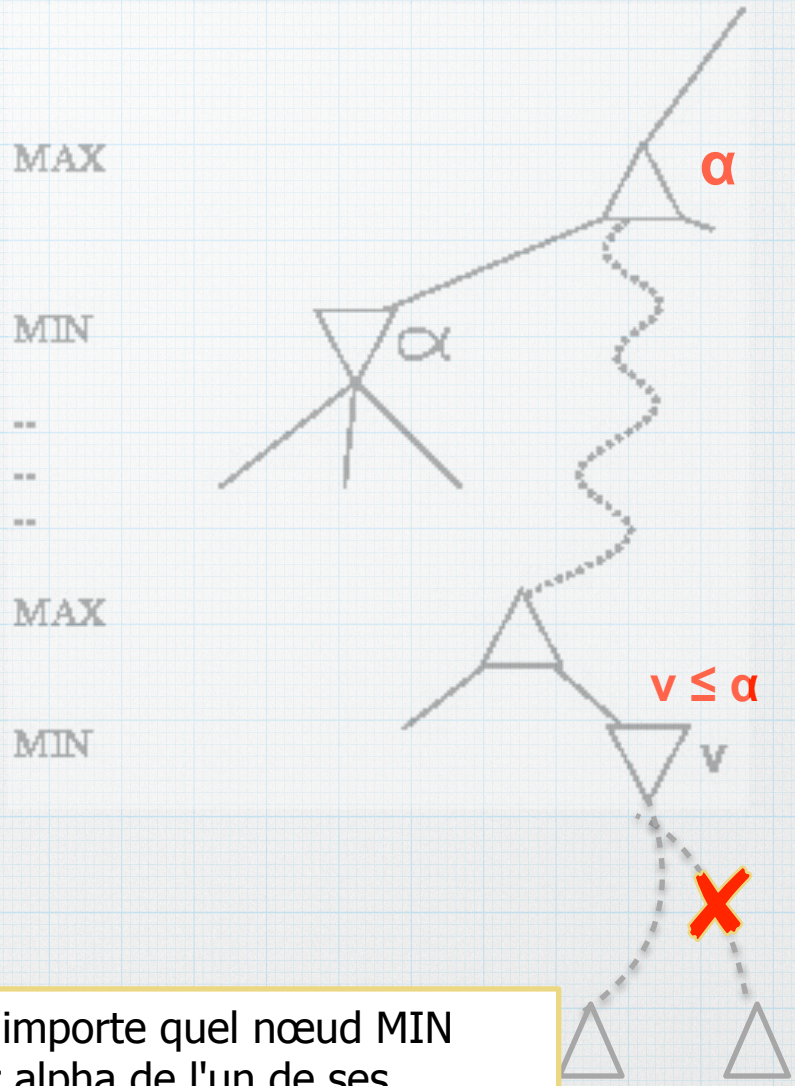
Sur quelle *condition peut-on*

couper une branche (sous arbre) d'un nœud Min?

couper une branche (sous arbre) d'un nœud Max?

Alpha-beta Pruning
Condition pour couper dans un nœud Min

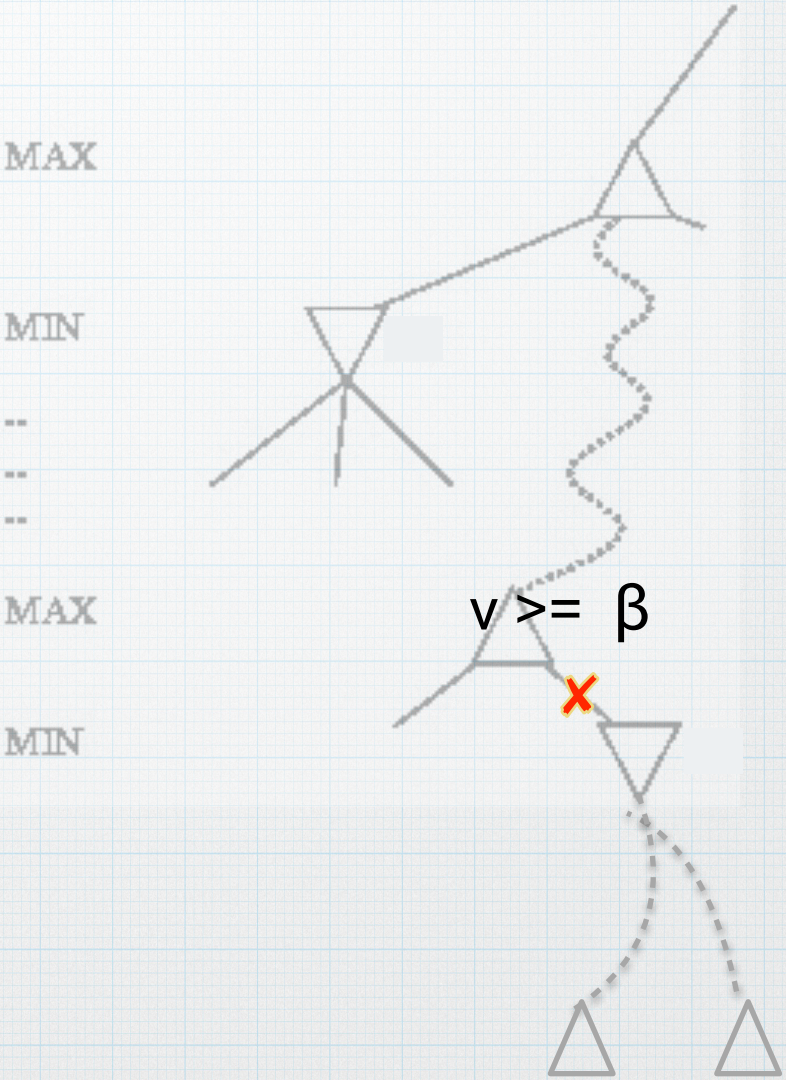
- Supposons que α est la valeur du meilleur choix pour Max (c.-à-d., plus grande valeur) trouvé jusqu'à présent:
  - Si on est **dans un nœud Min** et que sa valeur ***v*** **devient inférieure α** (donc « pire que α» du point de vue de Max), il faut arrêter la recherche (couper la branche).



La recherche peut être interrompue en dessous de n'importe quel nœud MIN dont la valeur bêta est inférieure ou égale à la valeur alpha de l'un de ses ancêtres MAX

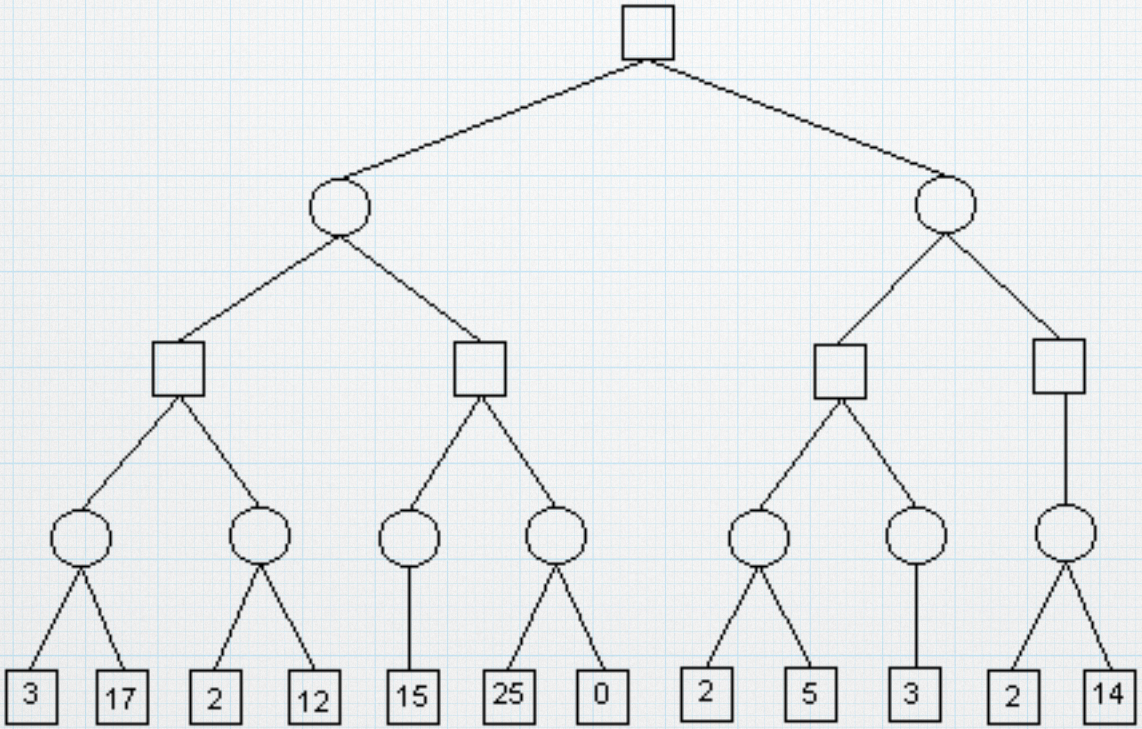
Alpha-beta Pruning
Condition pour couper dans un nœud Max

* Sachant que β est la valeur du meilleur choix pour Min (c.-à-d., plus petite valeur) trouvé jusqu'ici:
    * Si on est **dans un nœud Max** et que sa valeur **devient supérieur à β** (donc « pire que β» du point de vue de Min), il faut arrêter la recherche (couper la branche).



Exemple d'*Alpha-beta pruning*

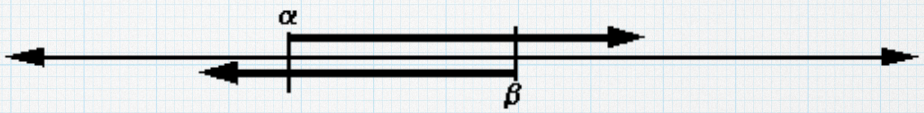
* Pour illustrer le minimax avec l'élagage alpha-bêta, nous utilisons l'arbre minimax suivant comme exemple :



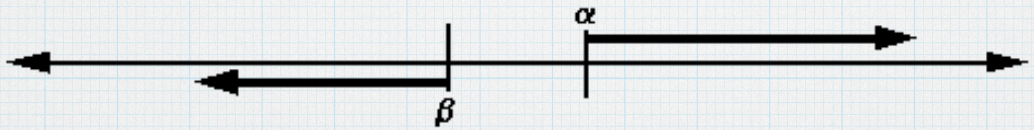
Exemple d'*Alpha-beta pruning*
Considérations

* Lorsqu'un nouveau nœud est considéré comme un chemin possible vers la solution, il ne peut fonctionner que si :

$$\alpha \leq N_{\mathcal{V}} \leq \beta$$



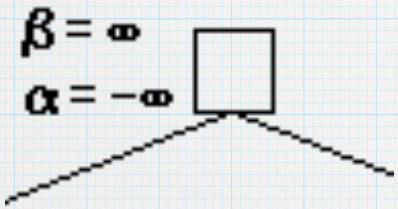
* Lorsqu'il n'y a plus de chevauchement entre les plages d'alpha et de bêta



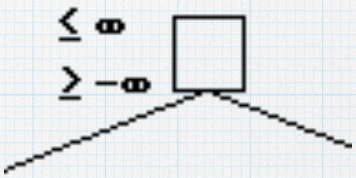
Ce nœud ne pourrait jamais aboutir à un chemin de solution

Exemple d'*Alpha-beta pruning*

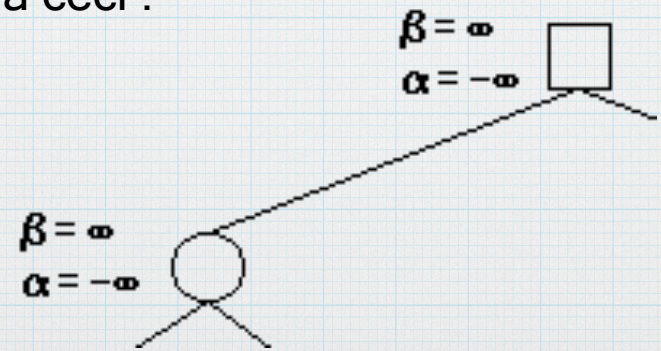
- Au début du problème, seul l'état initial est disponible
- Les limites supérieure et inférieure sont initialisées à (+/-) l'infini
- Ainsi, voici à quoi ressemble la situation initiale :



qui équivaut à

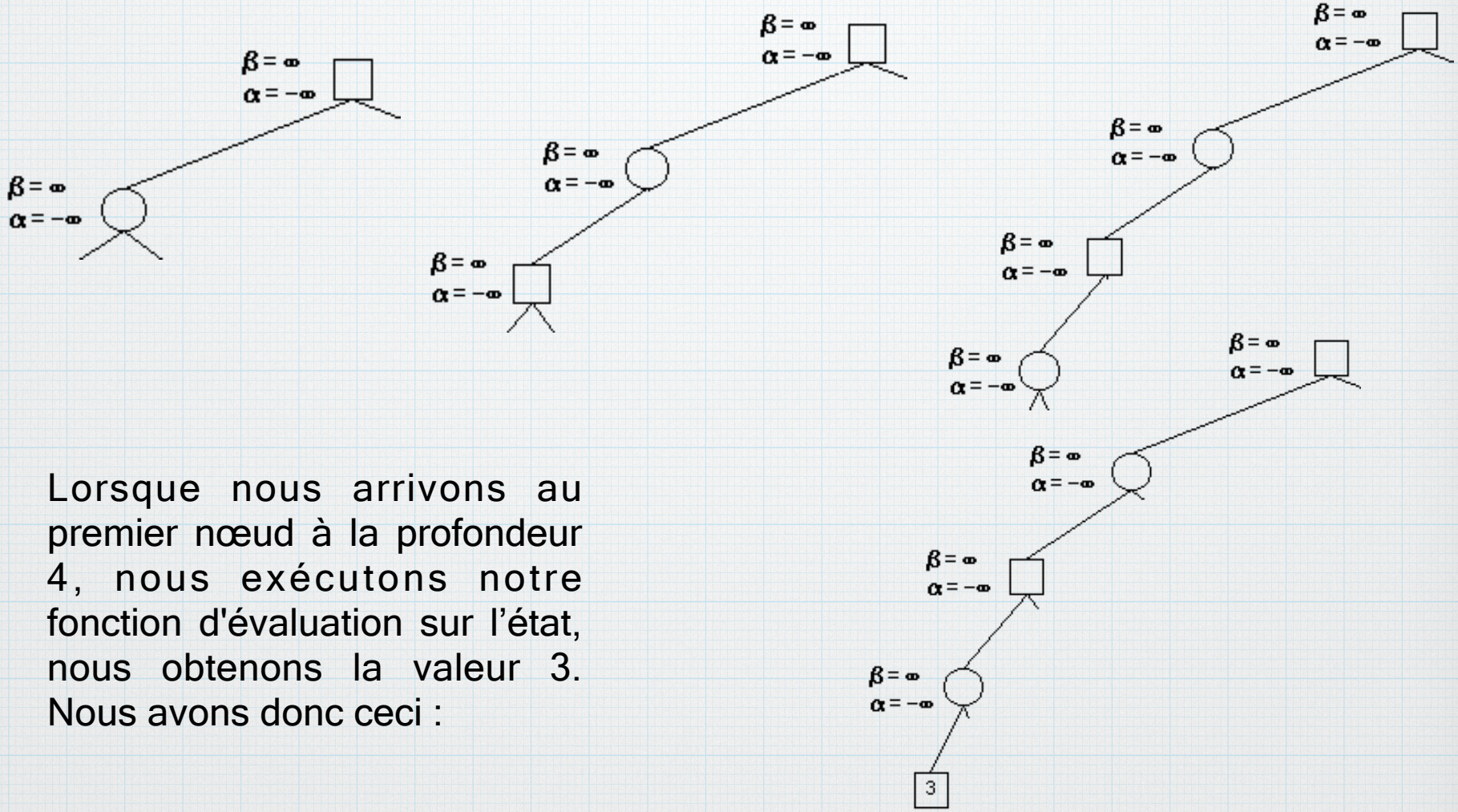


- Étant donné que les bornes contiennent toujours une plage valide, nous commençons le problème en générant le premier état enfant et en transmettant l'ensemble actuel de bornes. À ce stade, notre arbre de recherche ressemble à ceci :

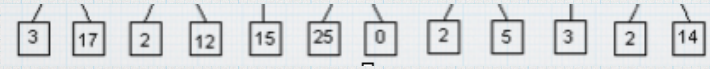


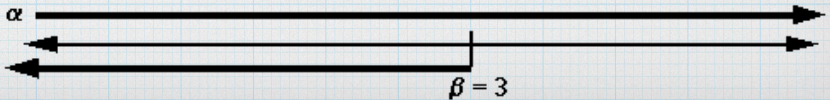
Exemple d'*Alpha-beta pruning*

* Nous ne sommes pas encore à la profondeur 4, ou dans un noeud feuille, nous générons donc à nouveau le premier nœud enfant et transmettons nos valeurs alpha et bêta actuelles :

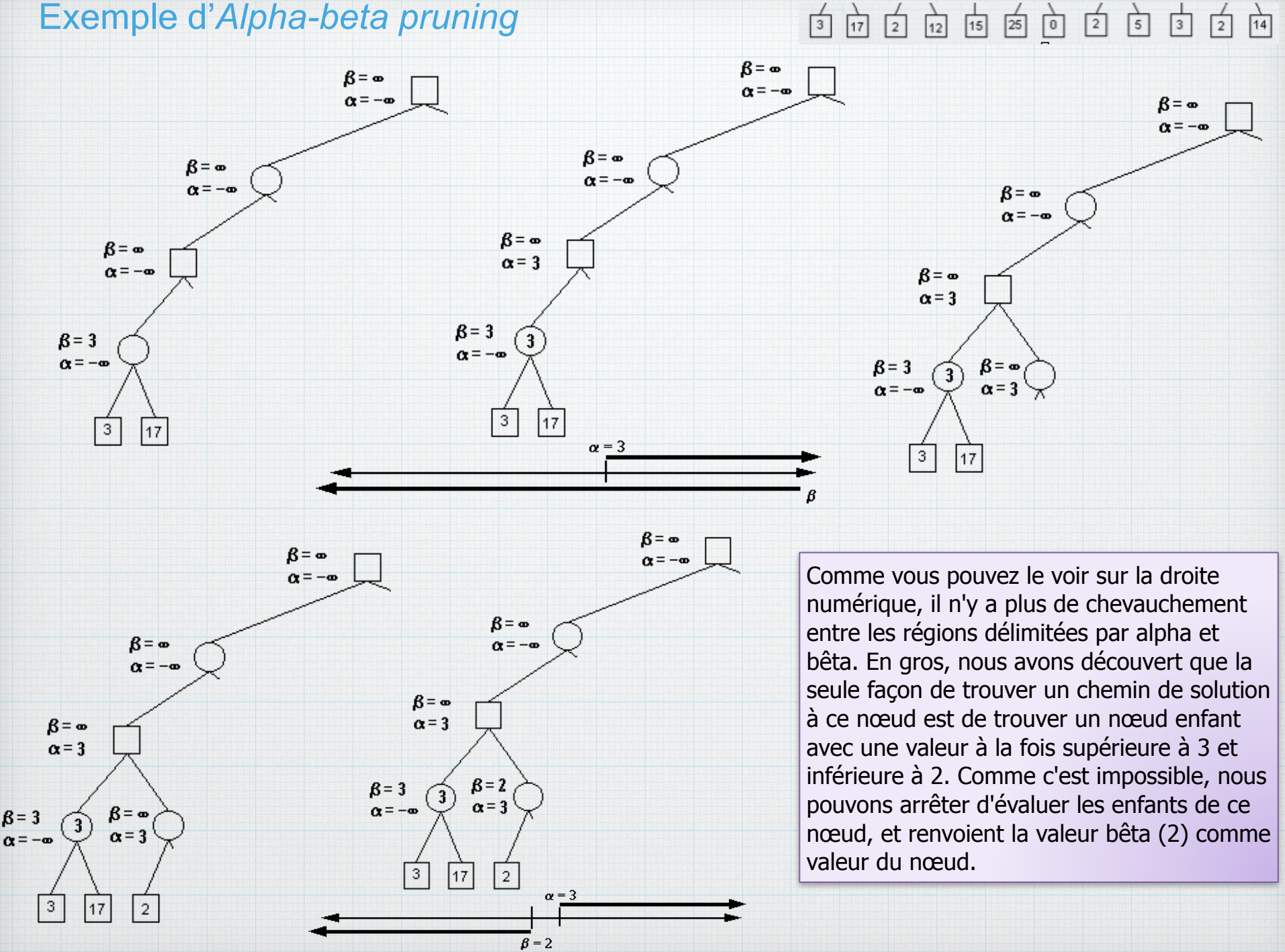


* Lorsque nous arrivons au premier nœud à la profondeur 4, nous exécutons notre fonction d'évaluation sur l'état, nous obtenons la valeur 3. Nous avons donc ceci :

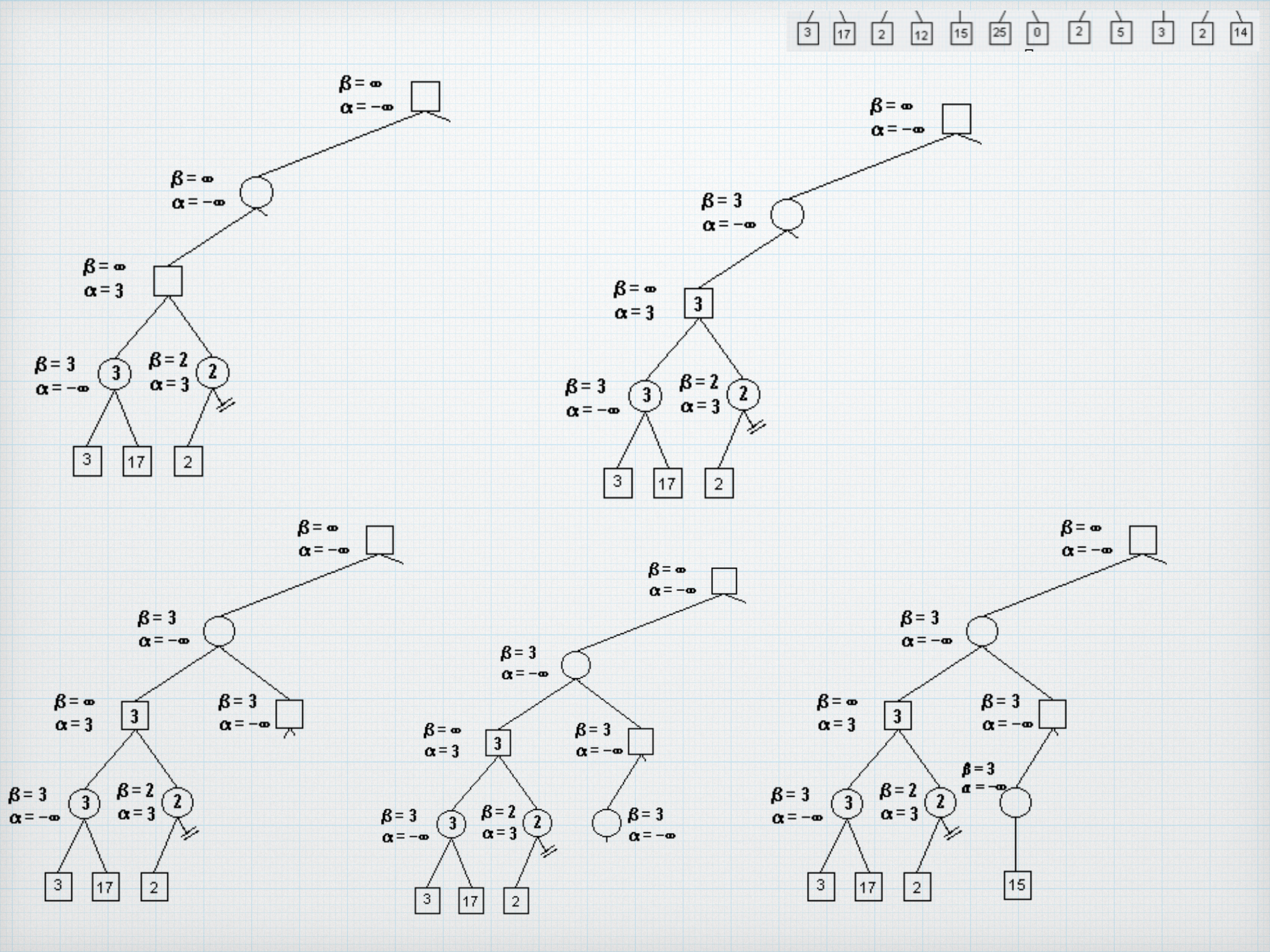


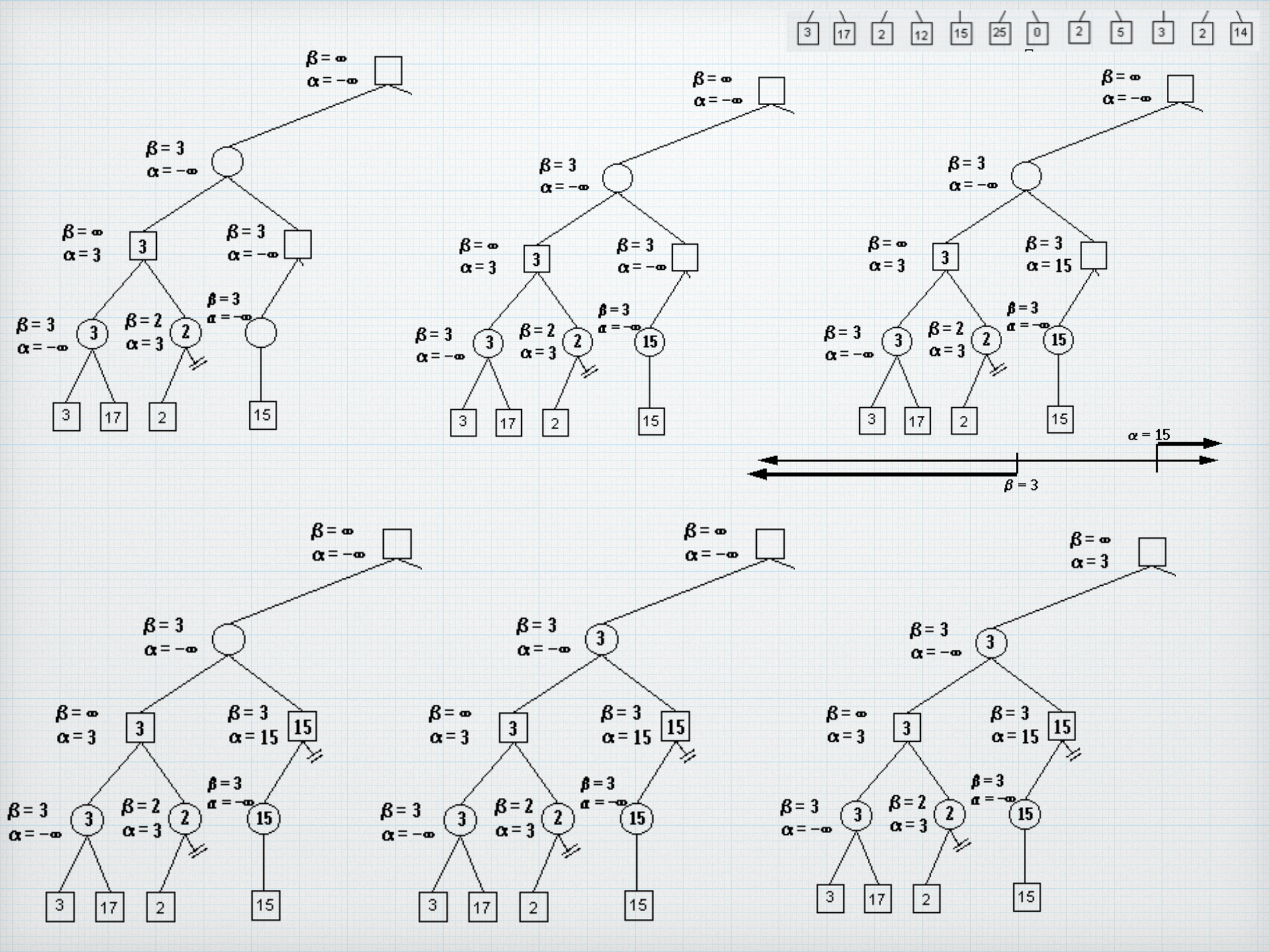


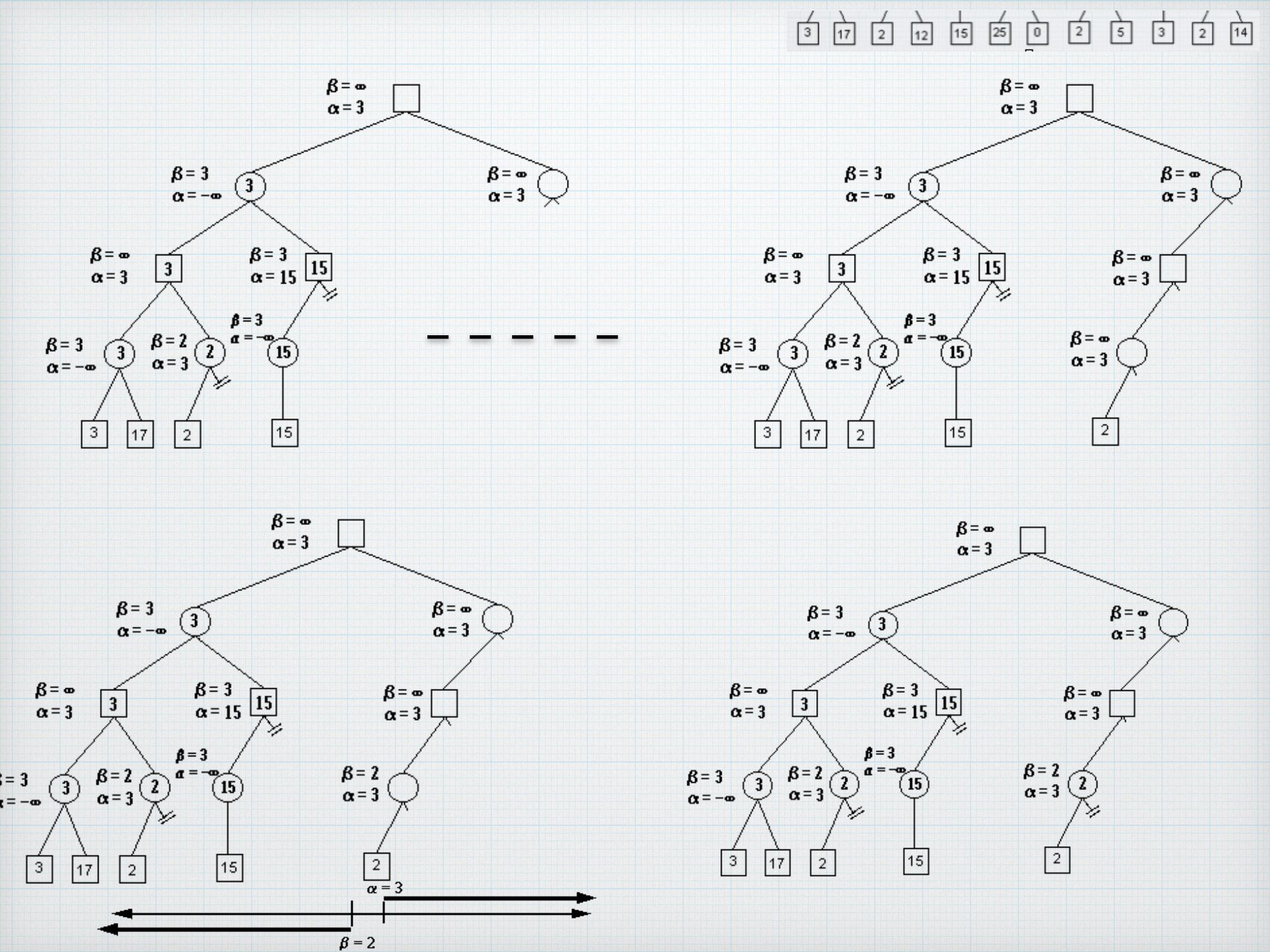
Exemple d'*Alpha-beta pruning*

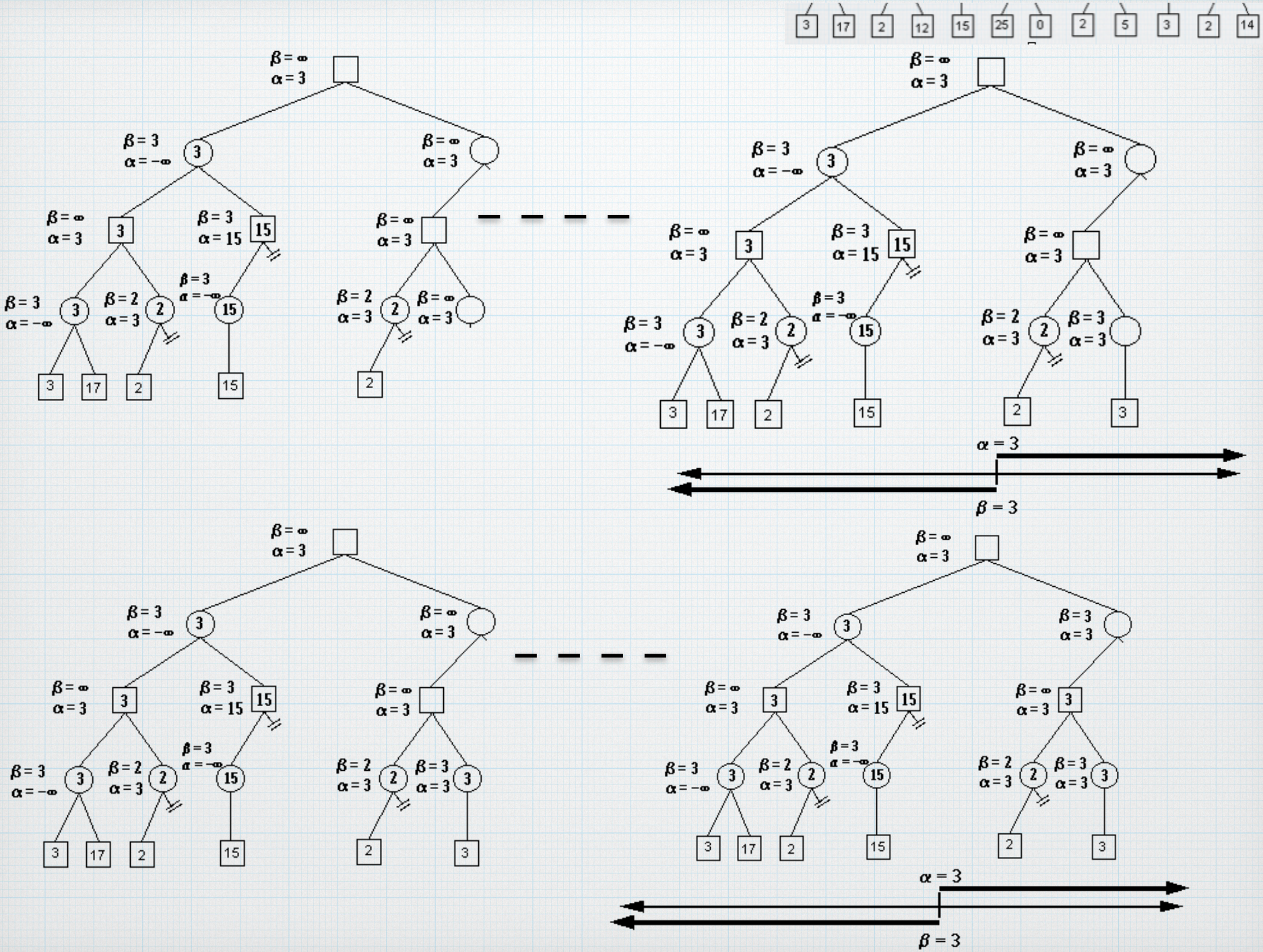


Comme vous pouvez le voir sur la droite numérique, il n'y a plus de chevauchement entre les régions délimitées par alpha et bêta. En gros, nous avons découvert que la seule façon de trouver un chemin de solution à ce nœud est de trouver un nœud enfant avec une valeur à la fois supérieure à 3 et inférieure à 2. Comme c'est impossible, nous pouvons arrêter d'évaluer les enfants de ce nœud, et renvoient la valeur bêta (2) comme valeur du nœud.


3 17 2 12 15 25 0 2 5 3 2 14
β = ∞ α = −∞
β = ∞ α = −∞
β = ∞ α = 3
β = 3 α = −∞
β = 2 α = 3
3 17 2
β = ∞ α = −∞
β = 3 α = −∞
β = ∞ α = 3
β = 3 α = −∞
β = 2 α = 3
3 17 2
β = ∞ α = −∞
β = 3 α = −∞
β = ∞ α = 3
β = 3 α = −∞
β = 3 α = −∞
β = 2 α = 3
3 17 2
β = ∞ α = −∞
β = 3 α = −∞
β = ∞ α = 3
β = 3 α = −∞
β = 3 α = −∞
β = 2 α = 3
β = 3 α = −∞
3 17 2
β = ∞ α = −∞
β = 3 α = −∞
β = ∞ α = 3
β = 3 α = −∞
β = 3 α = −∞
β = 2 α = 3
β = 3 α = −∞
3 17 2 15

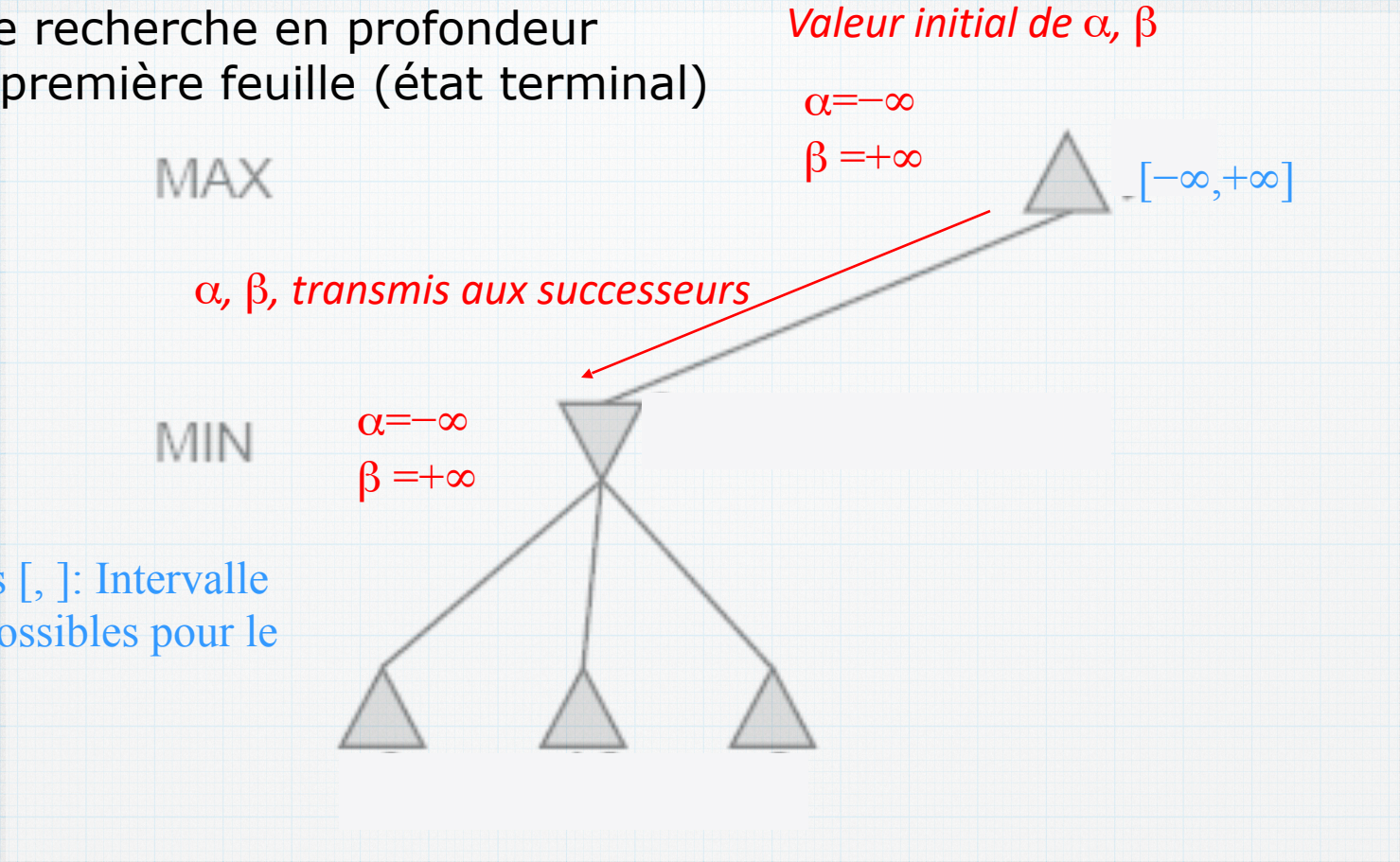

3 17 2 12 15 25 0 2 5 3 2 14
β = ∞
α = −∞
β = 3
α = −∞
β = ∞
α = 3
β = 3
α = −∞
β = 3
α = −∞
β = 2
α = 3
β = 3
α = −∞
β = 3
α = 15
α = 15
β = 3
3 17 2 15


3 17 2 12 15 25 0 2 5 3 2 14
β = ∞
α = 3
β = 3
α = −∞
β = ∞
α = 3
β = ∞
α = 3
β = 3
α = 15
β = 3
α = −∞
β = 2
α = 3
β = 3
α = −∞
3 17 2 15
α = 3
β = 2


3 17 2 12 15 25 0 2 5 3 2 14
β = ∞ α = 3
β = 3 α = −∞
β = ∞ α = 3
β = ∞ α = 3
β = 3 α = 15
β = ∞ α = 3
β = 3 α = −∞
β = 2 α = 3
β = 3 α = −∞
β = 2 α = 3
β = ∞ α = 3
3 17 2 15 2
α = 3
β = 3
β = 3 α = −∞
β = ∞ α = 3
β = ∞ α = 3
β = 3 α = 15
β = ∞ α = 3
β = 3 α = −∞
β = 2 α = 3
β = 3 α = −∞
β = 2 α = 3
β = 3 α = 3
3 17 2 15 2 3
α = 3
β = 3
β = ∞ α = 3
β = 3 α = −∞
β = ∞ α = 3
β = ∞ α = 3
β = 3 α = 15
β = ∞ α = 3
β = 3 α = −∞
β = 2 α = 3
β = 3 α = −∞
β = 2 α = 3
β = 3 α = 3
3 17 2 15 2 3
β = ∞ α = 3
β = 3 α = −∞
β = 3 α = 3
β = ∞ α = 3
β = 3 α = 15
β = ∞ α = 3
β = 3 α = −∞
β = 2 α = 3
β = 3 α = −∞
β = 2 α = 3
β = 3 α = 3
3 17 2 15 2 3
α = 3
β = 3

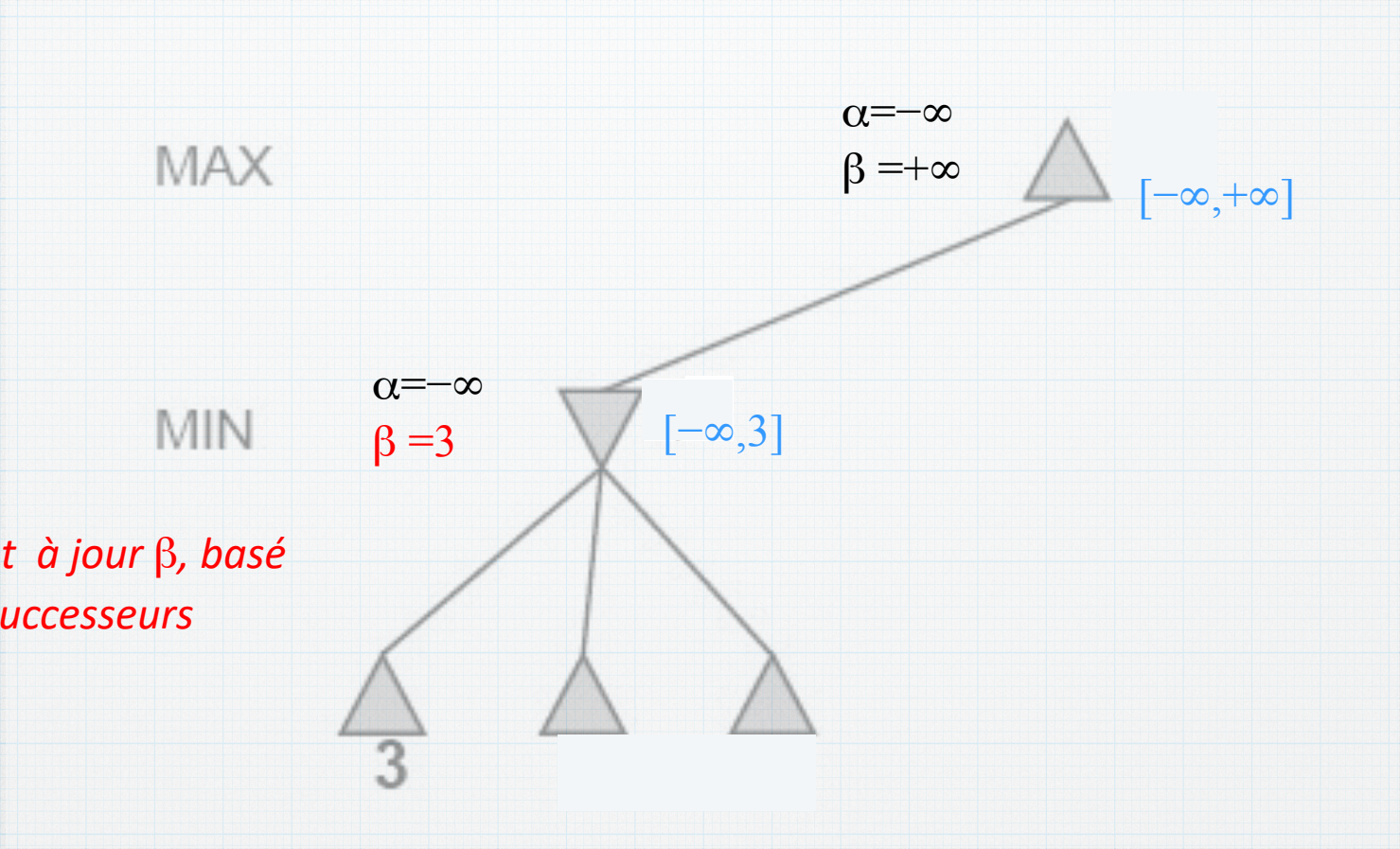
Exemple d'*Alpha-beta pruning*

On fait une recherche en profondeur jusqu'à la première feuille (état terminal)



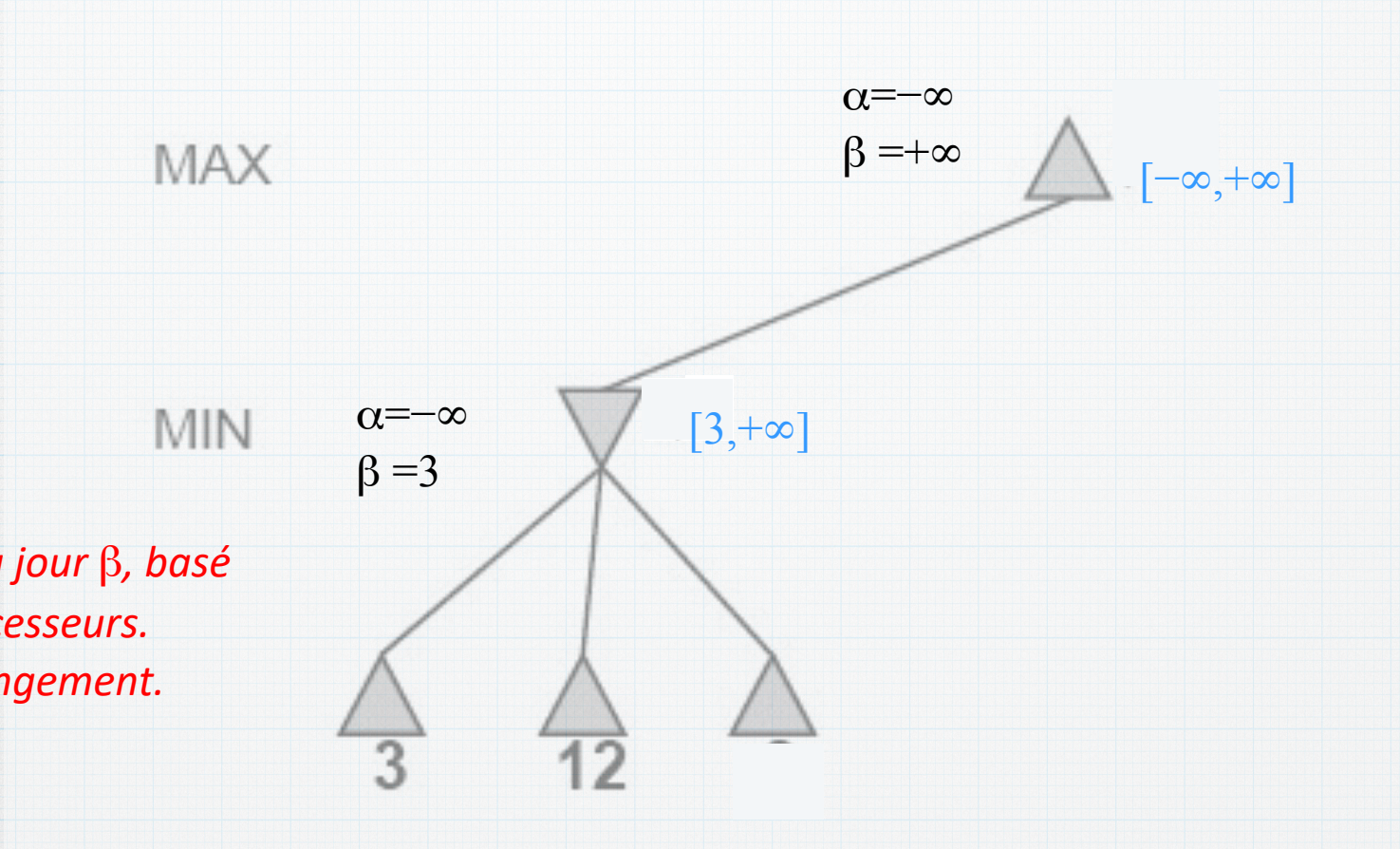
Entre croches [,]: Intervalle des valeurs possibles pour le nœud visité.

Exemple d'*Alpha-beta pruning*



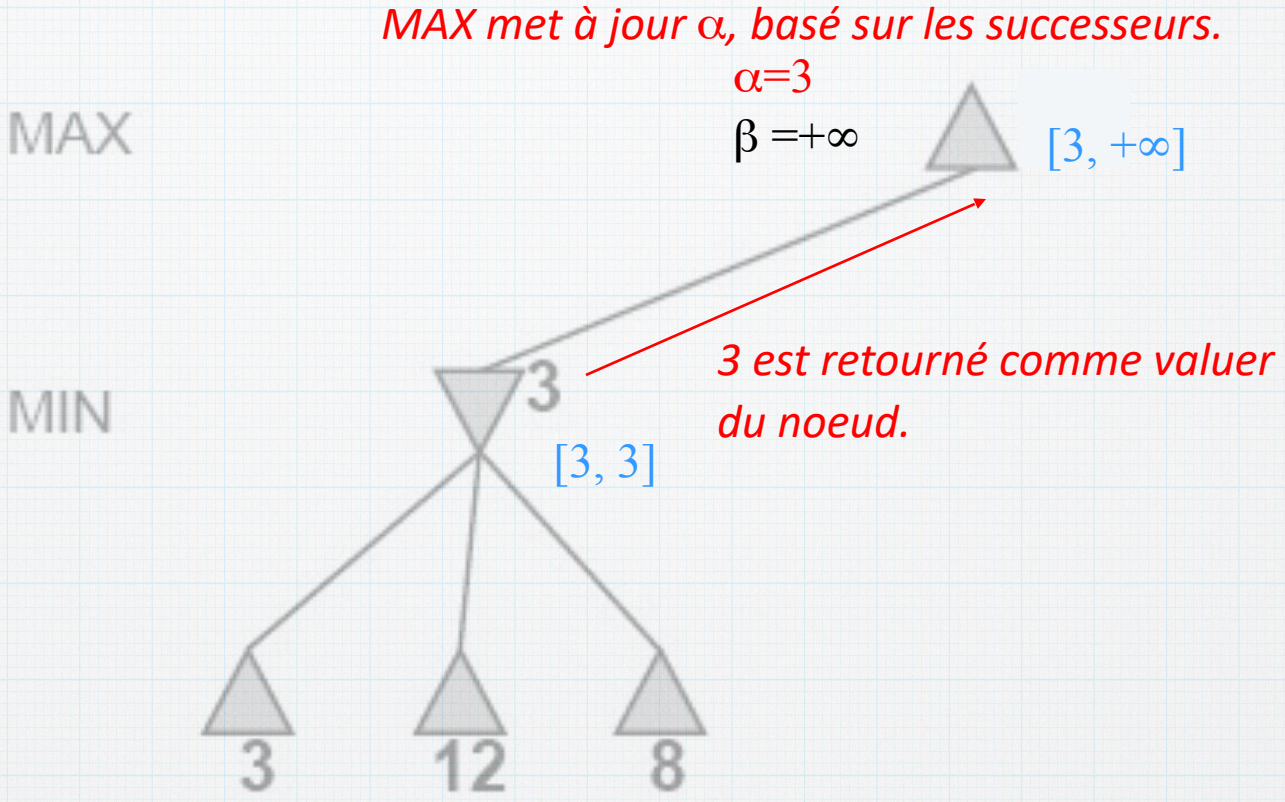
MIN met à jour β, basé sur les successeurs

Exemple d'*Alpha-beta pruning*

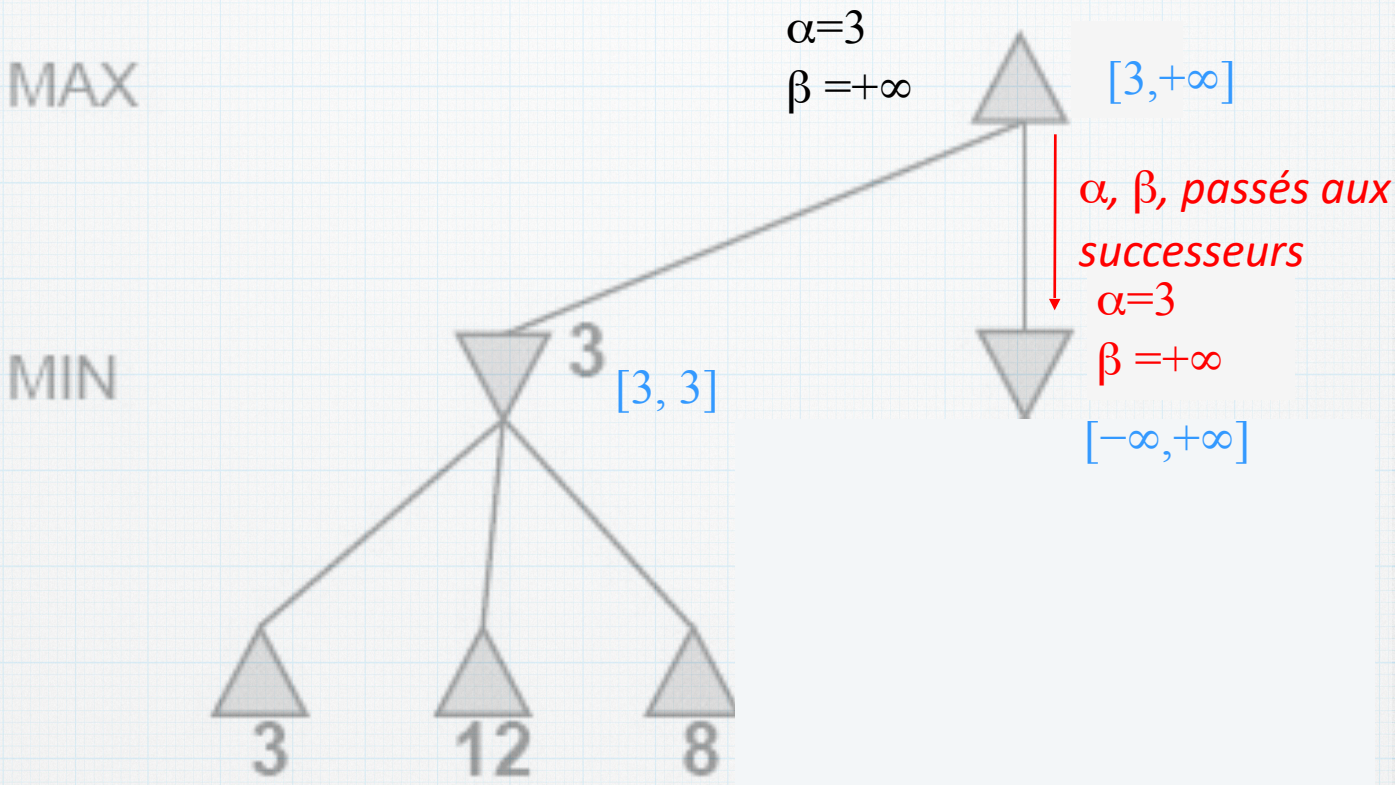


MIN met à jour β, basé sur les successeurs. Aucun changement.

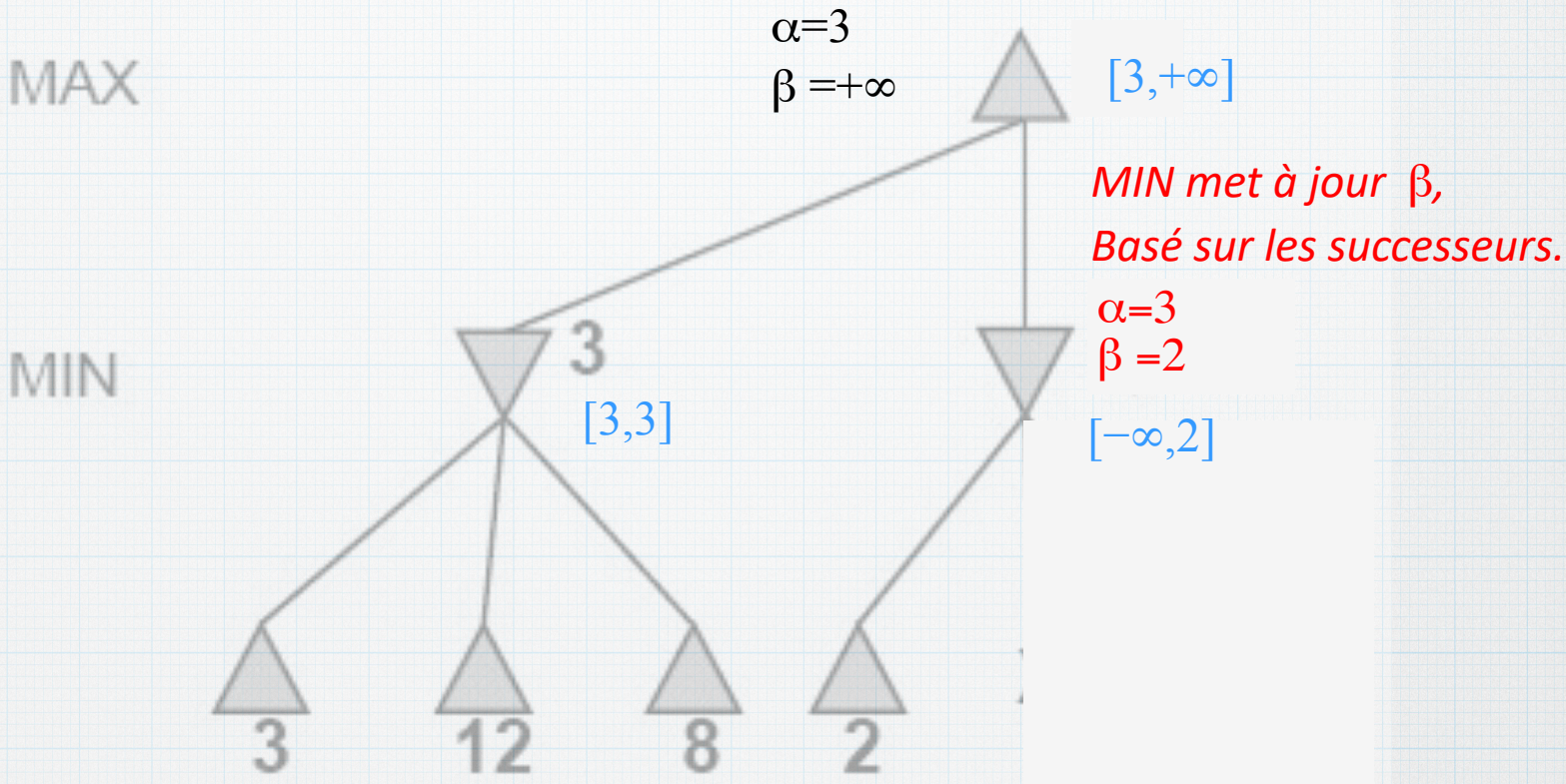
Exemple d'*d'Alpha-beta pruning*



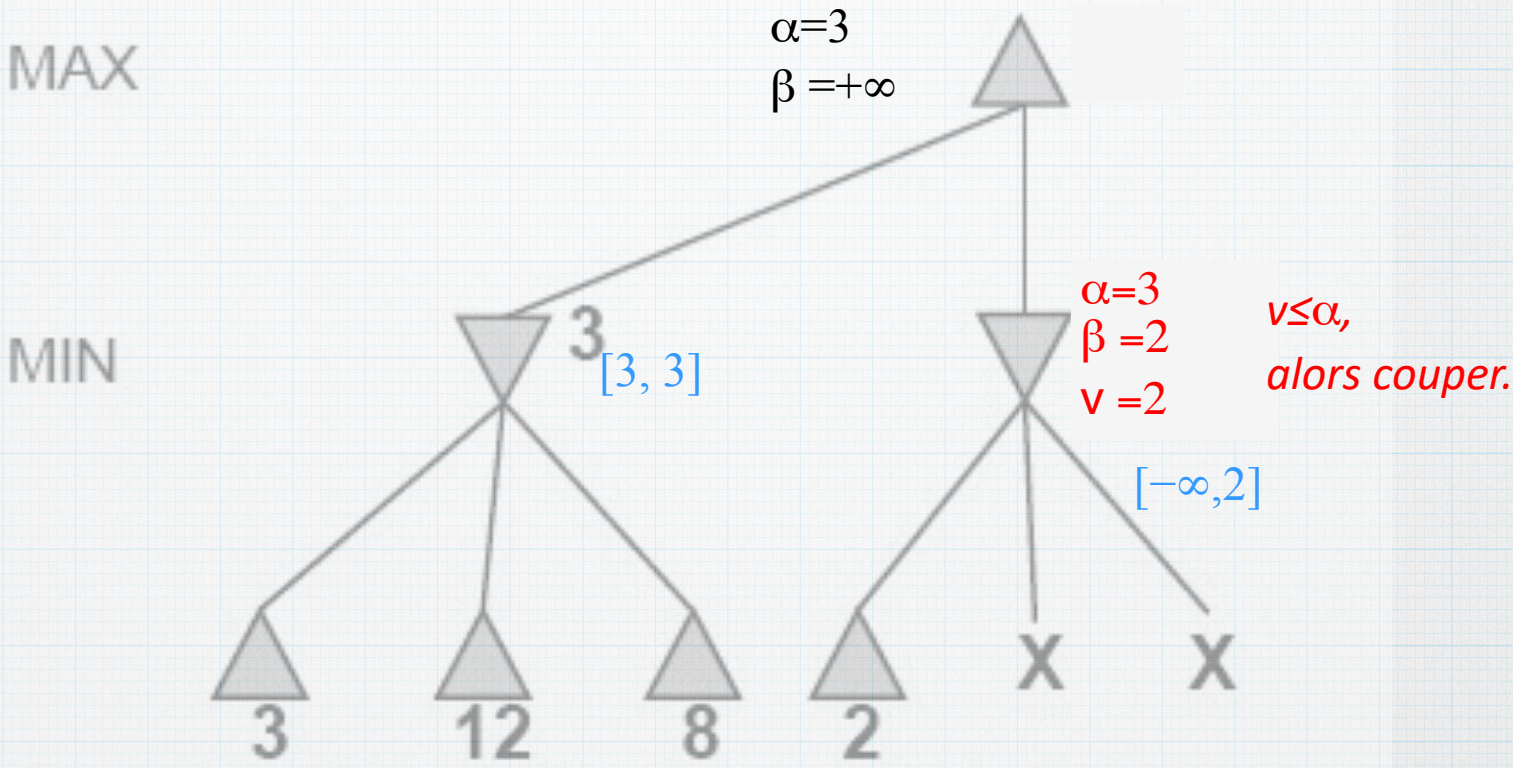
Exemple d'*d'Alpha-beta pruning*



Exemple d'*d'Alpha-beta pruning*



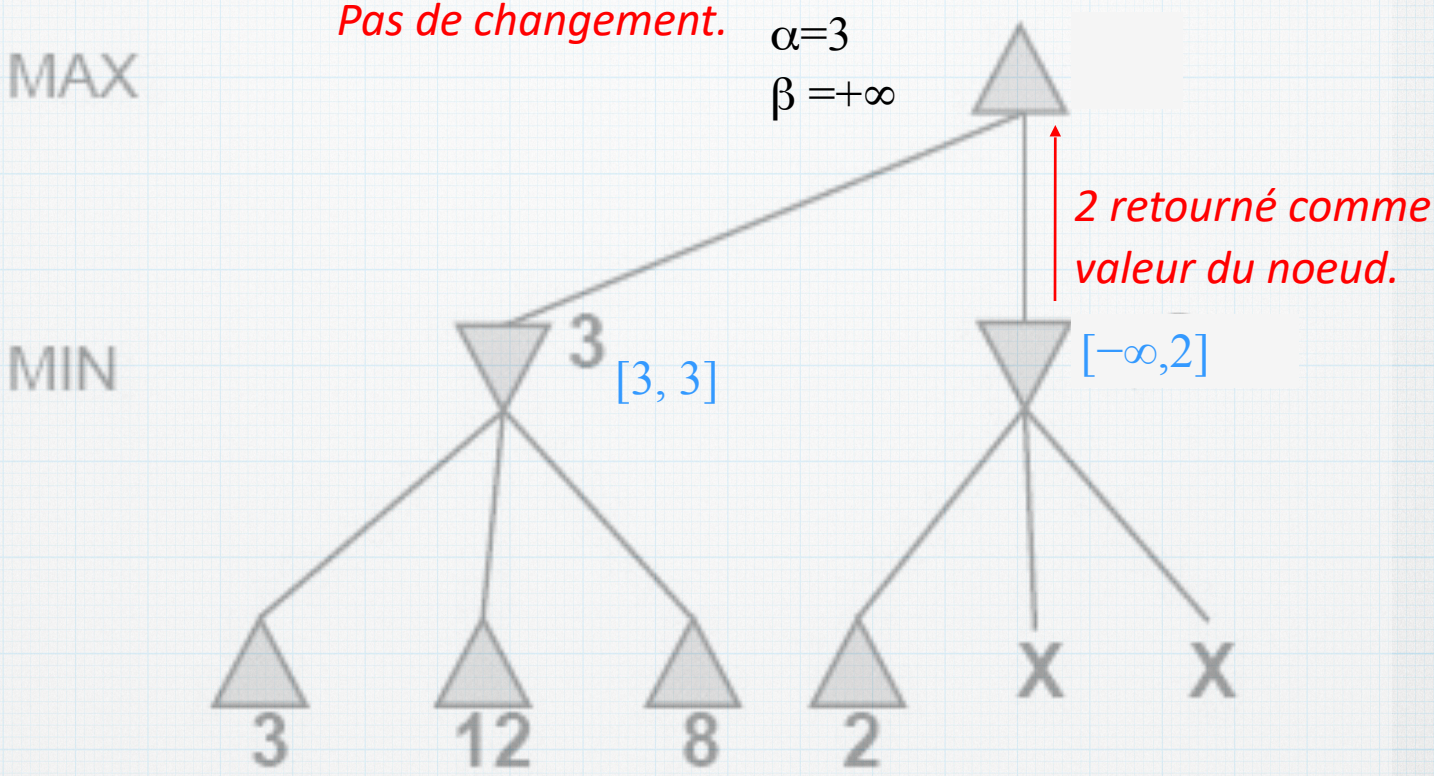
Exemple d'*d'Alpha-beta pruning*



Exemple d'*d'Alpha-beta pruning*

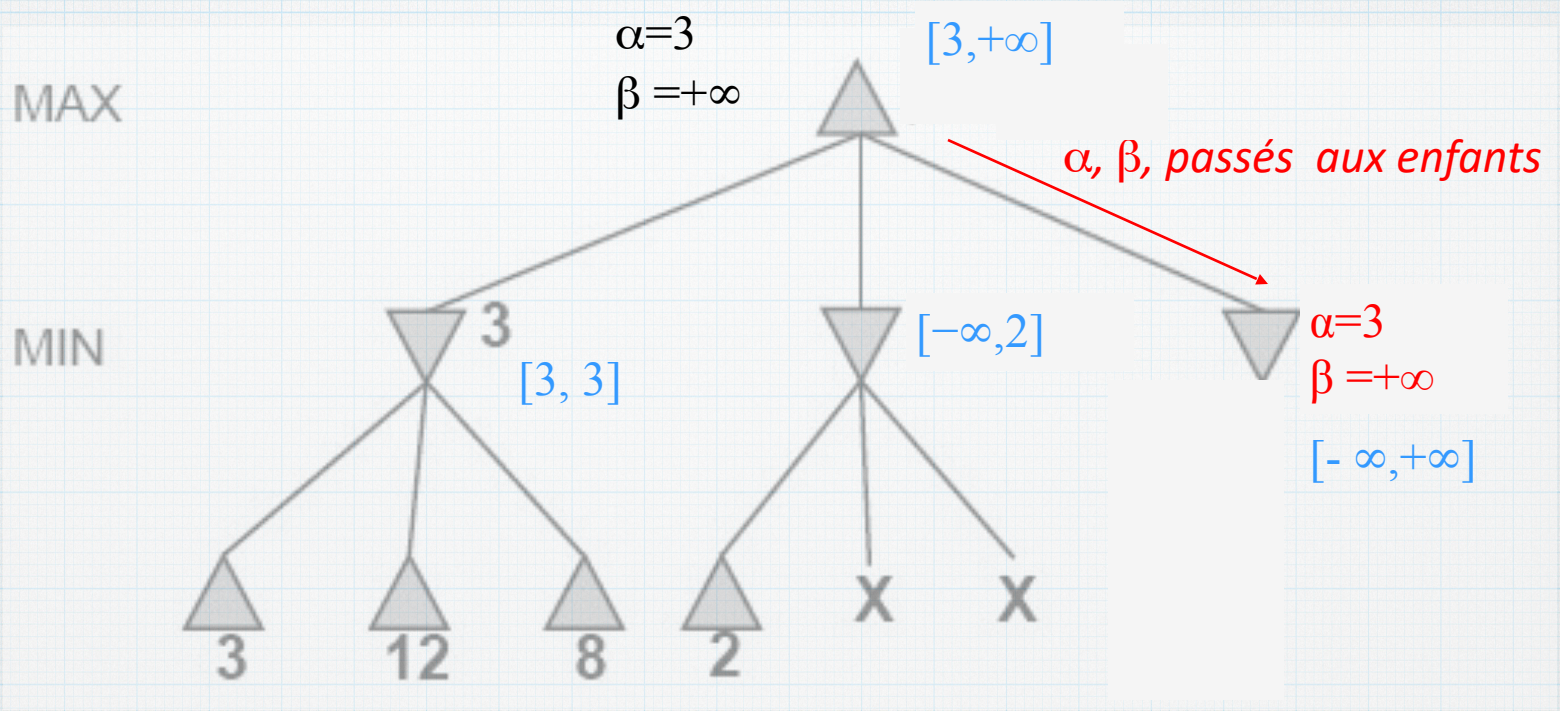
MAX met à jour α, basé sur les enfants.

Pas de changement.

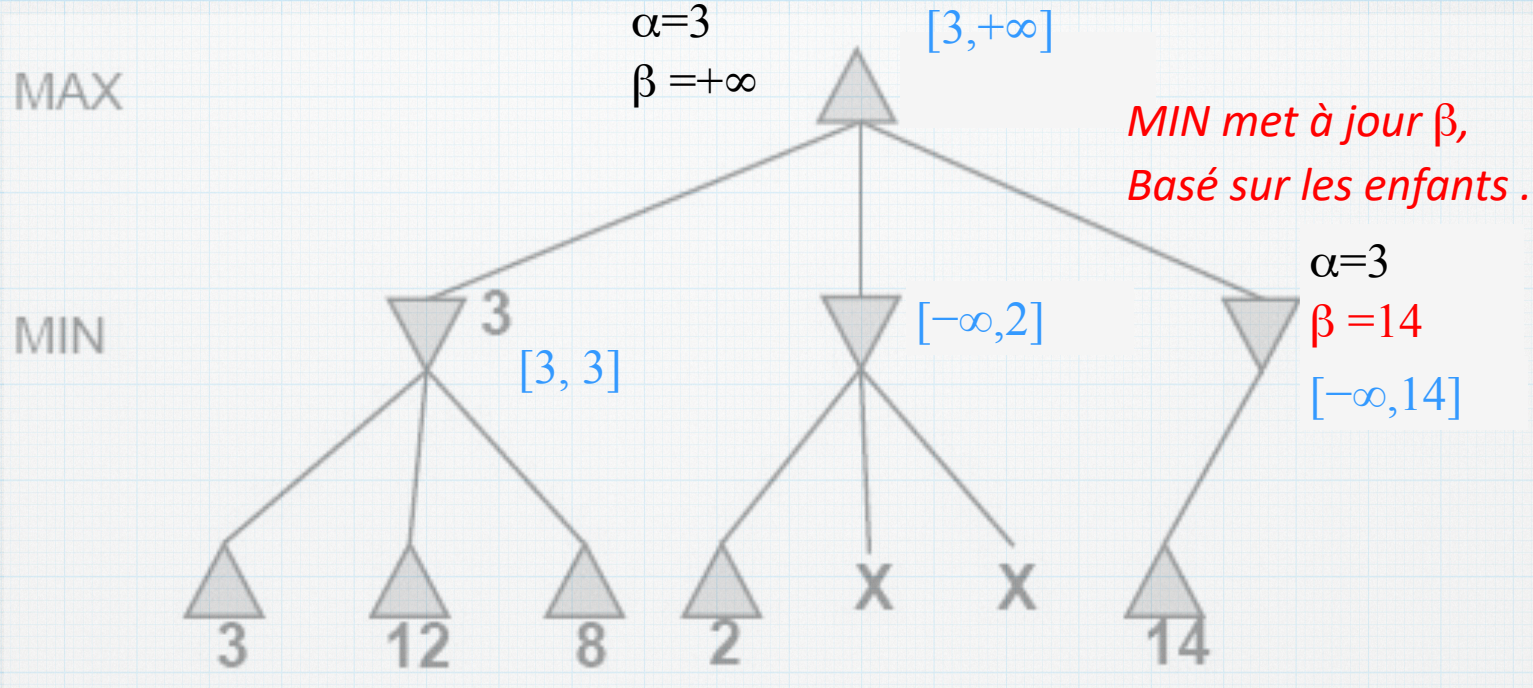


2 retourné comme valeur du noeud.

Exemple d'*d'Alpha-beta pruning*

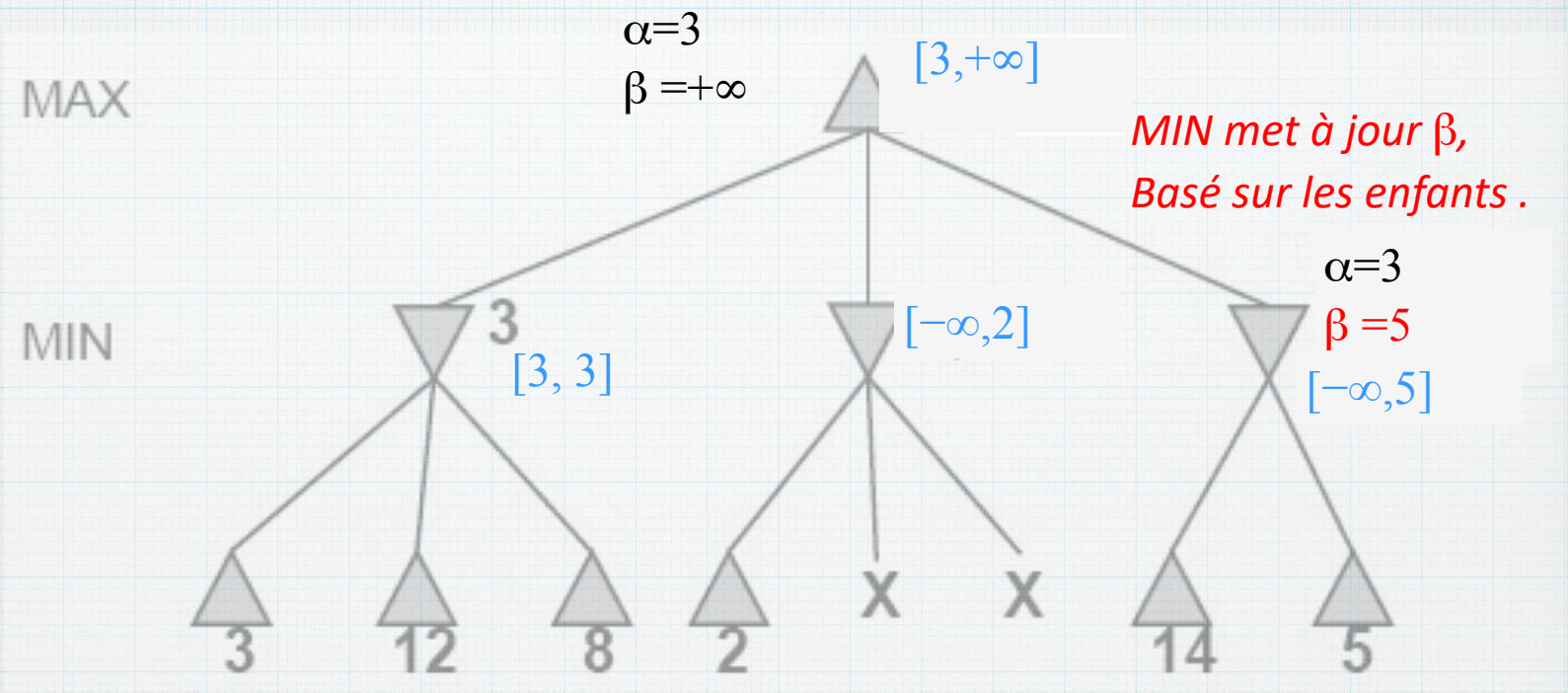


Exemple d'*d'Alpha-beta pruning*

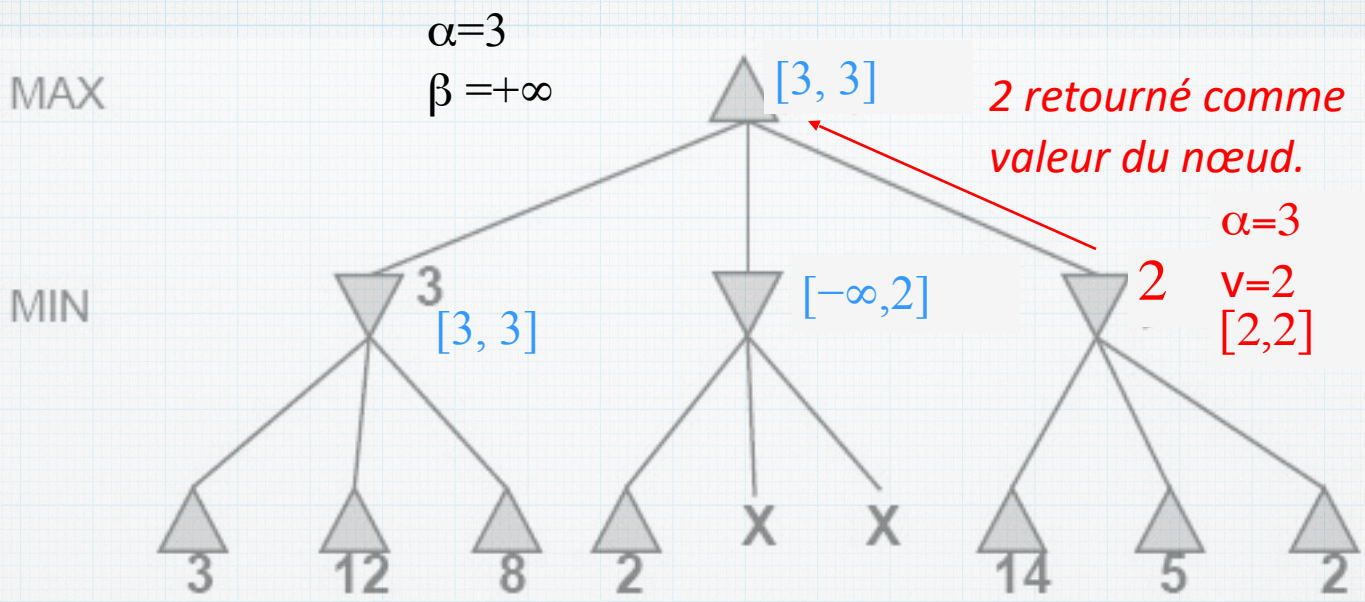


MIN met à jour β,
Basé sur les enfants .

Exemple d'*d'Alpha-beta pruning*

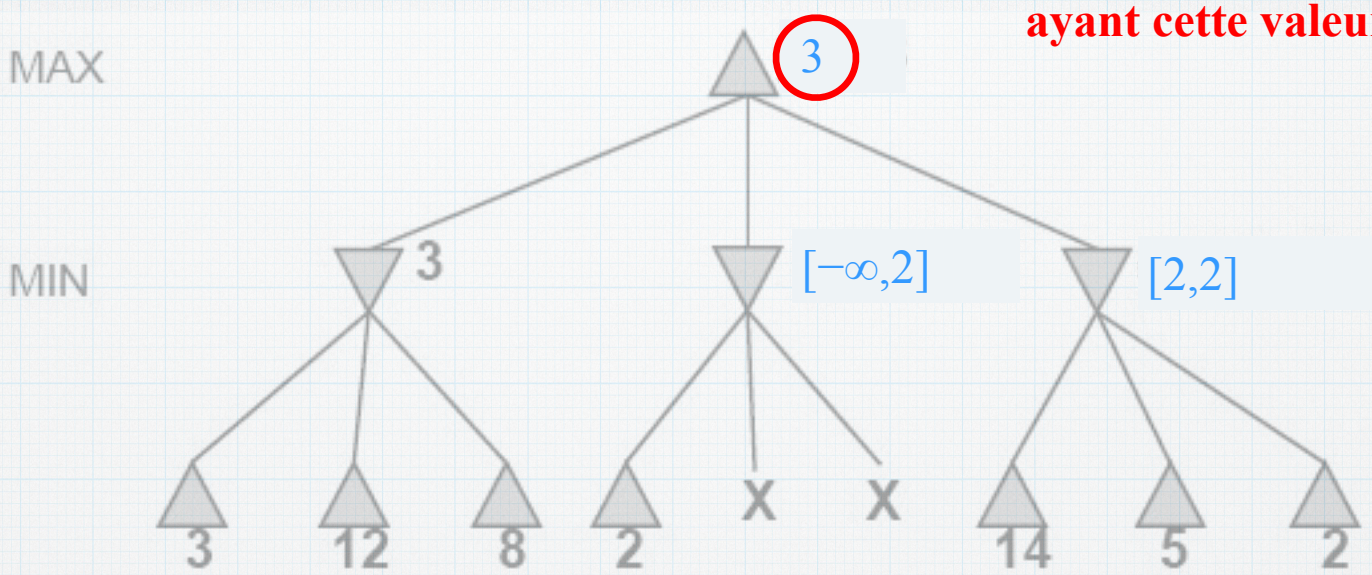


Exemple d'*d'Alpha-beta pruning*



Exemple d'*d'Alpha-beta pruning*

Max calcule la valeur du nœud, et se déplace vers le nœud ayant cette valeur!



Algorithme *alpha-beta pruning*

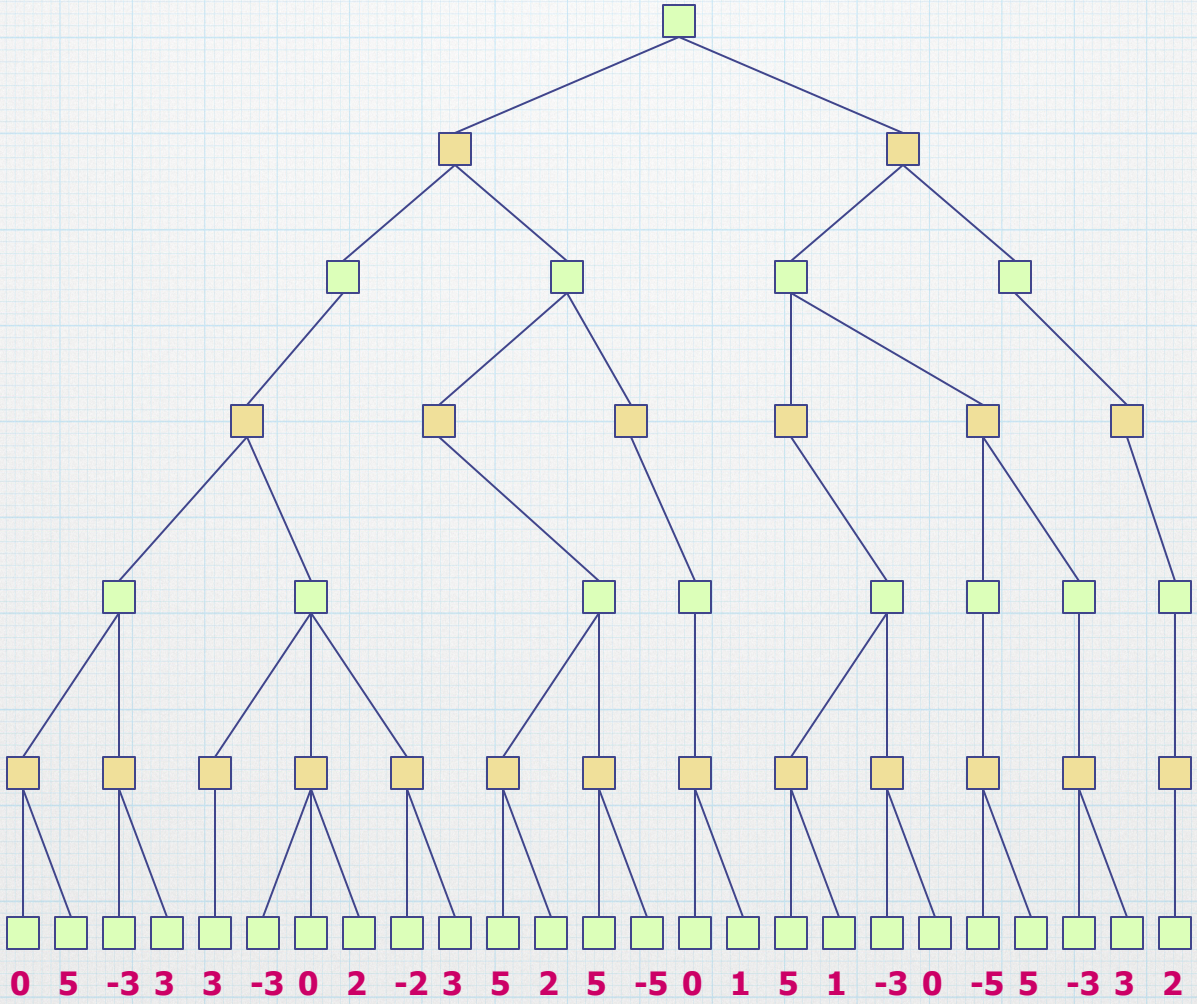
```
function ALPHA-BETA-SEARCH(state) returns an action
    inputs: state, current state in game

    v ← MAX-VALUE(state, −∞, +∞)
    return the action in SUCCESSORS(state) with value v
```

```
function MAX-VALUE(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for MAX along the path to state
            β, the value of the best alternative for MIN along the path to state

    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← −∞
    for a, s in SUCCESSORS(state) do
        v ← MAX(v, MIN-VALUE(s, α, β))
        if v ≥ β then return v
        α ← MAX(α, v)
    return v
```

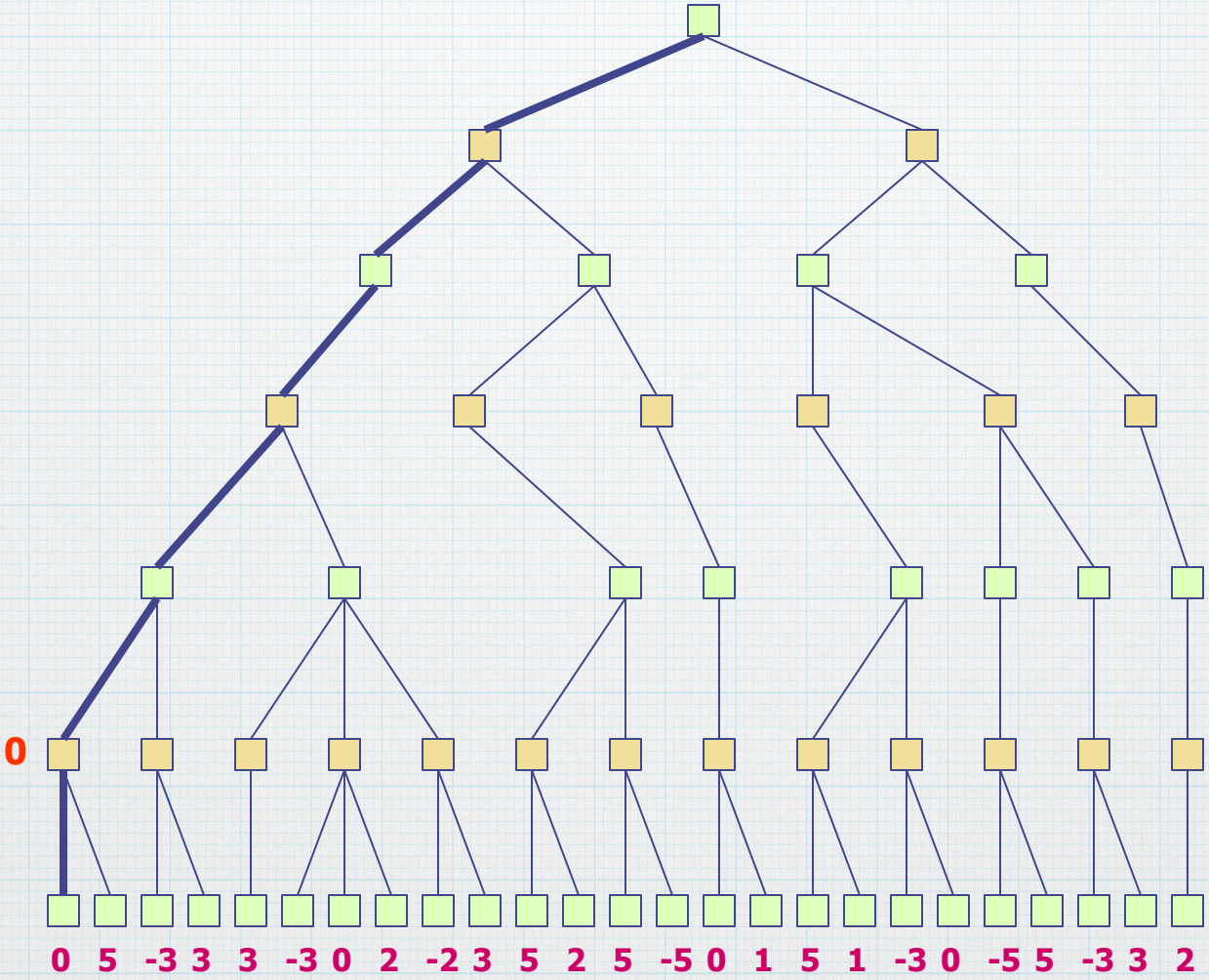
Algorithme *alpha-beta pruning (suite)*

```
function MIN-VALUE(state, α, β) returns a utility value
    inputs: state, current state in game
            α, the value of the best alternative for MAX along the path to state
            β, the value of the best alternative for MIN along the path to state

    if TERMINAL-TEST(state) then return UTILITY(state)
    v ← +∞
    for a, s in SUCCESSORS(state) do
        v ← MIN(v, MAX-VALUE(s, α, β))
        if v ≤ α then return v
        β ← MIN(β, v)
    return v
```

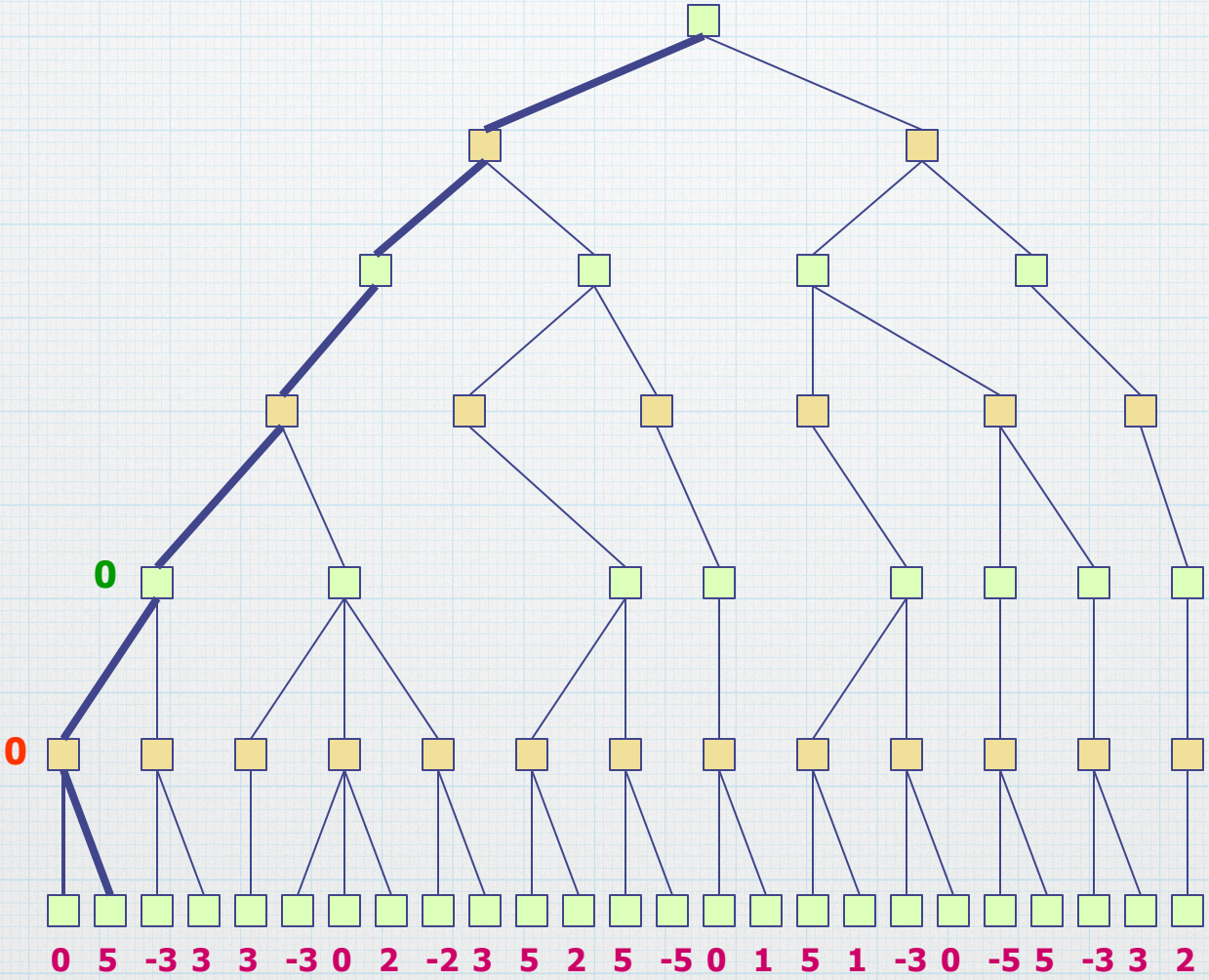
Exemple Alpha-Beta



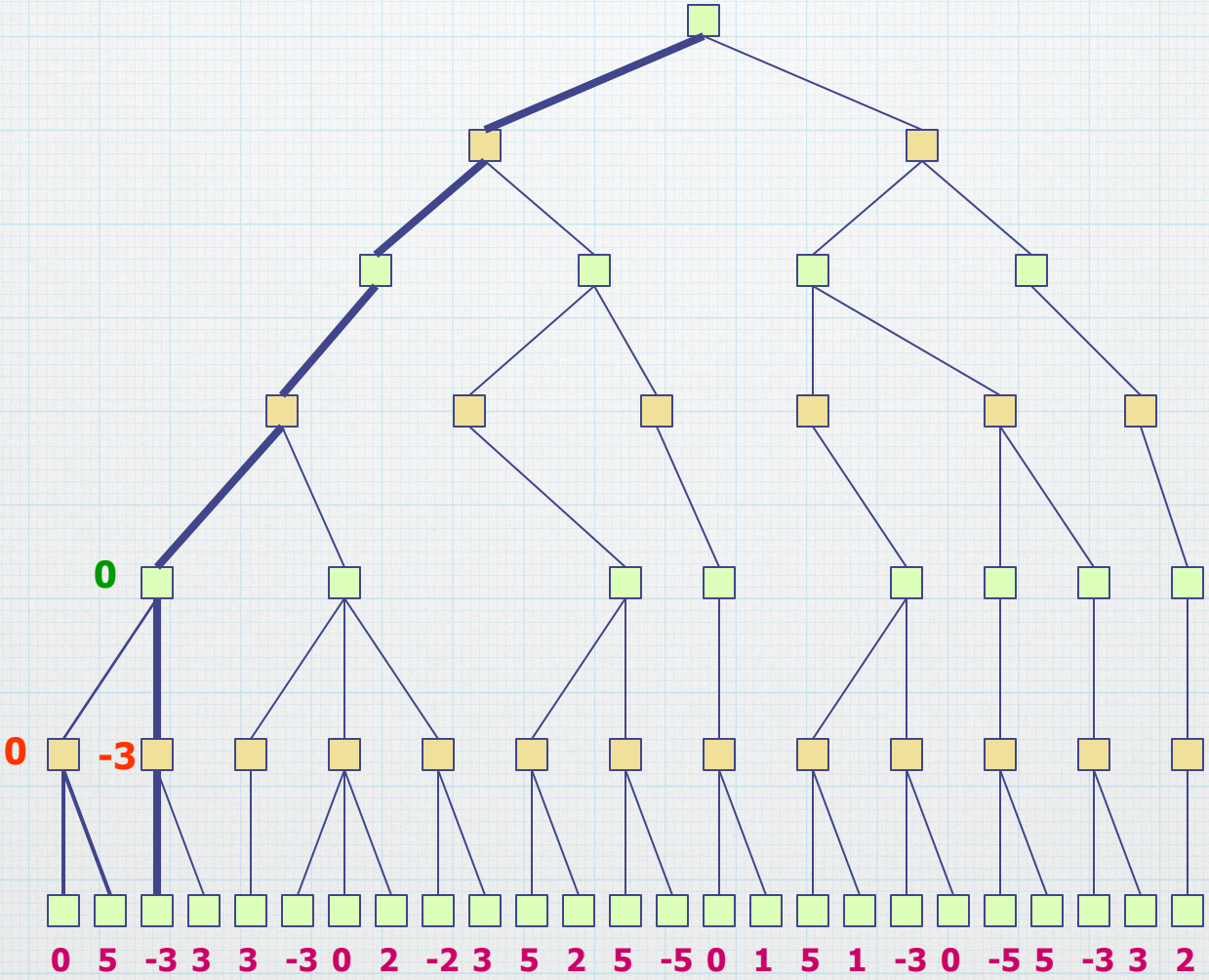
Exemple Alpha-Beta



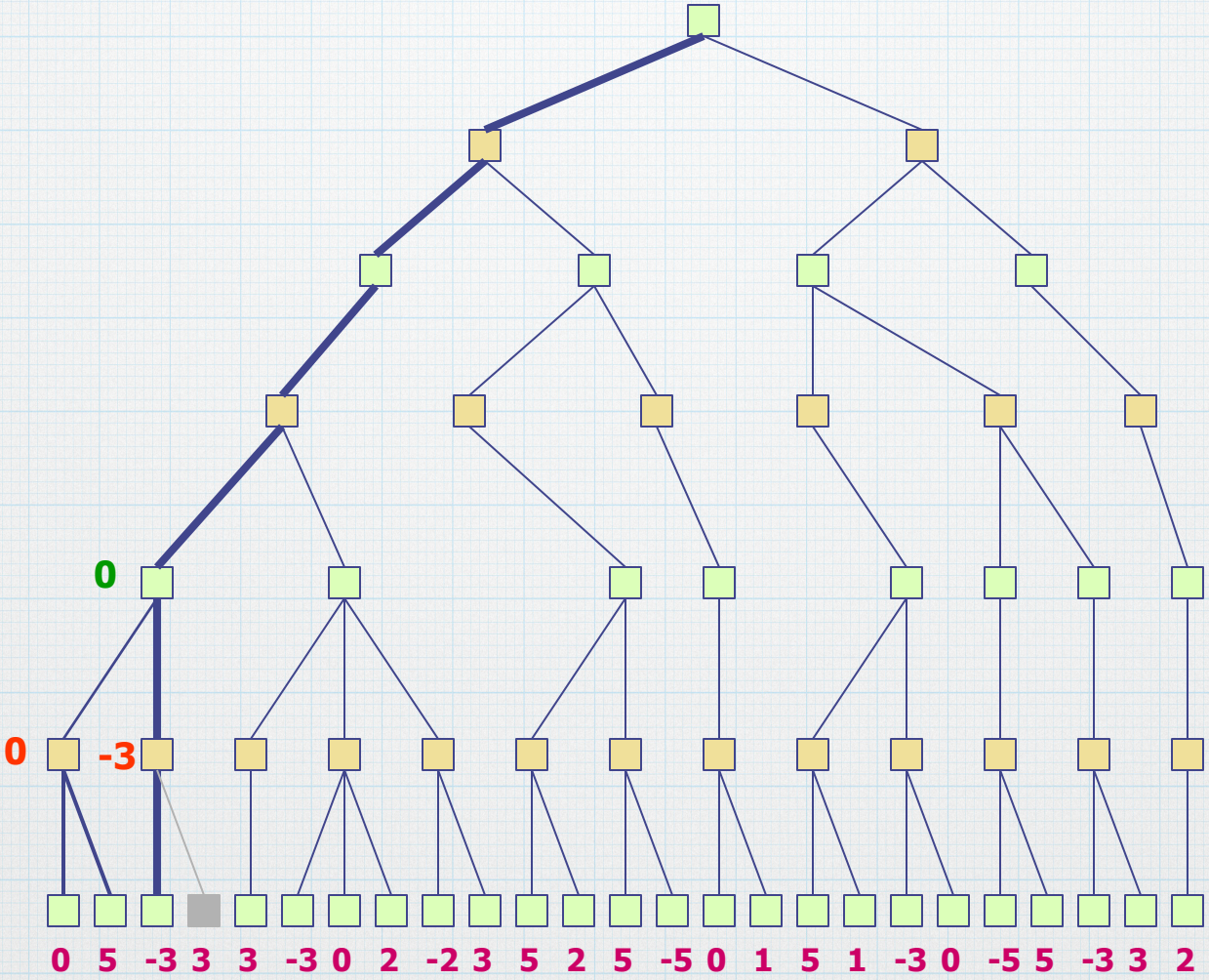
Exemple Alpha-Beta



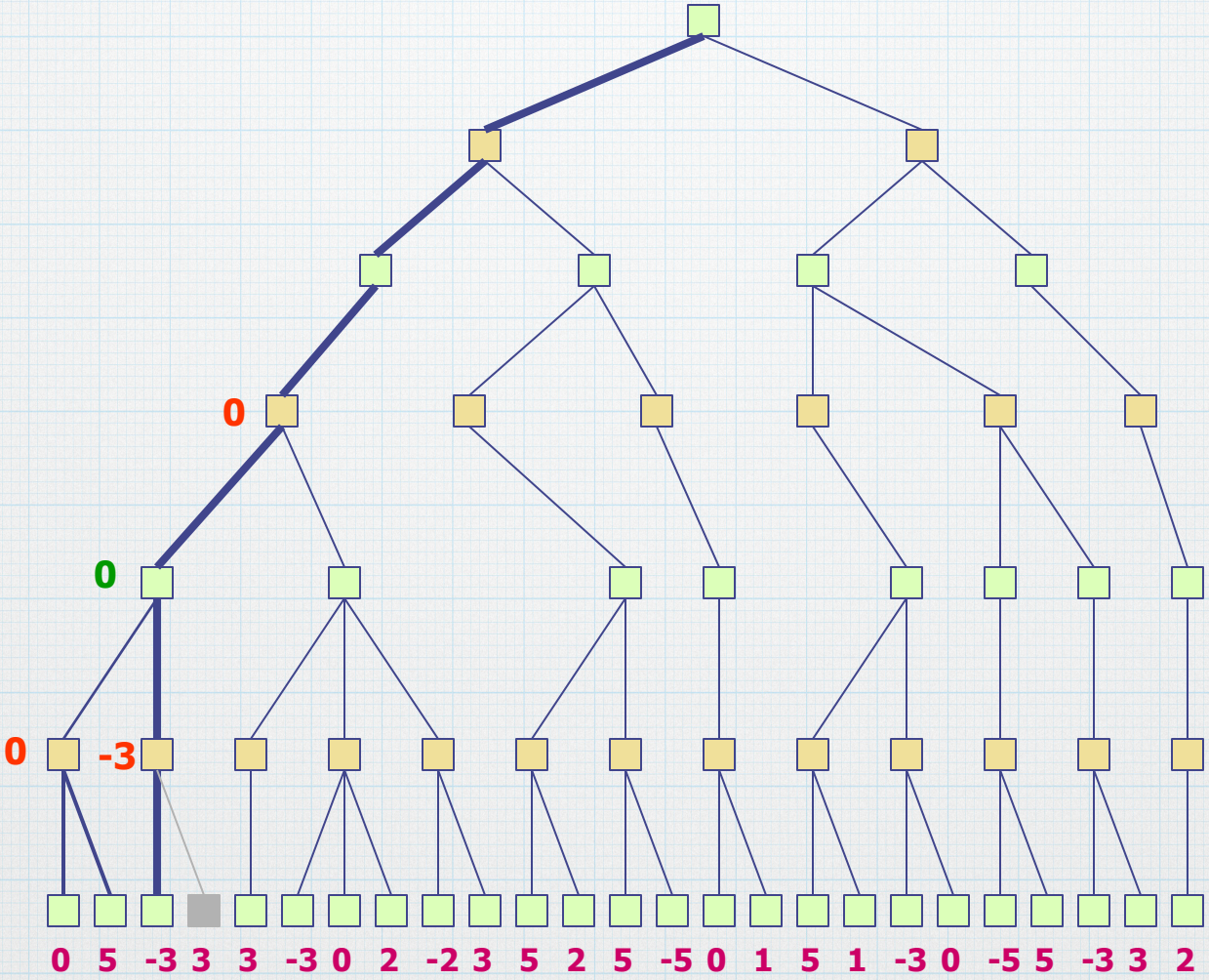
Exemple Alpha-Beta



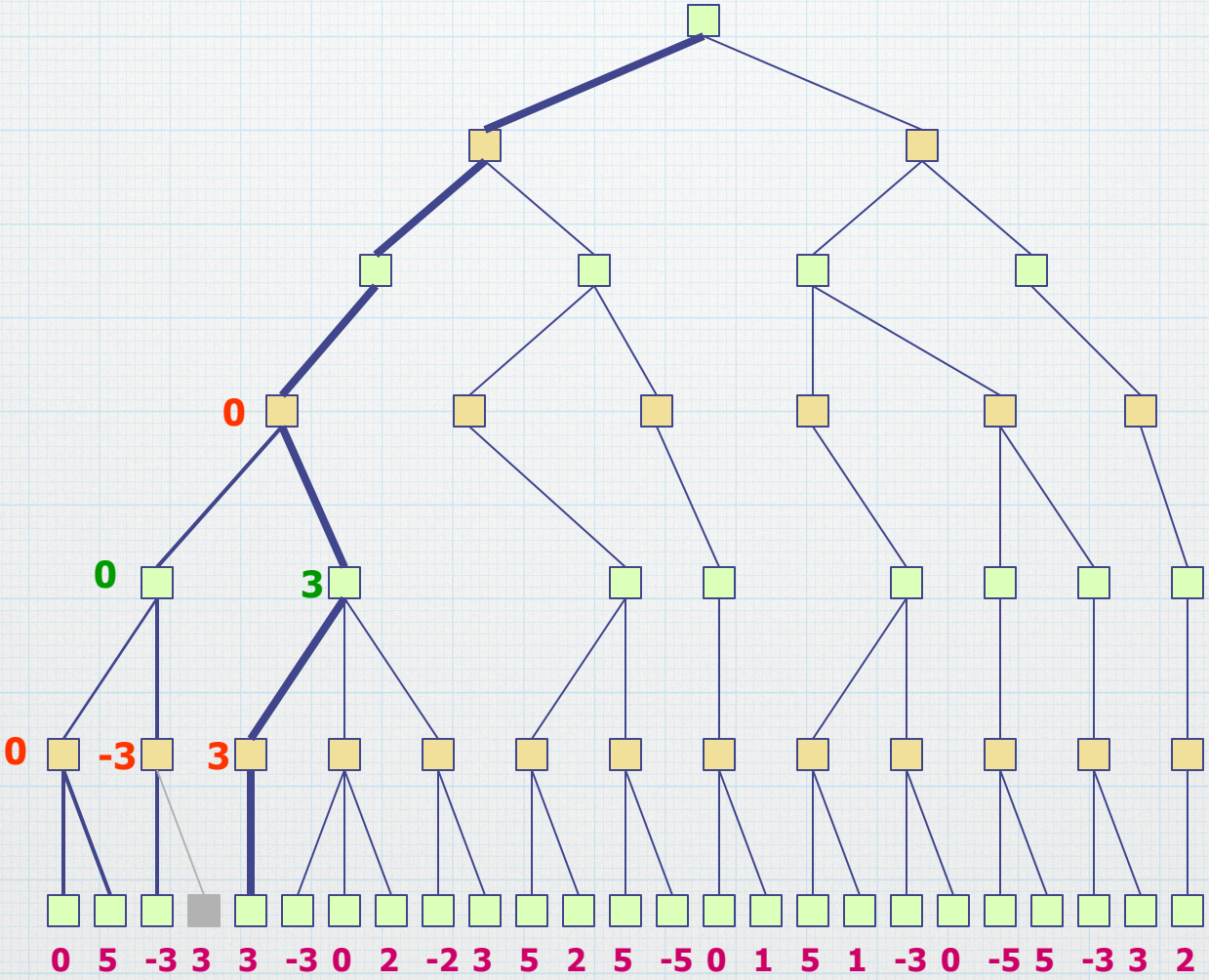
Exemple Alpha-Beta



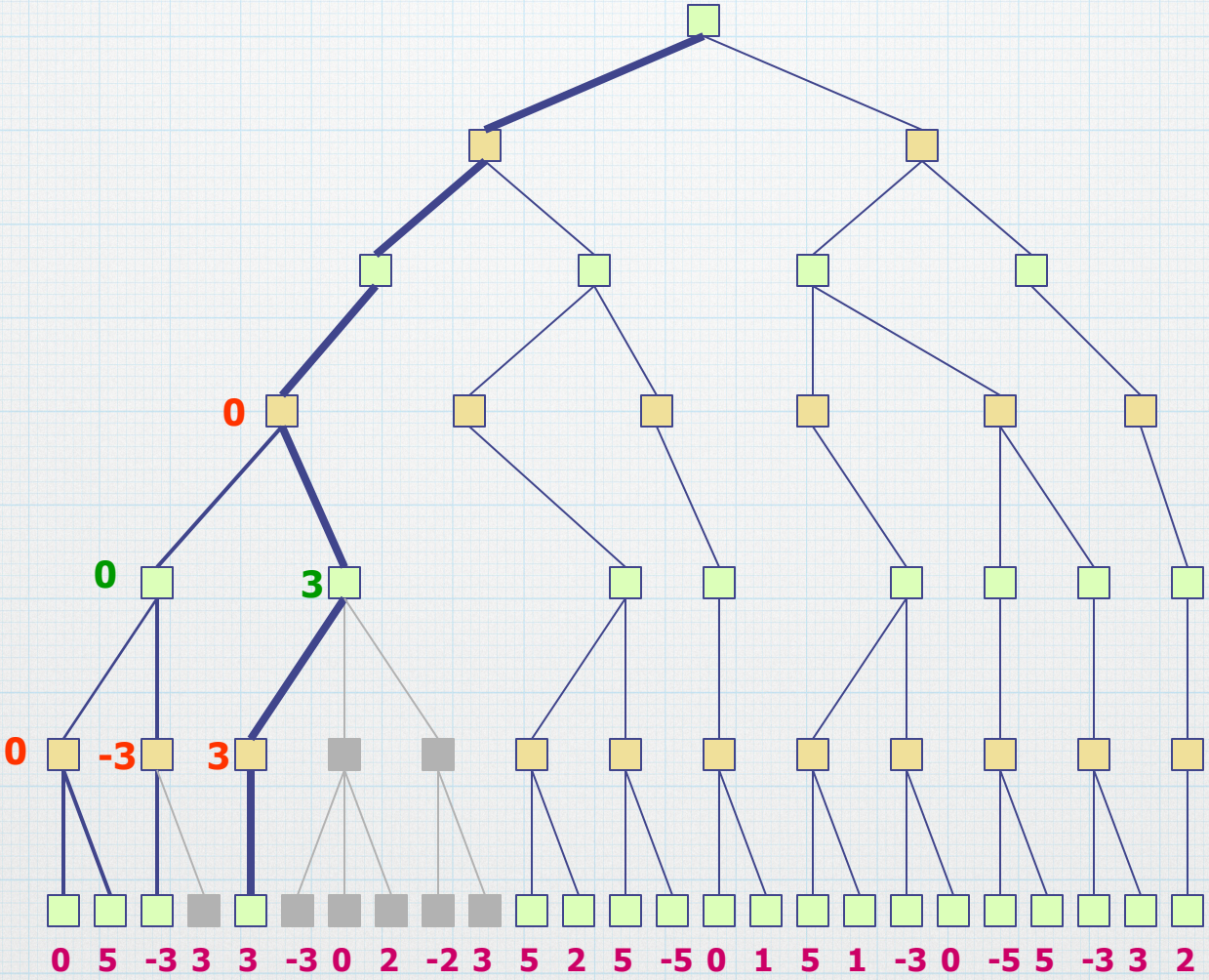
Exemple Alpha-Beta



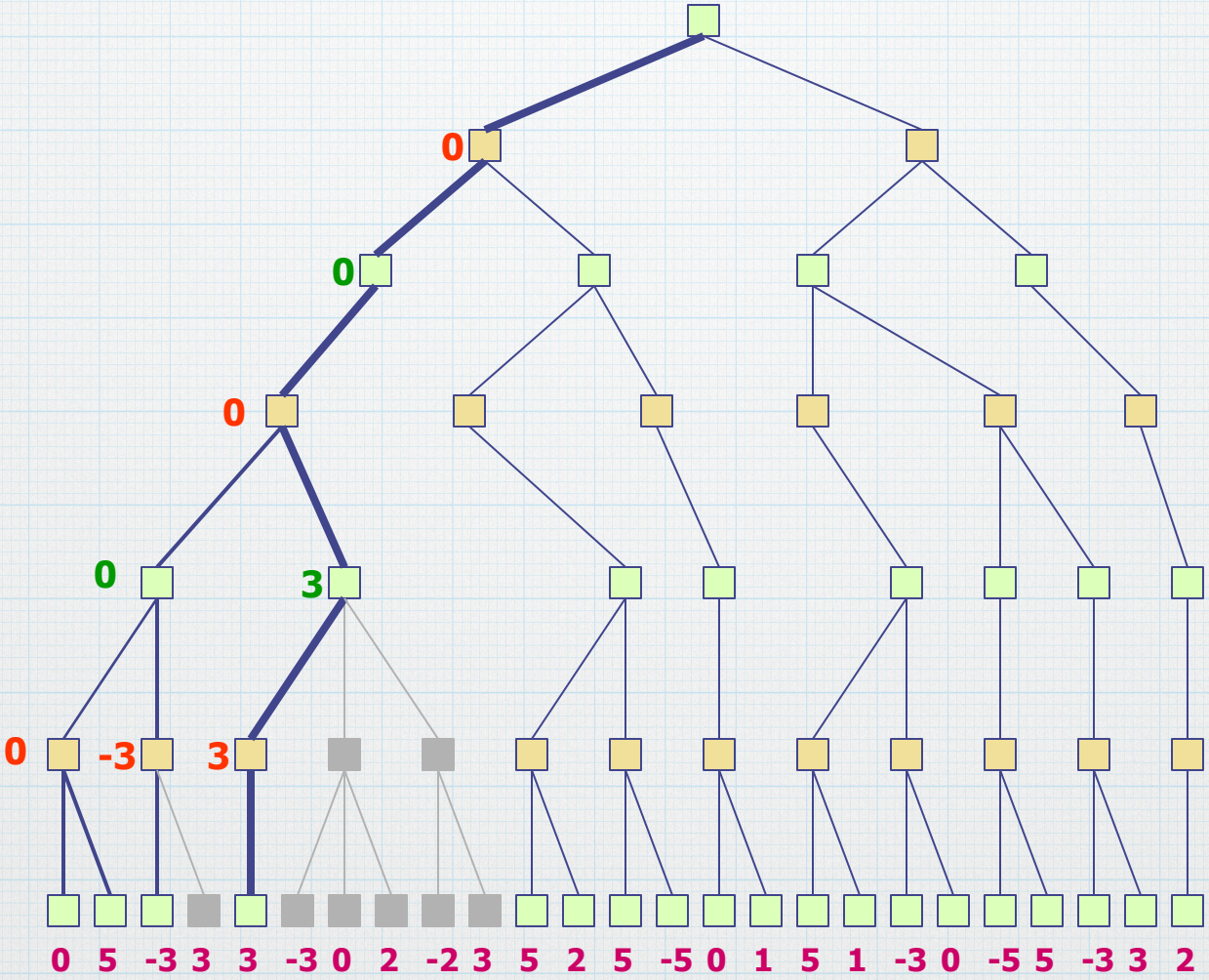
Exemple Alpha-Beta



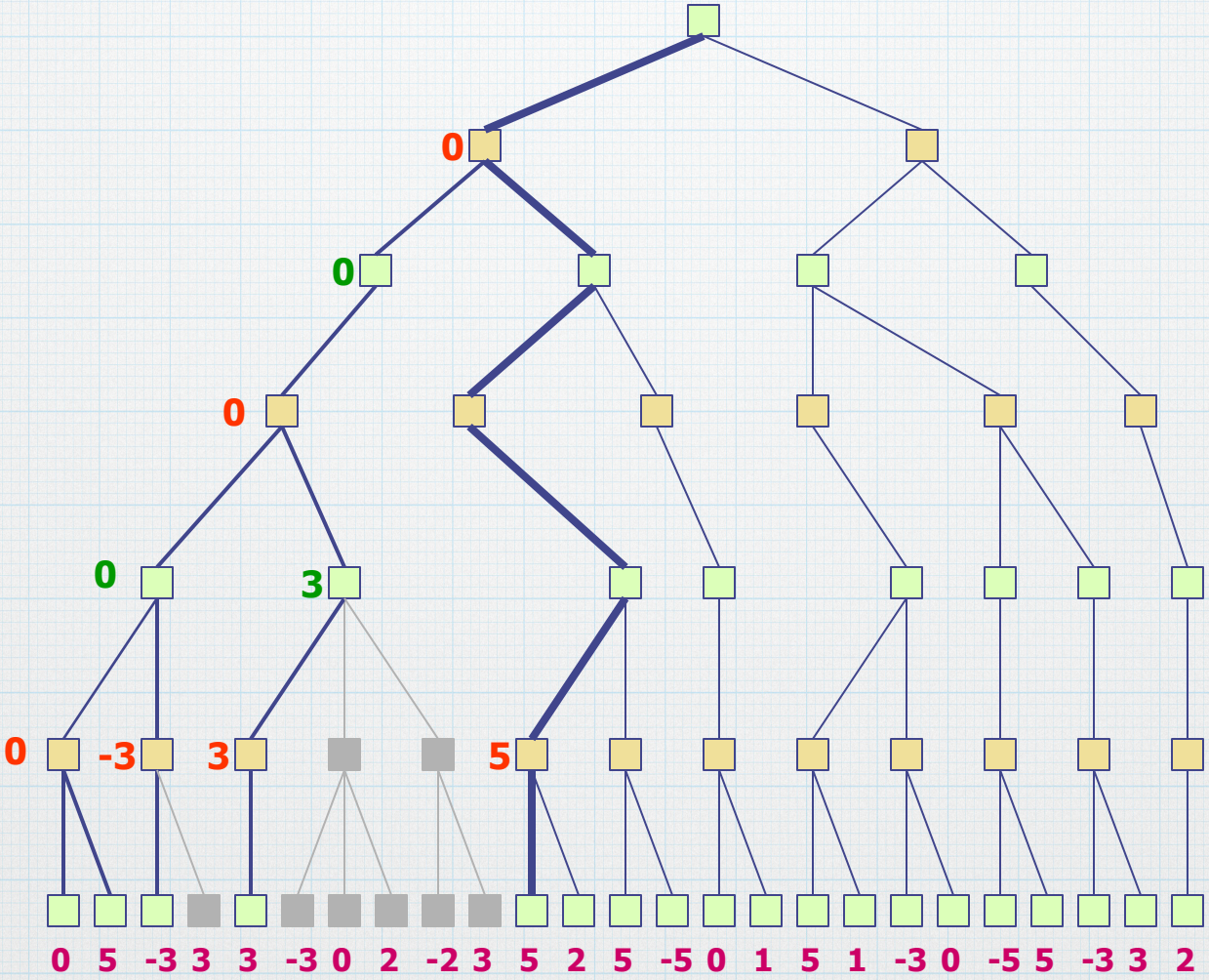
Exemple Alpha-Beta



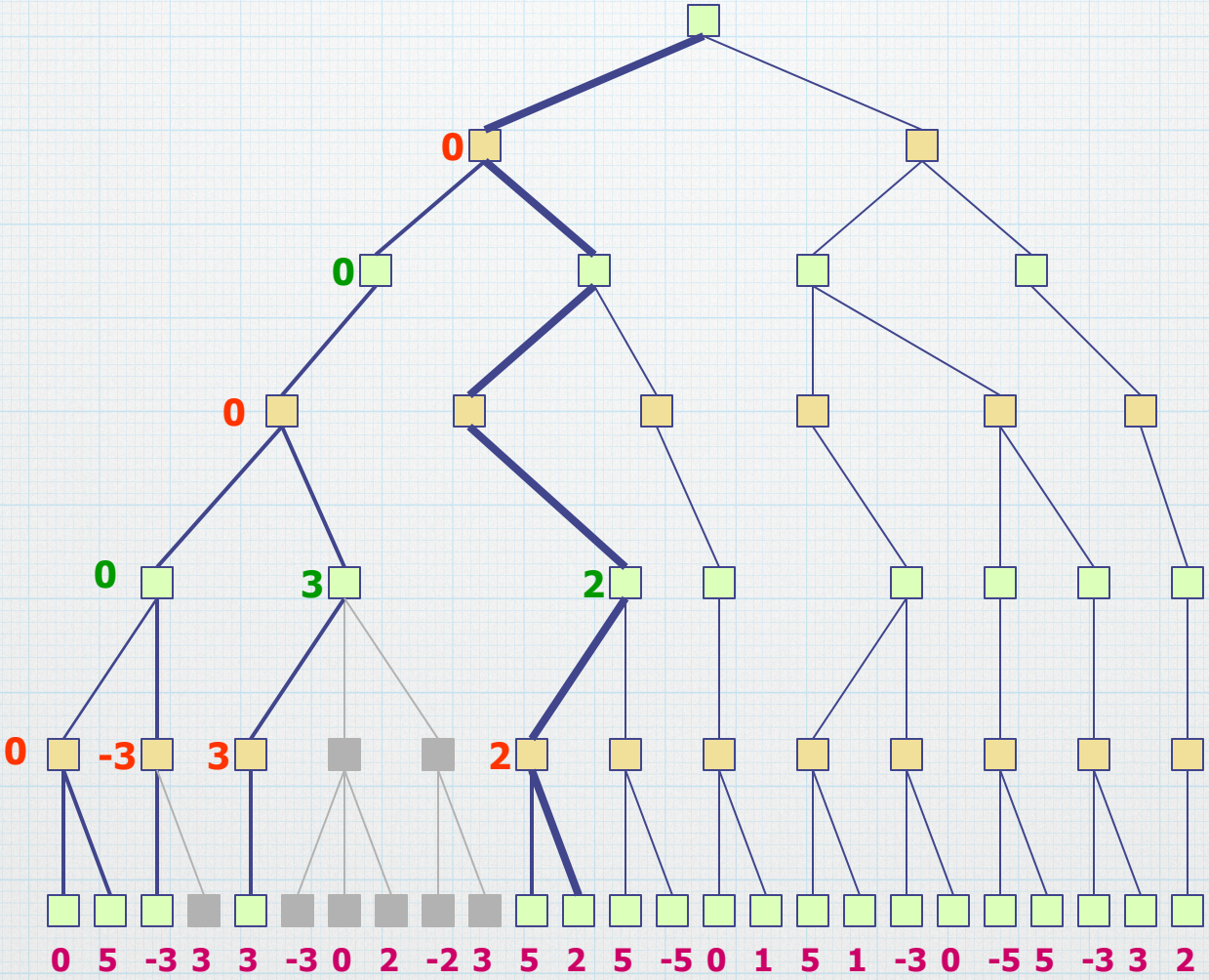
Exemple Alpha-Beta



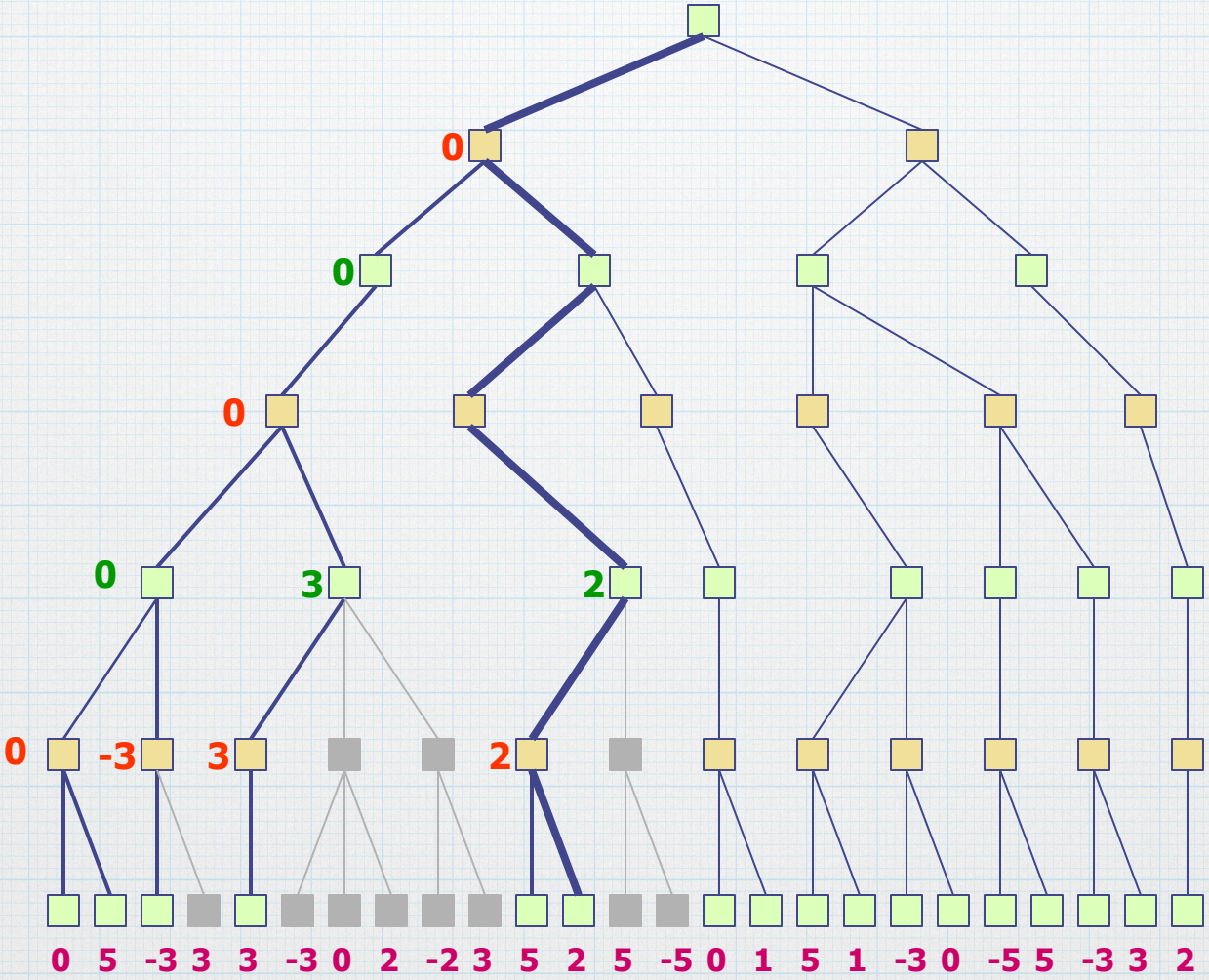
Exemple Alpha-Beta



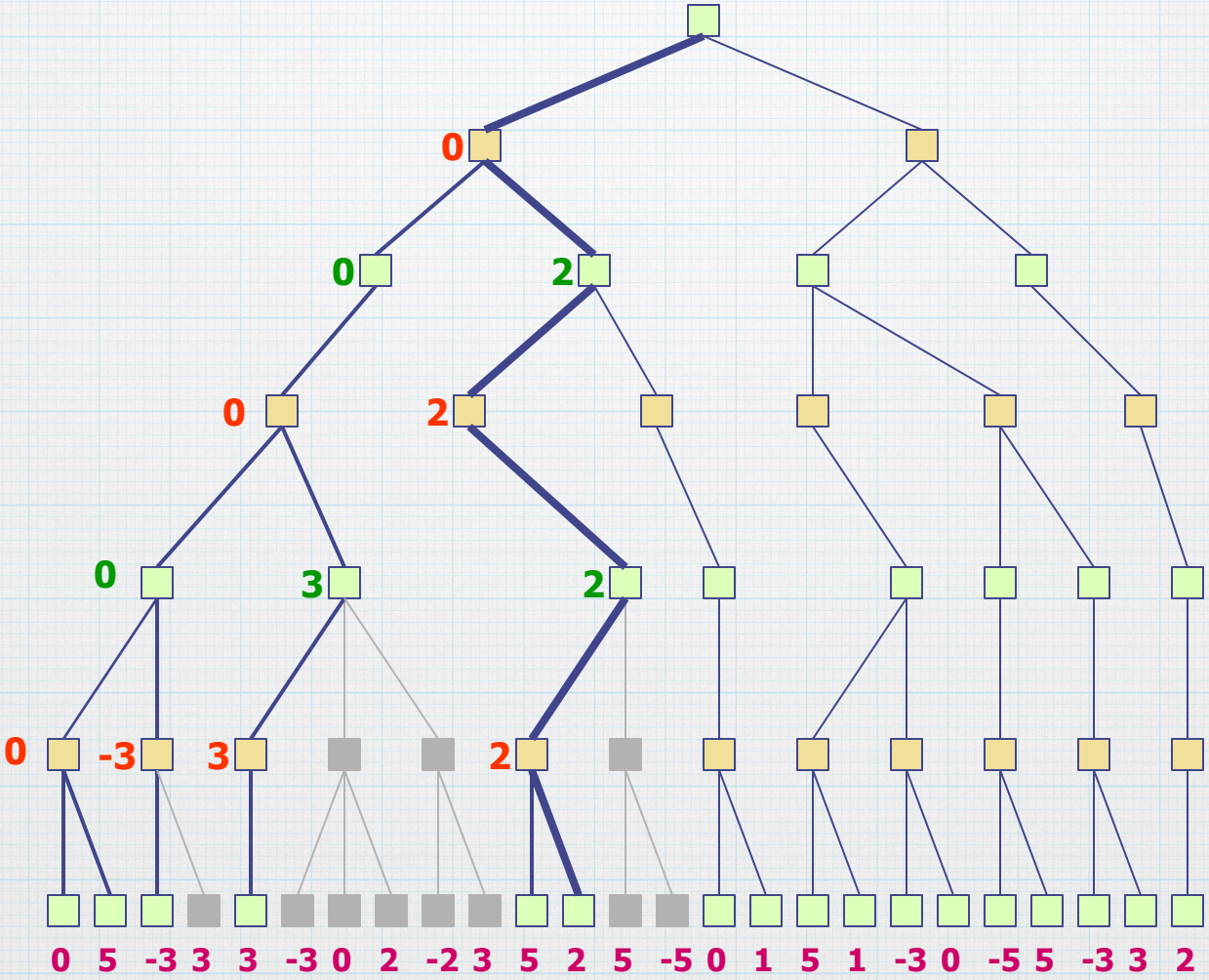
Exemple Alpha-Beta



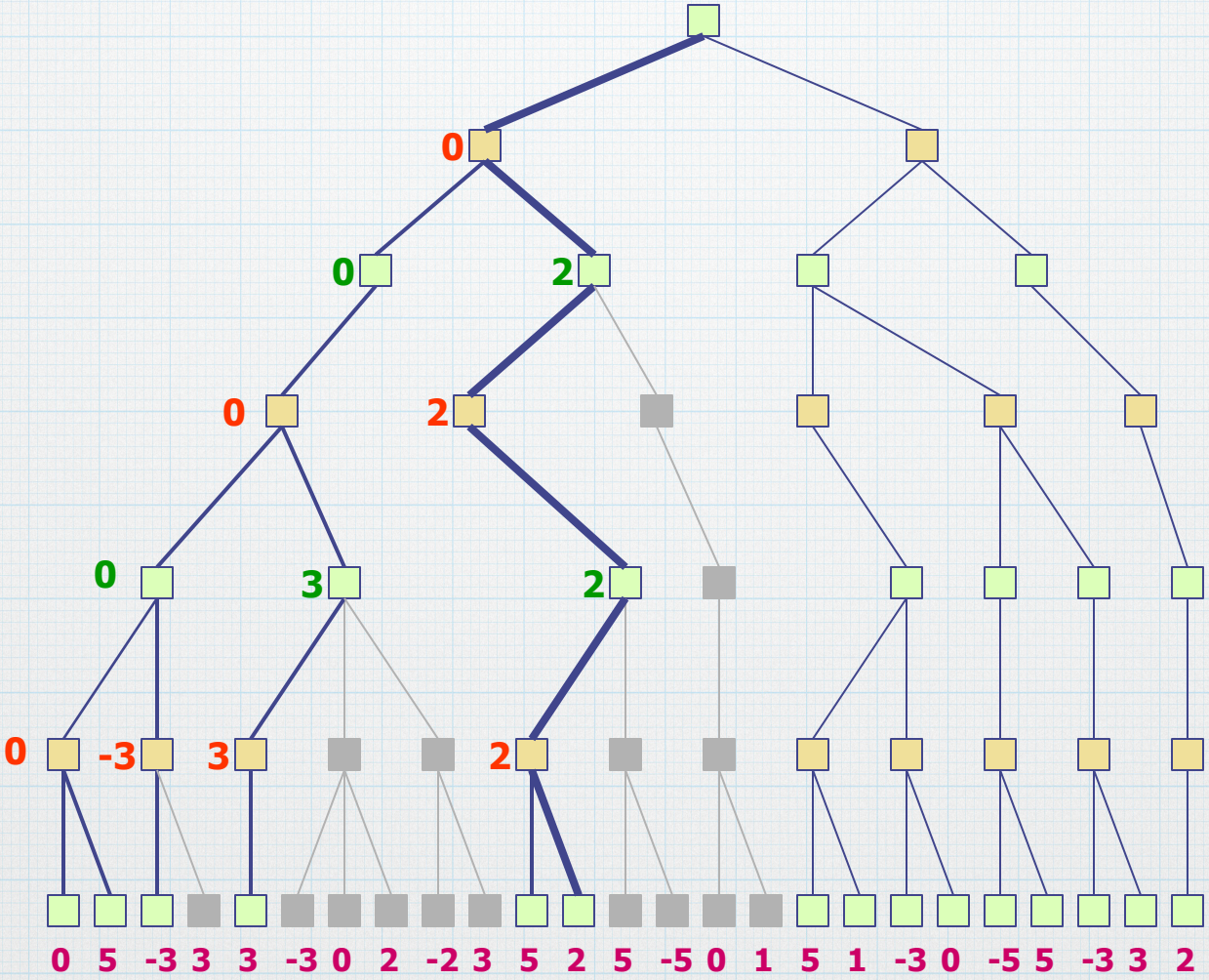
Exemple Alpha-Beta



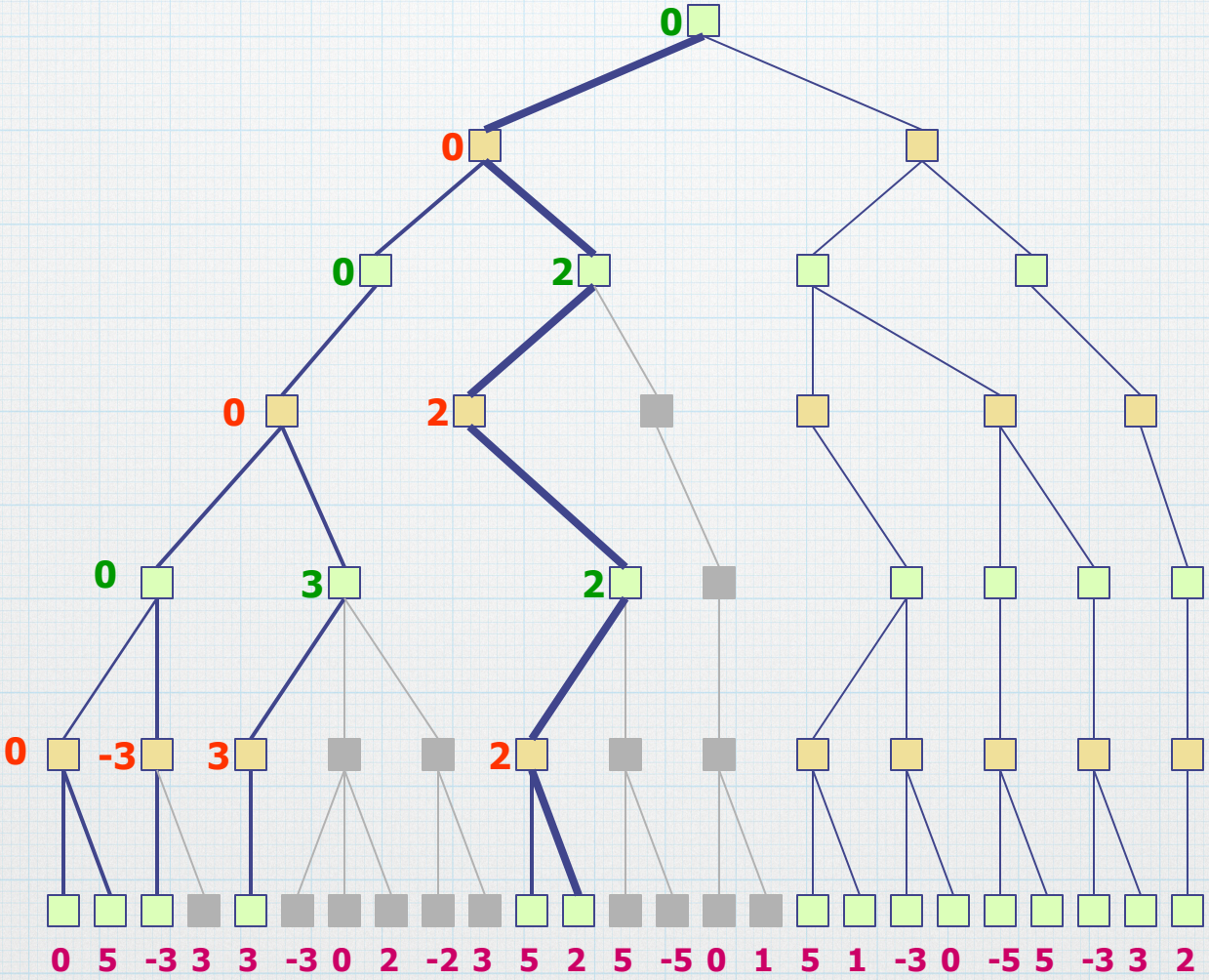
Exemple Alpha-Beta



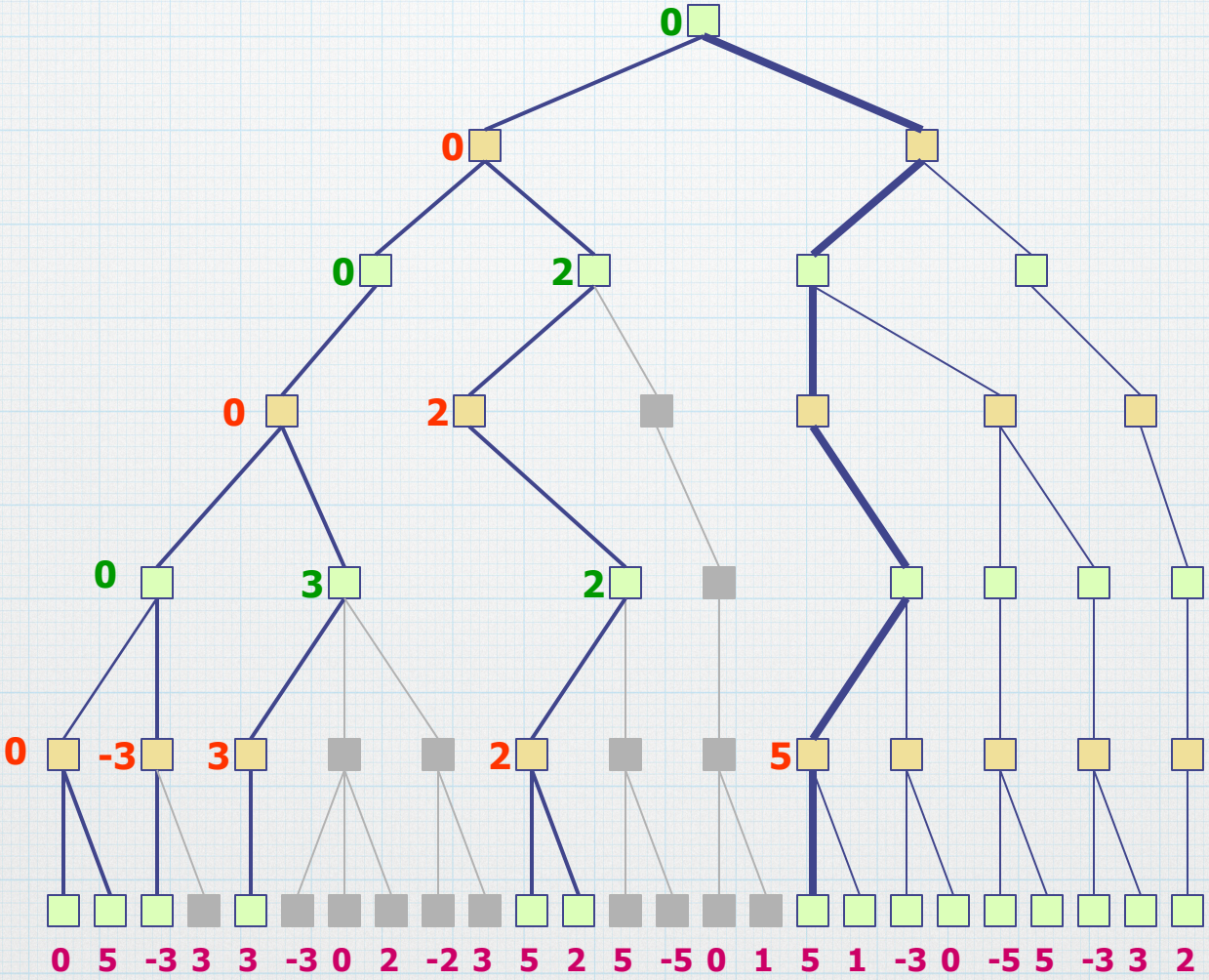
Exemple Alpha-Beta



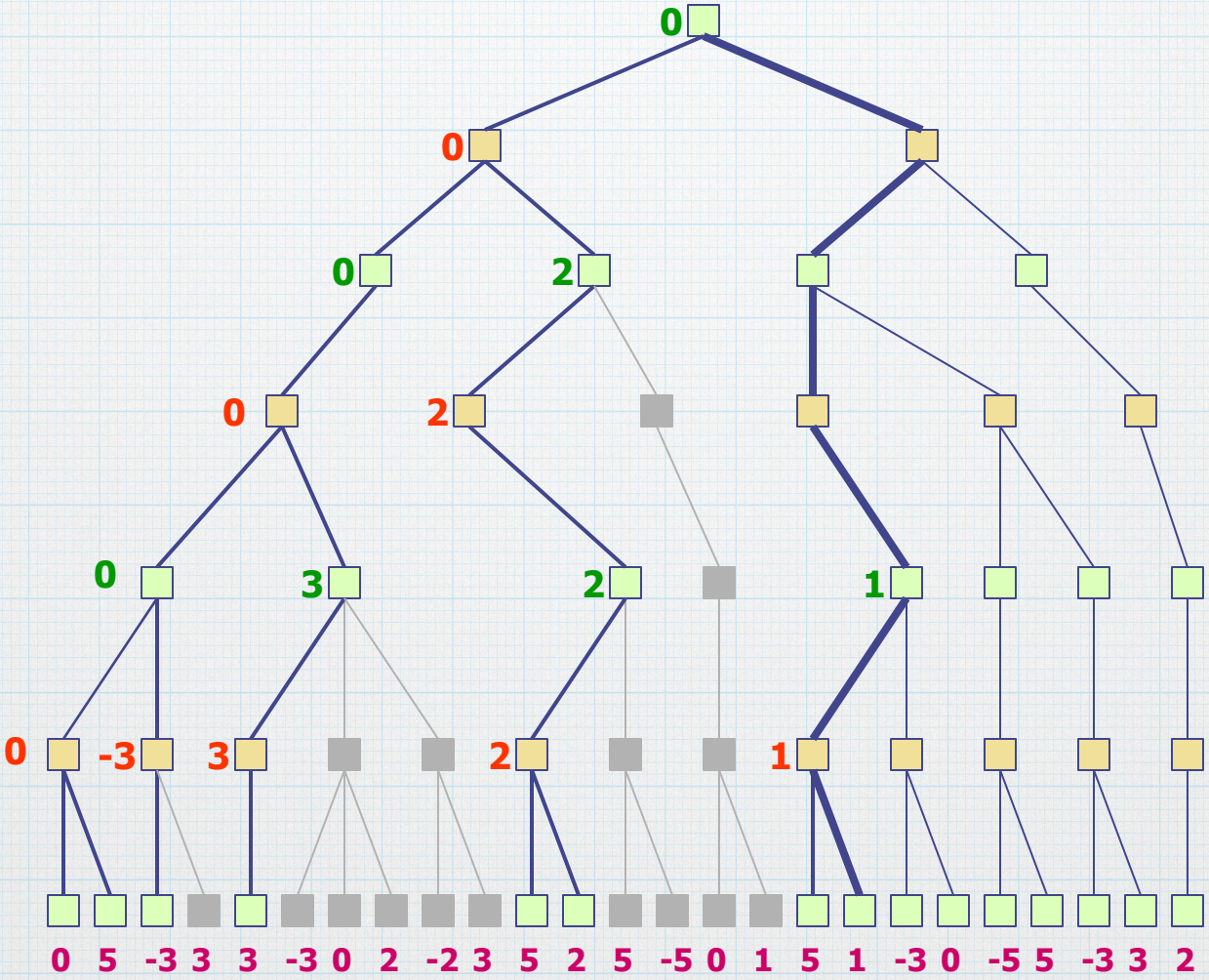
Exemple Alpha-Beta



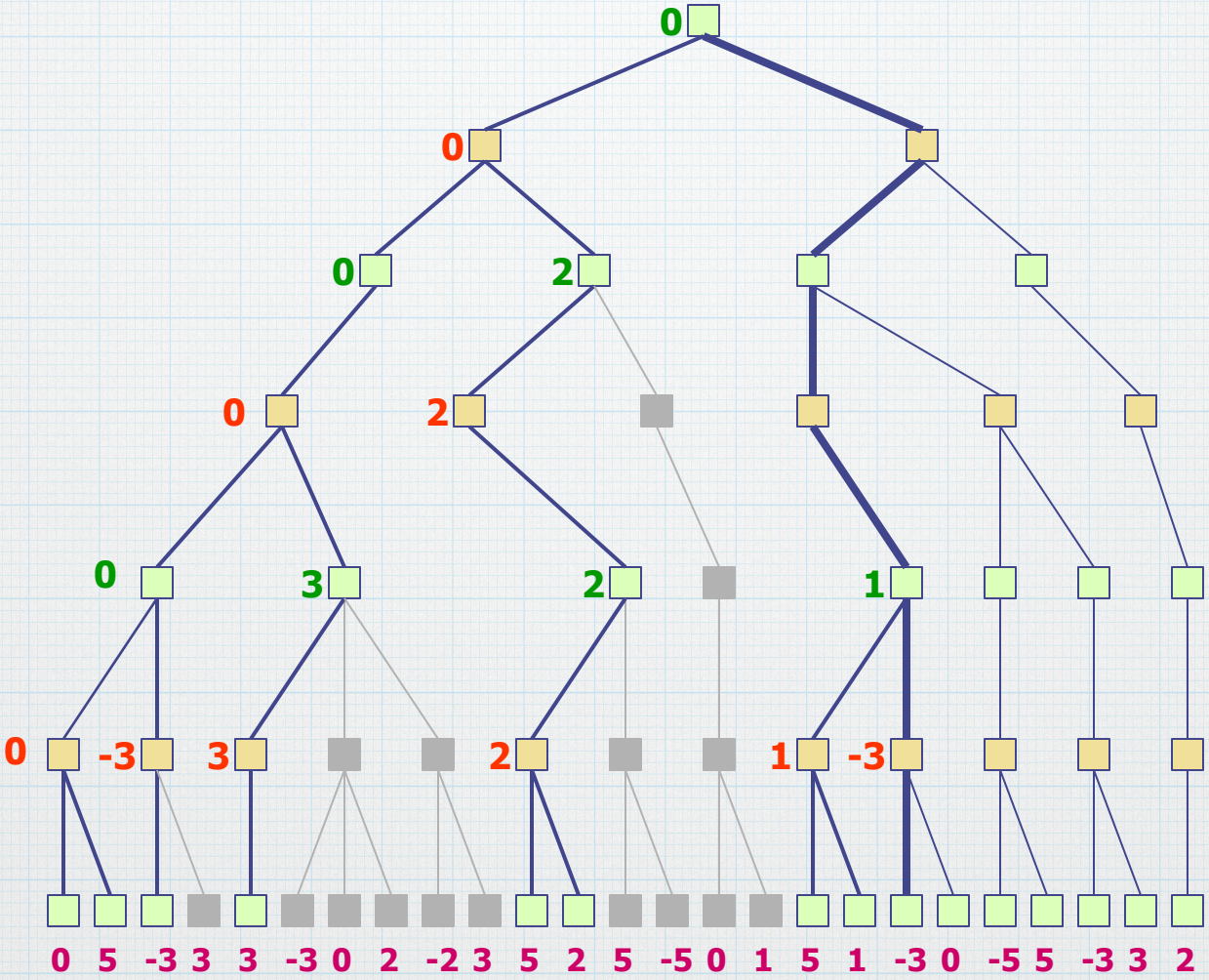
Exemple Alpha-Beta



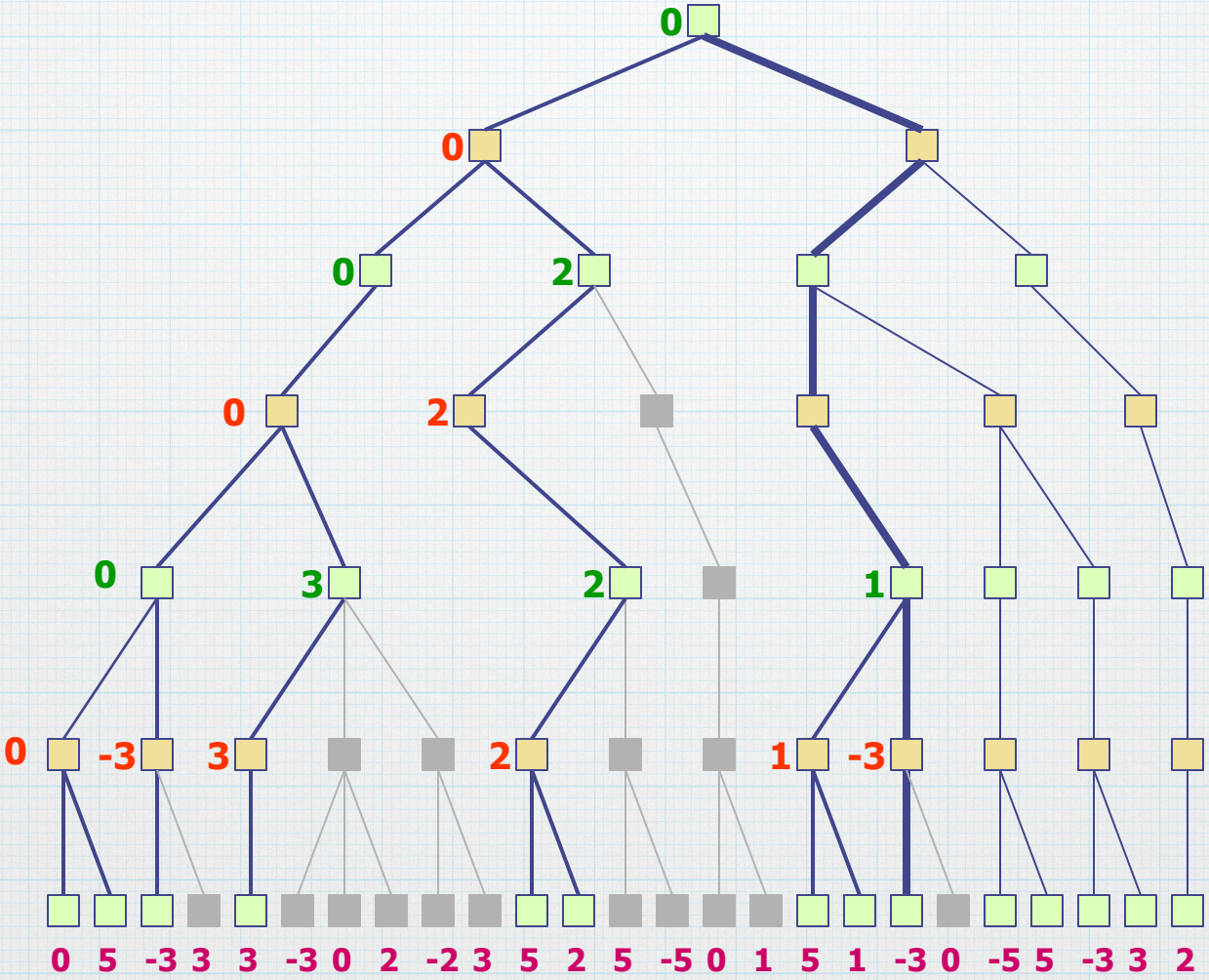
Exemple Alpha-Beta



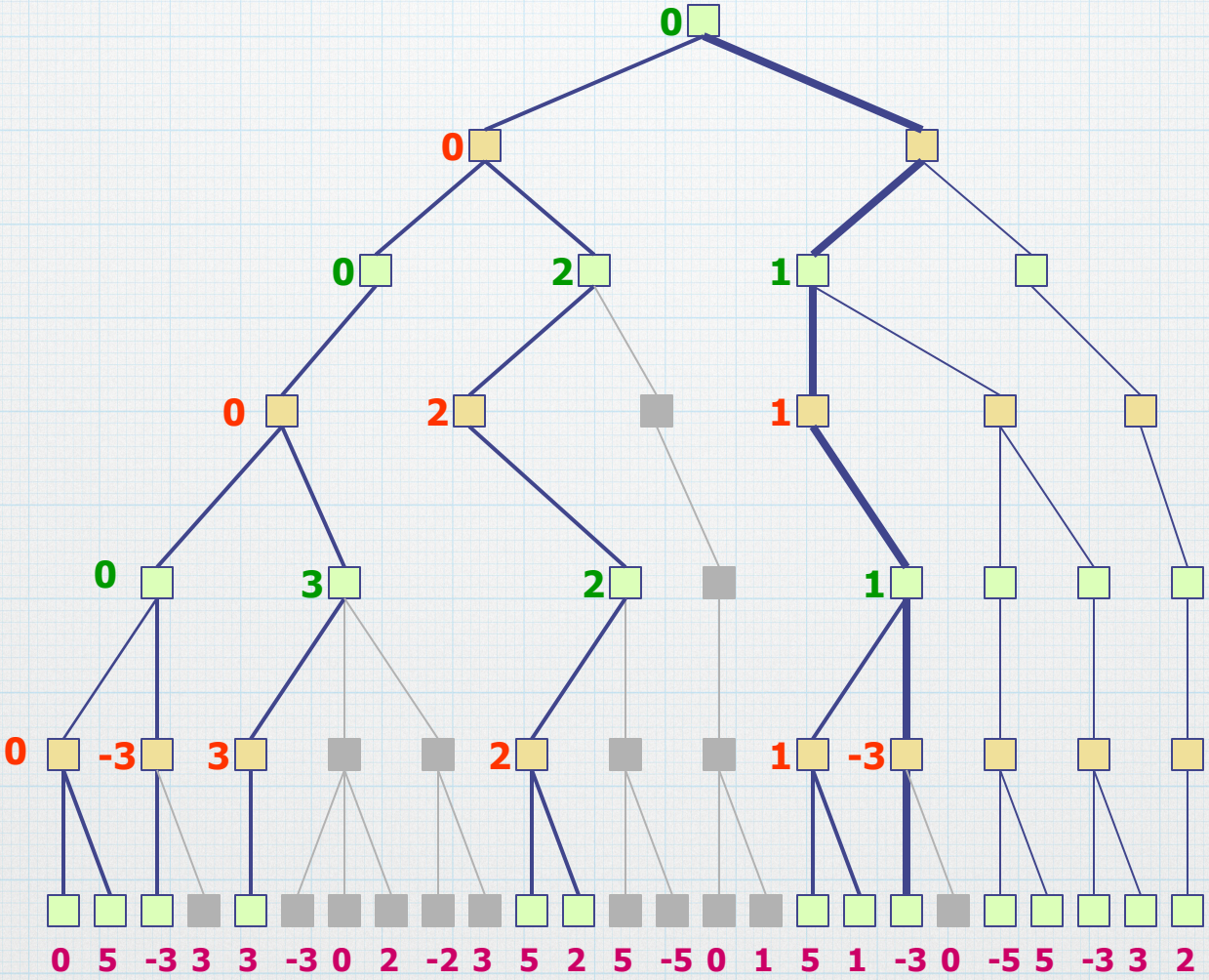
Exemple Alpha-Beta



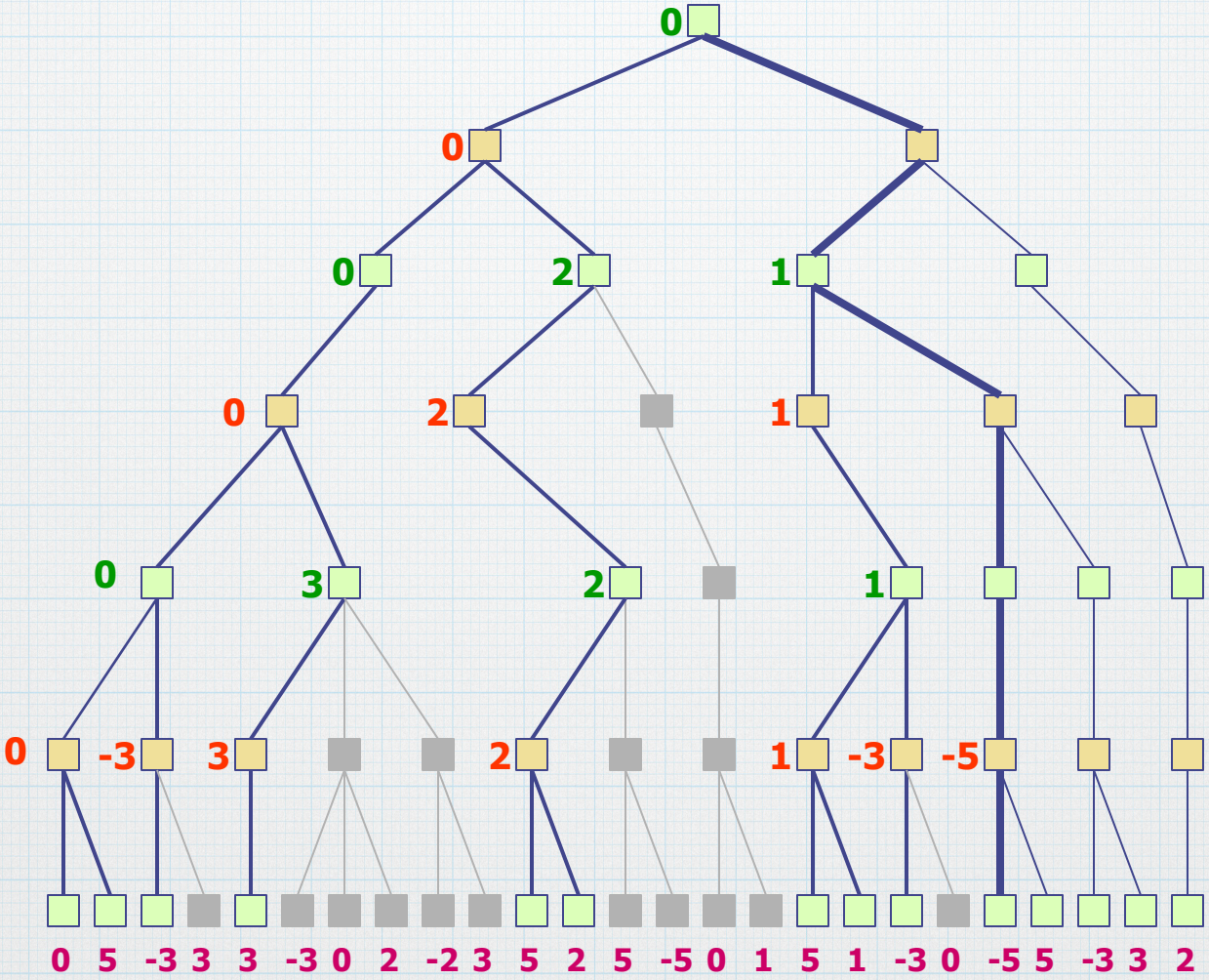
Exemple Alpha-Beta



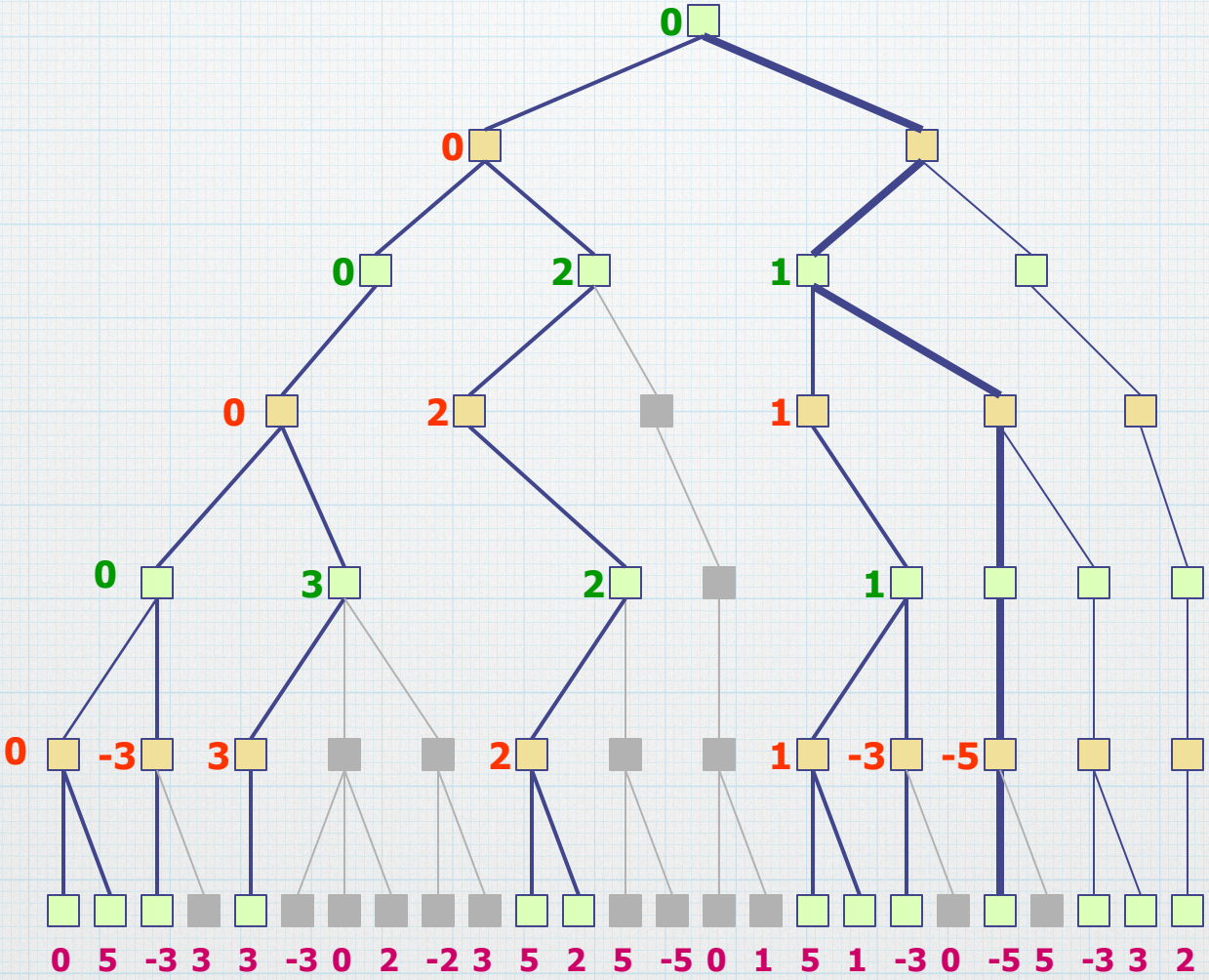
Exemple Alpha-Beta



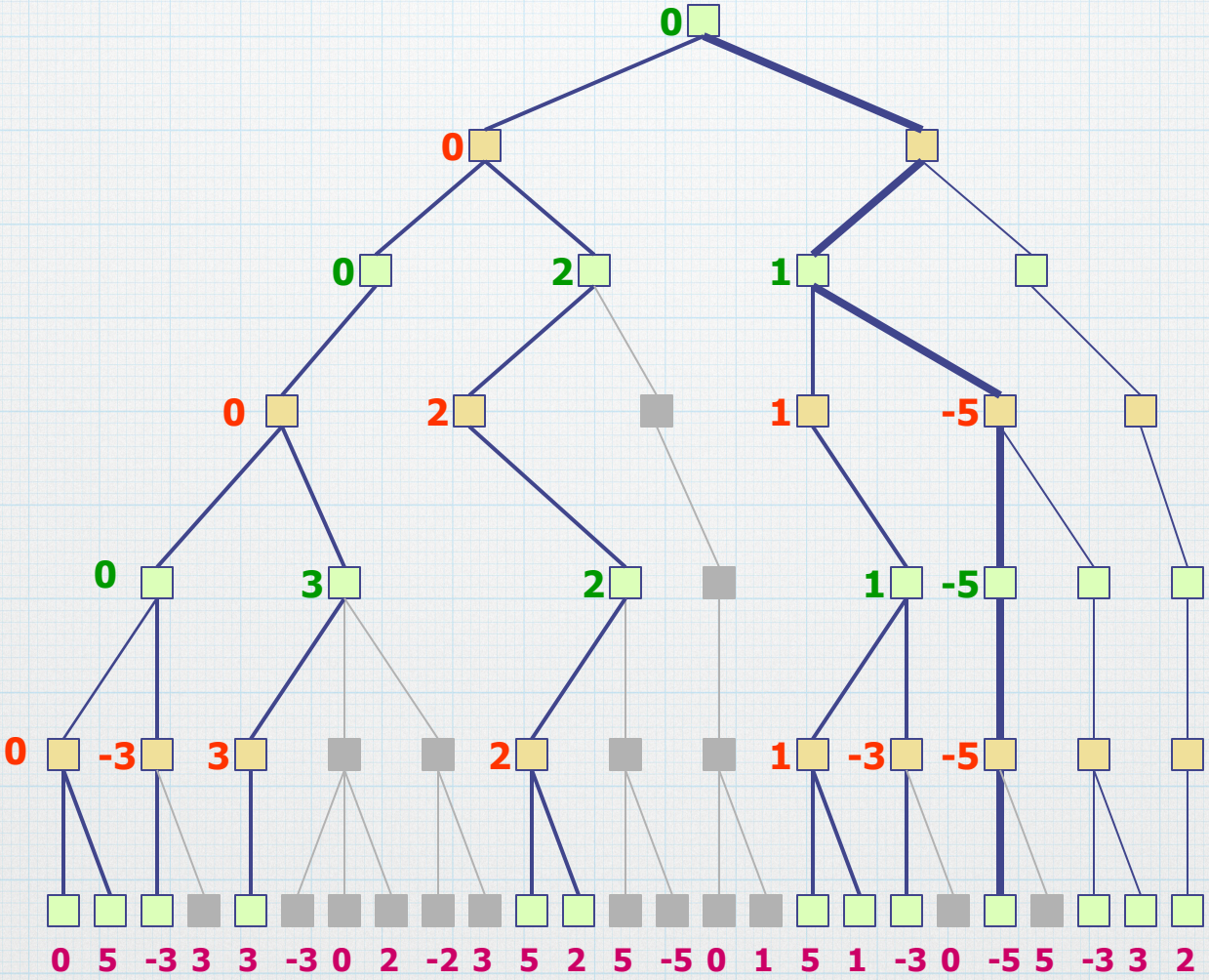
Exemple Alpha-Beta



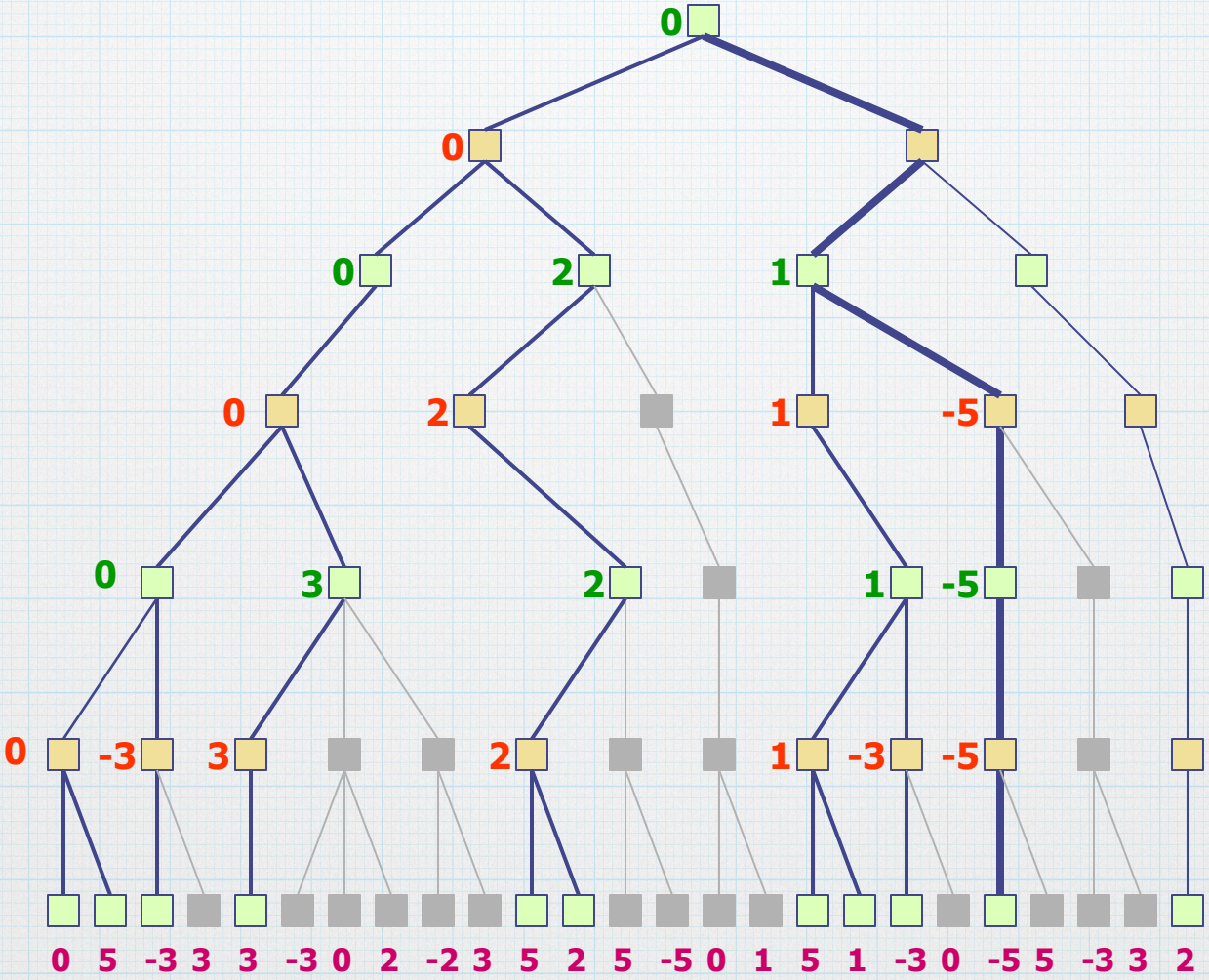
Exemple Alpha-Beta



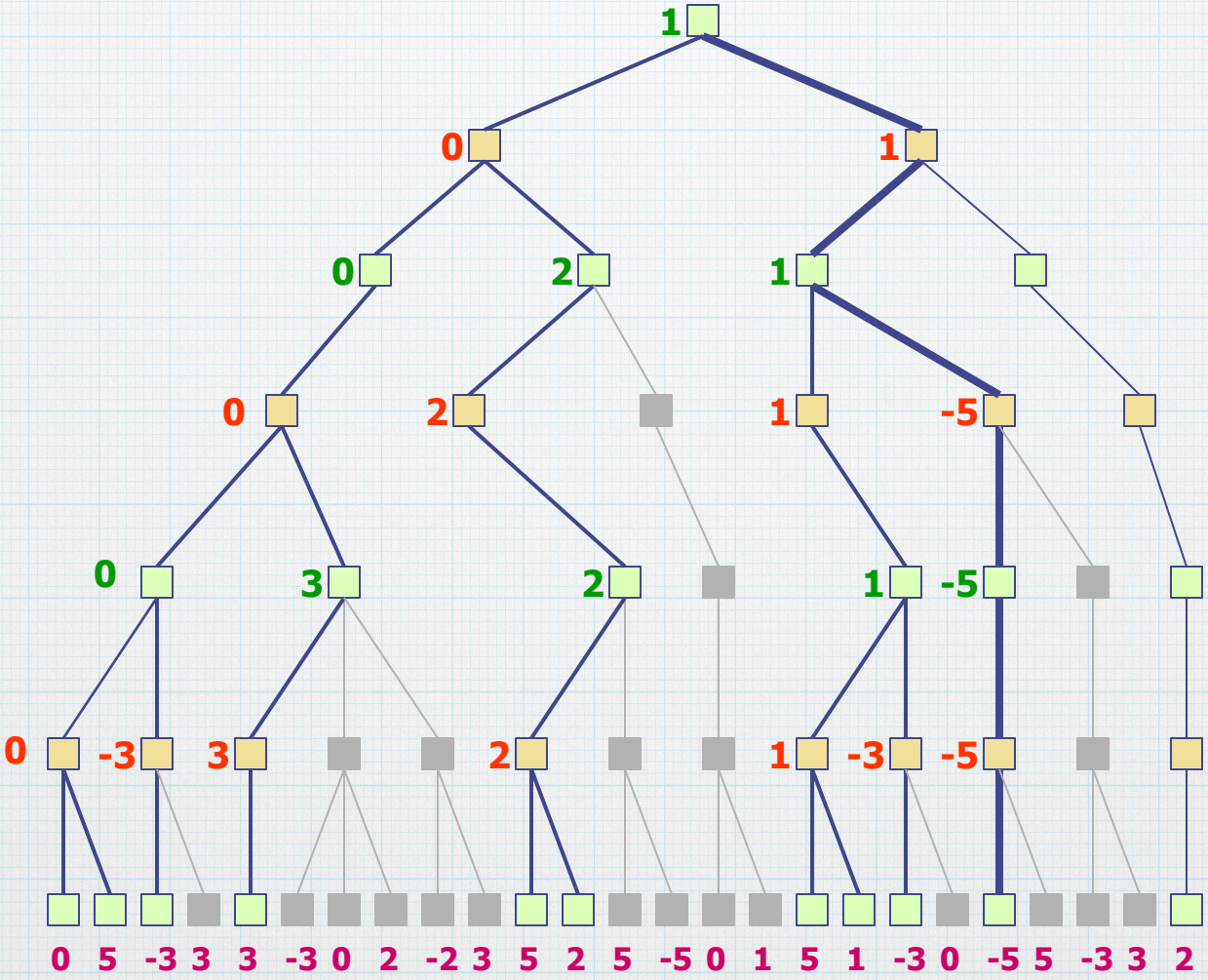
Exemple Alpha-Beta



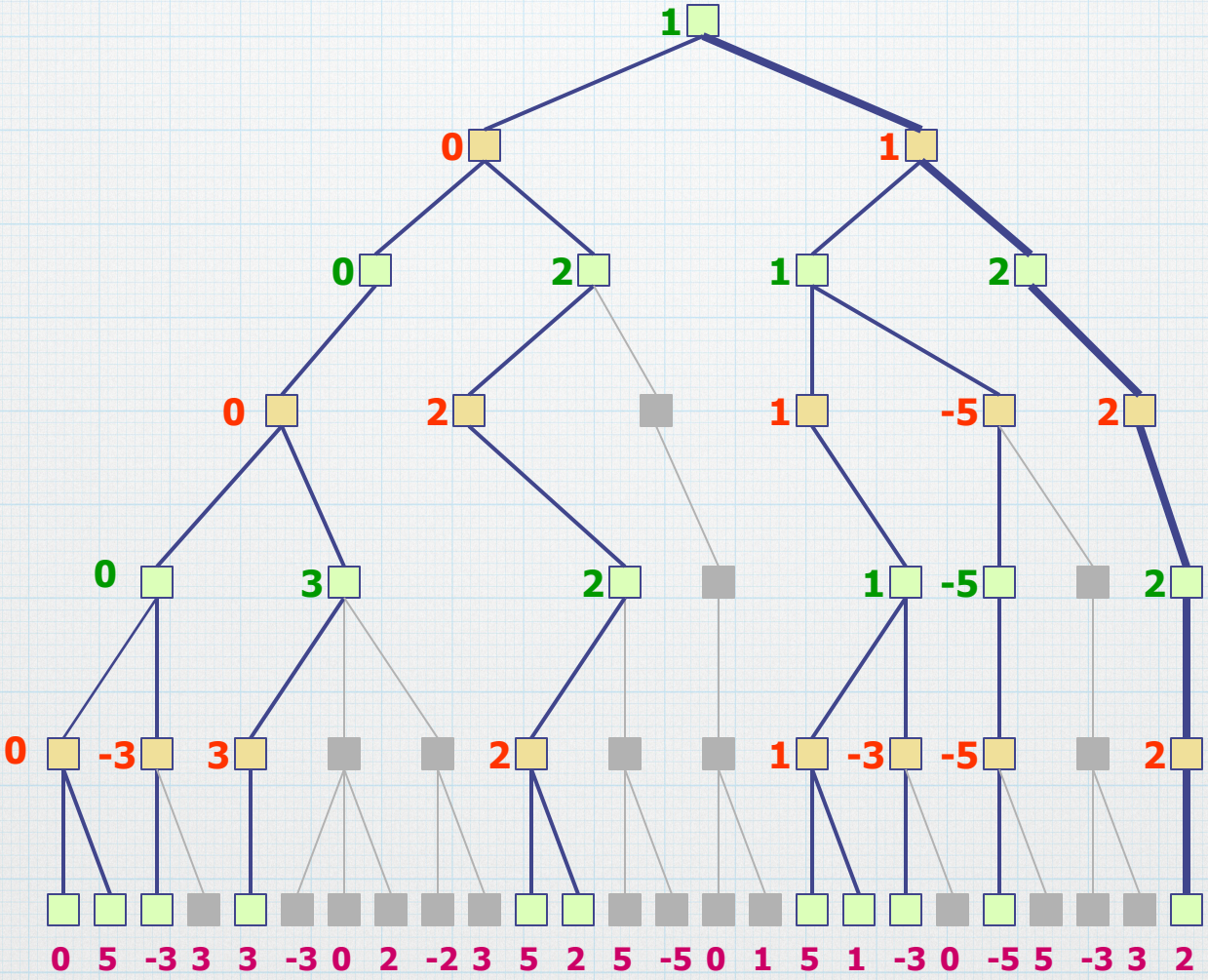
Exemple Alpha-Beta



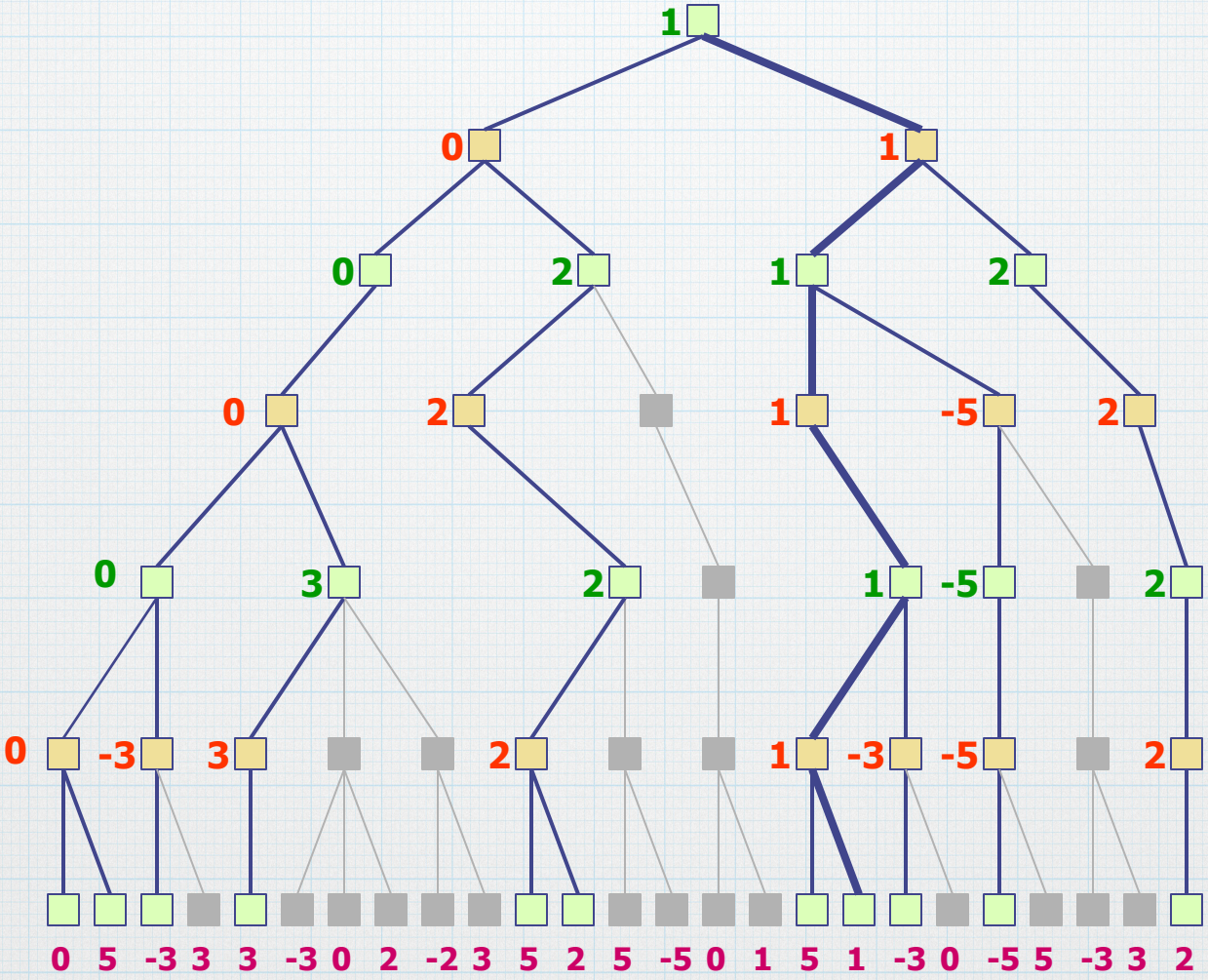
Exemple Alpha-Beta



Exemple Alpha-Beta



Exemple Alpha-Beta

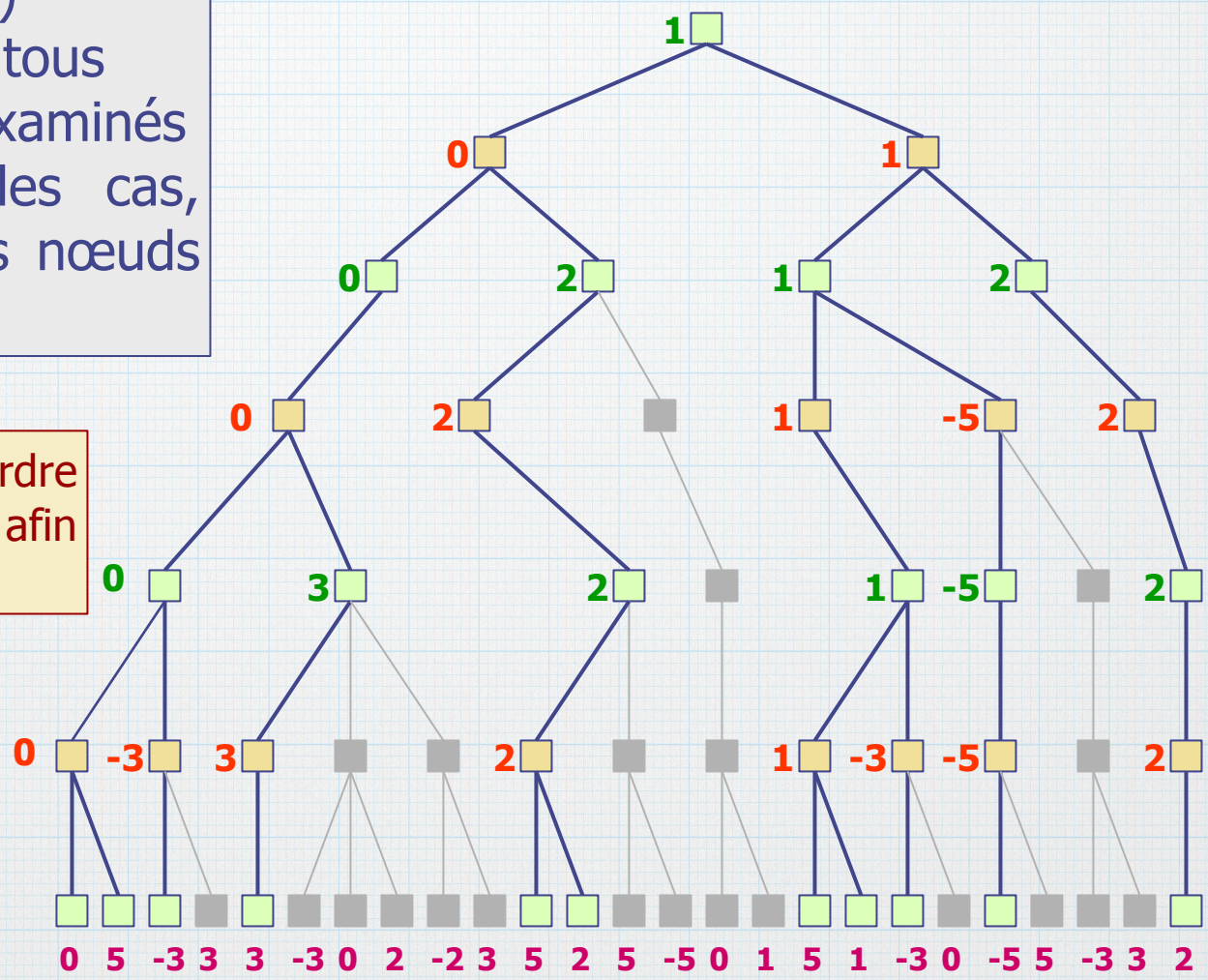


Combien gagnons-nous ?

Taille de l'arbre = $O(b^h)$

- Dans le pire des cas, tous les nœuds vont être examinés
- Dans le meilleur des cas, seulement $O(b^{h/2})$ des nœuds vont être examiné

Exercice : Dans quel ordre doit-on examiné les noeud afin d'atteindre le meilleur gain ?



Conclusion

* La recherche sur les jeux révèlent des aspects fondamentaux applicables à d'autres domaines
* La perfection est inatteignable dans les jeux : il faut approximer
* Alpha-bêta a la même valeur pour la racine de l'arbre de jeu que minimax
* Dans le pire des cas, il se comporte comme minimax (explore tous les nœuds)
* Dans le meilleur cas, il peut résoudre un problème 2 fois plus rapide que minimax